

VECTOR

APL+Win at Naples:

- | | |
|-----------------------------------|-----|
| • Meeting report and pictures | 32 |
| • Eric Lescasse on APL Santa Fe | 41 |
| • Bill Rutiser outlines □MOM | 69 |
| • Bill Rutiser shows :Try :Catch | 79 |
| • Nabavi Extends the reach of APL | 91 |
| • Objects in Dyalog APL (part 2) | 107 |



*The Journal of the
British APL Association*

ISSN 0955-1433

www.vector.org.uk

Vol.22 No.2 March 2006

A specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2005–2006

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: gill@apl385.com.

Contents

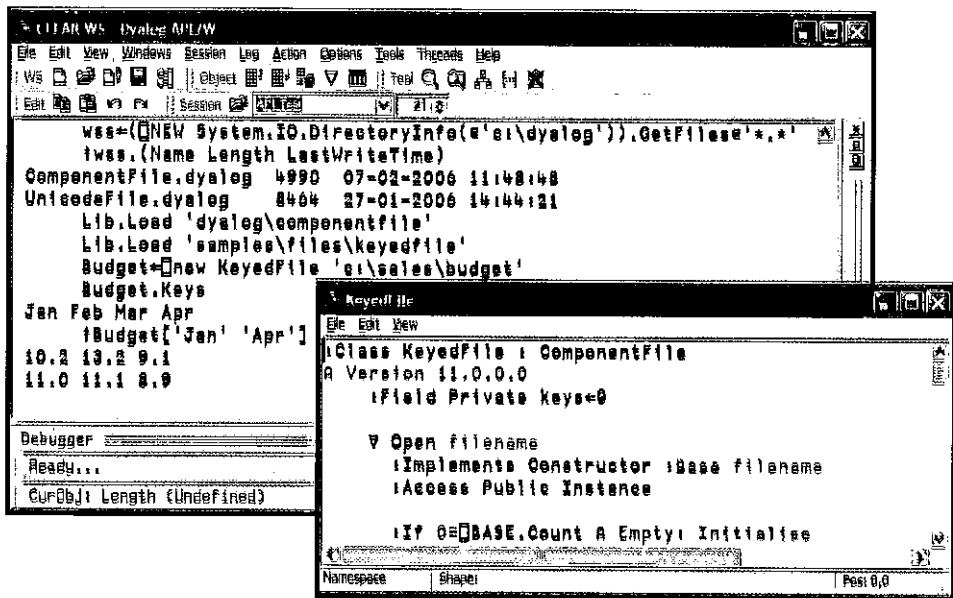
	Page
Editorial: Winter in Naples	3
RESPONSES	
Idioms in I-APL and APL2	Mansour, Quario, Henriod 10
Thoughts on the URN problem	Devon McCormick 14
Sudoku Madness	Neville Holmes 16
NEWS	
British APL Association AGM 2006	19
Product Guide	Gill Smith 20
APL2000 Users Meeting, Naples Beach, November 2005	
Meeting Report	Adrian Smith 32
APL+SantaFe previewed	Eric Lescasse 41
DISCOVER	
G�rard Langlet: a Man of Distinction	Sylvia Camacho 56
LEARN	
Object-Oriented Programming in APL+Win	William Rutiser 69
Static Exception Handling for APL+Win	William Rutiser 79
Up, Down and Sideways: Extending the Reach of APL	Richard Nabavi 91
In Search of a Cool Smooth	Adrian Smith 98
Object-Oriented Programming for Dyalog APL (part II of III)	Morten Kromberg 107
J-ottings 47: RU14 txt msgs? pls 1!:1 on	Norman Thomson 131
PROFIT	
Source Code Management of APL Code	Richard Renich 138
Index to Advertisers	143

DYALOG

The tool of thought for expert programming

Dyalog Version 11.0 is the first commercially available APL system to provide a complete Object Oriented framework for APL. Object Orientation complements the functional and array tools already provided by APL. With version 11.0, you can

- Provide APL components to developers using other languages with the same ease as saving a workspace
- Use COM and Microsoft .Net tools as easily – and using the same syntax – as components written in APL



Since 1983, Dyalog Ltd. has developed a host of powerful language features and an unparalleled ability to interface with other software. This makes Dyalog the most powerful APL-based application development system available in the market today.

In addition to Object Orientation, Version 11.0 adds numerous enhancements to the APL language. 64-bit versions remove workspace limitations under Windows XP/64 and 64-bit Unix systems.

Dyalog APL is available for Windows from 98 to XP/64 and PocketPC, and for Linux and Unix. Version 11.0 will be available in June 2006. For more information, visit www.dyalog.com.

Editorial: Winter in Naples

The annual APL2000 conference in Naples, Florida always produces interesting papers, and this winter has yielded a good crop. Your editor had his lifetime quota of hot weather when living in Australia, but the tireless Adrian Smith was there with golf club, camera and laptop, and filed a report.

The big news in the APL world this year is interpreter extensions to support classes and instances. Organising software into objects is the best programming paradigm devised for representing complex states that must respond to unpredictable events. The paradigm appeared with the graphical user interfaces it made possible. GUIs quickly made themselves indispensable, and object-oriented programming became the standard way to write software. Smalltalk-80, both operating system and programming language for the Xerox Star, demonstrated that all programs can be written as objects; programmers who started work after 1990 could be forgiven for thinking it is the only way.

Modern APLs support GUIs and provide object-oriented interfaces to them. Programmers writing in the APLs have found OOP a useful paradigm for tackling some tasks, and some have devised their own ways to implement the class-instance relationship. But I am fond of quoting the J lab on OOP:

It is a mistake to think of OOP as an alternative to FP (functional programming).

All languages, at the low level, have FP, and at a higher level, have OOP.

I am unsure of the truth of the last statement (is it true about Forth?), but have found it an excellent guide to designing applications.

Even without user-defined classes — arguably the heart of OOP — APL programmers can write about arrays of objects in ways that quicken the pulse. For example, in Dyalog APL, to arrange six button objects, B1 to B6 in a grid anchored at `offset` with `spacing` between rows and columns (`offset` and `spacing` are pairs):

```
(↑(B1 B2 B3)(B4 B5 B6)).Posn←offset+“spacing×”⊔2 3
```

Adding user-defined classes immensely extends this, blending two powerful abstraction paradigms. In tackling this, the interpreter writers have had to address problems largely unknown to OOP; for example, what is the prototype of an object, that enables arrays of objects to be empty or overtaken? Just as when

nested array systems were explored, the consequences of alternative solutions are hard to foresee. Our best wishes go to the implementers at Dyalog and APL2000.

In this issue, Bill Rutiser introduces the MOM object system in APL+Win, and Morten Kromberg continues his 3-part introduction to OOP. Much of the interest in OOP arises from the need to interface to systems through object models. Adrian Smith has done useful work finding out how to develop in APL and deploy in C#. In *Vector* 21.2 Richard Smith described Rowan, an APL-like interpreter constructed from .Net components; here Eric Lescasse previews APL+Santa Fe, a similar project with larger ambitions for commercial use. MicroAPL has long focused on spanning platforms and supporting important interfaces; Richard Nabavi reviews just how far APLX can reach.

APL+Win has recently added try-catch exception handling, and Bill Rutiser explains how it can be used to write more clearly. Clarity is also in Norman Thomson's thoughts, as he uses J to explain internal rate of return calculations in text messages.

The small size of array-programming teams limits our experience of collaboration tools such as libraries and source code managers (SCM). Often we are able to write our own simple tools, well adapted to our lesser need. This was the experience of SoftMed Systems, which is now migrating from its own SCM, written in APL, to a commercial product, Perforce. Richard Renich tells the story.

Lastly, Sylvia Camacho writes about the work of the late Gérard Langlet, some of it published in *Vector*, who used APL to question the mathematics underlying the physics, chemistry and biology of perception.

Stephen Taylor

Quick Reference Diary

Date	Venue	Event
Oct 16 2006	Denmark	Dyalog 2006 Conference and "All APLs Day" at LO-skolen

Dates for Future Issues of VECTOR

	Vol.22 No.3	Vol.22 No.4	Vol.23 No.1
Copy date	in press	in press	10th June
Ad booking	in press	in press	17th June
Ad Copy	in press	25th May	24th June
Distribution	May 2006	June 2006	July 2006

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

RESPONSES

Ajay Askoolum's APL Idioms

from Steve Mansour (steve@carlislegroup.com)

All solutions are written using dynamic functions in Dyalog APL, $\square m1+3$. All are origin independent.

Some notes:

I have two solutions to idiom #1, CE and CE1. CE1 doesn't use the rho symbol or the comma symbol.

$$CE \leftarrow \{ \times / \rho \omega \}$$

$$CE1 \leftarrow \{ + / \epsilon \omega \equiv '' \omega \}$$

I have two solutions to idiom #2, RA and RA1. Both require a 2-item vector on the left as the range. RA requires a scalar right argument. RA1 permits any array as the right argument and doesn't use each. For more details on this solution, please see my article "Arithmetic Boundary Checking" in Vector Vol 21 No. 2. Solution to this particular problem is at the top of page 122.

$$RA \leftarrow \{ + / \times \omega - \alpha \}$$

$$RA1 \leftarrow \{ + / \times \omega \circ . - \alpha \}$$

Idiom #3:

$$SN \leftarrow \{ \omega [\uparrow 10 \oplus | \omega ;] \}$$

Idiom #4:

$$LD \leftarrow \{ \{ \omega \} \neq \omega \}$$

Idiom #5: Curried left argument can be any suitable value > 0 . Domain error may result if the base of the exponential function is not chosen correctly.

$$LS \leftarrow 0.9 \circ \{ \alpha \otimes x \neq \alpha * \omega \}$$

Idiom #6: Note - $\square D$ is equivalent to '0123456789':

$$CN \leftarrow \{ 10 \perp - \square IO - \square D \iota \omega \}$$

Idiom #7:

$$ZM \leftarrow \{ 0 + / \omega \}$$

Idiom #8:

$$FO \leftarrow \{ 2 < / 0, \omega \}$$

Idiom #9:

$$LO \leftarrow \{ 2 > / \omega, 0 \}$$

Idiom #10:

$$RE \leftarrow \{ \uparrow'' \downarrow \circ \alpha'' \downarrow \omega - 1 \}$$

More Solutions ...

from Gianluigi Quario (gianguiquario@hotmail.com)

1. How many elements does a given variable have?

Restriction: assume that the keyboard does not have the comma (,) symbol.

$$CE \leftarrow x / \circ \rho \quad A \quad \omega \leftarrow A$$

2. Is a value within a given range?

Restriction: assume that the keyboard does not have any relational operators.

$$C2 \leftarrow \{ + / x \omega [\downarrow \omega] - \alpha \} \quad A \quad \alpha \leftarrow D0 \quad \omega \leftarrow D1 \quad \rho \omega \leftrightarrow 2$$

3. Sort a numeric array of integers in ascending order of the number of digits.

Restriction: assume that the keyboard does not have the format (π) or quad ($\square$) symbol.

SN+{ ω [4[10 ω]($\omega+2\times 0=\omega$)+ $\times\omega$;]} A $\omega+I2$ (1 $\downarrow\rho\omega$) $\leftrightarrow 1$

4. Return the element(s) of a numeric array indexed by its first dimension.

Restriction: assume that the keyboard does not have the shape of (ρ) symbol.

NB.(GQR) the definition of this problem is not complete:

- a) should return an array with a lesser rank ?
- b) a scalar has no "first dimension", but it is required by the challenge:

LD 6 $\leftrightarrow 6$

LD+{, [1, (0 $\leq$ +/-0 $\times\omega$)/2]1 $\uparrow$ [1], [(0 $\leq\omega$)/1] $\leq$ [0] ω }
A $\omega+D$ (also scalar)

or

LD_+{, [1, (0 $\leq$ +/-0 $\times\omega$)/2]1 $\uparrow$ [1] ω 1 $\uparrow$ [1], [1 2] ω , [0]0, ω }
A $\omega+D$ (also scalar)

or

LD__+{1 $\uparrow$ [1] ω } A $\omega+D$; ω not scalar;
NB.result has same rank of ω

5. Return the sum of element(s) of a numeric array on its first dimension.

Restriction: assume that the keyboard does not have the plus (+) symbol.

NB.(GQR) a scalar has not a "first dimension", but it is required by the challenge:

LS 6 $\leftrightarrow 6$

LS+{ $\square$ ML+2 $\diamond$ t''p'' ϵ '' $\leq$ [(~0 $\in\rho\rho\omega$)/1] $\omega\rho$ ''1}
AAPL2 A $\omega+D$ (also scalar)

6. Convert the string representation of integers to numbers.

Restriction: assume that the keyboard does not have the execute ($\pm$) symbol.

$CN \leftarrow \{10 \downarrow 2 \downarrow 4 + [\square IO] \& 11 \square DR, [\theta] \omega\} \text{ A } \omega \leftarrow C1 \text{ (also scalar)}$

$CN_ \leftarrow \{10 \downarrow 2 \downarrow 4 + \text{'11' } \square DR \text{' } \omega\} \text{ A } \omega \leftarrow C1 \text{ (also scalar)}$

$CN_ \leftarrow \{10 \downarrow (-\square AV \downarrow 0') + \square AV \downarrow \omega\} \text{ A } \omega \leftarrow C1 \text{ (also scalar)}$

$CN_ \leftarrow \{10 \downarrow (-\square IO) + '0123456789' \downarrow \omega\} \text{ A } \omega \leftarrow C1 \text{ (also scalar)}$

7. Return a numeric array as zeros, increasing the last dimension by 1

Restriction: assume that the keyboard does not have the times ($\times$), take (?), minus ($-$), or comma ($,$) symbols and it does not end with $\rho 0$.

$ZM \leftarrow \{(((\neg 1 + \rho \rho \omega) \rho 0), 1) + \rho \omega\} \rho \omega = \omega + 1 \text{ A } \omega \leftarrow D$

8. Return the first ones from a Boolean vector.

Restriction: assume that the keyboard does not have the not ($\neg$) or rotate (ϕ) symbols.

$FO \leftarrow \{\omega^{\wedge} 1, (1 \downarrow \omega) \neq \neg 1 \downarrow \omega\} \text{ A } \omega \leftarrow B1$

9. Return the last ones from a Boolean vector.

Restriction: assume that the keyboard does not have the not ($\neg$) or rotate (ϕ) symbols.

$LO \leftarrow \{\omega^{\wedge} ((1 \downarrow \omega) \neq \neg 1 \downarrow \omega), 1\} \text{ A } \omega \leftarrow B1$

10. Return the elements of a numeric array found at given coordinates.

Restrictions: assume that a looping solution is not allowed nor one involving semi-colon and that index origin is 1.

$RE \leftarrow \{(\alpha) [1 + (\rho \alpha) \downarrow \phi \omega - 1]\} \text{ A } \alpha \leftarrow D2 ; \omega \leftarrow D 1$

More Solutions ...

from Claude Henriod (chenriod@vtx.ch)

I present some solutions to the problems posed by Ajay Askoolum in *APL Idioms*, Vector 21:4, pp92-96.

The development is for a great part from my libraries originated in the 198x years on :

/APLSV IBM5110/APL68000 Wicat&Ampère/after/IBMAPL2 PS+PC/IBMwinAPL2/

All are shown as DDF valid I-APL. Tests are made in I-APL to be sure that the code is OK.

But the function code is cloned in APL2 Unicode, for a better understanding.

```

)LOAD UOL2104
UOL2104

)FNS
CN: FO: LIM: LO: LOOPLIM LS: RE: SN:

    LIM:
LIM: +/x(2ra)j.-w
    SN:
SN: (rw)r(,w)[qIX+~xIX_p101LG+~xLG_1,w]
    LS:
LS: ~cwxS(frW)r(1t(rw),1)r1`1
    CN:
CN: ((rw)r10)uw_'0123456789'iw : #IO_0 : 0
    FO:
FO: w>`1k0,w
    LO:
LO: w>1kw,0
    RE:
RE: (,a)[#IO+RuW-#IO] : v/0=(R_ra),rw_sw : i0

```

= Requested functions APL2, valid with several interpreters.

```

LIM: +/\(2\r\alpha)\circ.-\omega
SN: (p\omega)\rho(,\omega)[\&IX+\sim\times IX+\lceil 10\circ LG+\sim\times LG\sim\rceil,\omega]
LS: -/\omega\times\phi(\phi\rho\omega)\rho(1\uparrow(p\omega),1)p1\cdot 1
CN: ((p\omega)\rho10\downarrow\omega\leftarrow'0123456789'\downarrow\omega : \square IO+0 : 0
FO: \omega>\cdot 1\downarrow 0,\omega
LO: \omega>1\downarrow\omega,0
RE: (,\alpha)[\square IO+R\downarrow\omega-\square IO] : \vee/0=(R\leftarrow\rho\alpha),\rho\omega\leftarrow\phi\omega : \downarrow 0

```

= Functions which do not match prerequisite

```

CE: p1/\omega           A Answer fail
LD: \cdot 1\uparrow[1\uparrow(\downarrow\rho\rho\omega),1)\omega  A p is forbidden
ZM: \sim\omega+\omega\leftarrow\omega,1      A , is forbidden

```

```

" 2. Is a value within a given range ?
ULIM
5 10 15 20 25
CLIM
10 20
CLIM LOOPLIM ULIM
2 1 0 `1 `2
15 LOOPLIM ULIM
2 2 0 `2 `2

" 3. Sort a numeric array of integers in ascending order . . .
MSN1,SN MSN1
89 1
`78 89
1229 `78
32 32
129 11
11 129
90232 1229
1 90232
MSN2,SN MSN2
1334 0
25 25
77 77
0 1334
91234 91234

```

```

" 5. Return the sum of element(s) . . . on its first dimension
(hLSD1), ' = ', hLS LSD1
6 = 6
(hLSD2), ' = ', hLS LSD2
98 878 332 2.3 44 = 1354.3
    LSD3
9    3 4
0.98 22 3.4
    LS LSD3
9.98 25 7.4
    LSD4
1 2 3 4
5 6 7 8
9 10 11 12

13 14 15 16
17 18 19 20
21 22 23 24
    LS LSD4
14 16 18 20
22 24 26 28
30 32 34 36
    LSD5_3 3 3r5
    LS LSD5
15 15 15
15 15 15
15 15 15

```

```

" 6. Convert char-num integers to numbers
10 x #_ CN '898'
898
8980
    10 x #_ CN '707'
707
7070
    10 x #_ CN '34'
34
340

```

```

" 8. Return the first ones from a boolean vector
FO #_ 21 #_OF1
88 68 45 67 77 90 100 13 27 0
0 0 1 1 1 0 0 1 1 0
0 0 1 0 0 0 0 1 0 0
    FO #_OF2
1 0 1 1 1 0 1 1 0 1
1 0 1 0 0 0 1 0 0 1

```

```

" 9. Return the last ones from a boolean vector
LO #_ 21 #_OL1
88 68 45 67 77 90 100 13 27 0
0 0 1 1 1 0 0 1 1 0
0 0 0 0 1 0 0 0 1 0
    LO #_OL2
1 0 1 1 1 0 1 1 0 1
1 0 0 0 1 0 0 1 0 1

```

```

" 10. Return the elements found at given coordinates
  RA
78 90 22 2.3
43.9 92 12 67
23 33 88 9.34
  RB
1 3
2 4
  RA RE RB
22 67
  #RL_ 7x5
  #_ RC_ 2 5 6r60?1000
930 885 814 346 1000 850
193 212 651 470 522 215
178 929 543 316 433 635
35 491 733 111 315 529
495 175 422 643 466 412

554 188 500 230 340 684
669 923 966 872 937 839
381 822 570 255 395 65
560 66 324 296 834 347
462 700 133 59 662 683
  RD1
1 3 4
  RD2
1 5 5
  RD3
2 1 6
  #_RD4_3 3 r RD1, RD2, RD3
1 3 4
1 5 5
2 1 6
  RC RE RD4
316 466 684

```

C. Henriod, retired IBM Advisory SE, chenriod@vtx.ch
 December 7, 2005.

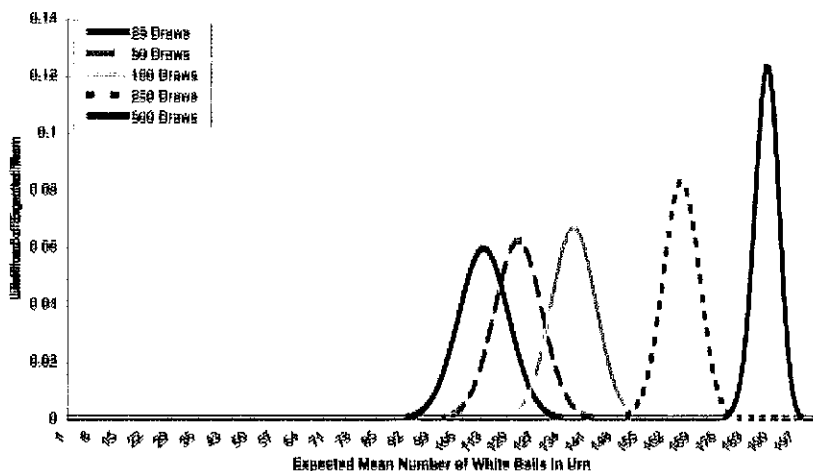
Reply to "Thoughts on the URN problem"

from Devon McCormick

In his response to my article on Bayesian statistics as used in dynamic linear modeling (in Vector Vol. 21 No. 2), E. A. Clough raises an interesting point about prior probabilities as they relate to "the urn problem". This problem is to estimate, given an urn known to contain only black and white balls, the probability that all of the balls are white after a number of white and only white balls have been drawn, with replacement.

In the original article, I did not delve deeply into this problem as it was mostly intended to introduce the notion of probability as a distribution in contrast to the naïve notion of probability as a point estimate. The ubiquity of this naïve notion is evident by how frequently we see probability estimates quoted without any mention of a measure of dispersion. A strength of the dynamic linear models that were the focus of the article is that they simultaneously estimate both means and variances, so it's important to understand probabilities as distributions. I attempted to give readers a picture of this by means of the simple urn example. The follow-up to the simple example with 5 balls in the urn is one with 200 balls in order to give smoother curves when graphed, as we see below.

Effect of New Evidence on Probability Distribution from "Urn Problem"



This illustrates how we see probabilities as distributions, not point estimates. It also shows how the shape and location of the curve changes with additional evidence.

Inasmuch as this was my aim, I paid scant attention to the notion of a “prior probability” and how we might derive one, though this is an important concept in Bayesian statistics. Much of Clough’s response to the article focuses on deriving the prior probability, or “prior”, assumed in my example. As he correctly derives, my initial prior for the distribution of white balls in the urn is 50%.

So, up to a certain point, we are in agreement. However, beyond this point, where we get to the question of how to establish a prior, Clough misrepresents the valid criteria for settling on one. Apparently the long-standing slur on Bayesian methods as “subjective” continues to distort the discussion. This is evident in the assertion that begins “In general a person’s choice will depend on whether they are optimistic ... or pessimistic....”

This is completely unwarranted. My initial assumption of a distribution of 50% white balls is based solely on the only piece of evidence I have: there are only two colors of balls. Given this, absent any other evidence, the best prior assumption is that the distribution is evenly divided between black and white.

There is a theory, called “maximum entropy”, behind this sort of simple assumption. I refer anyone interested in further pursuing this or any other aspect of Bayesian statistics to the excellent book by E. T. Jaynes “Probability Theory – The Logic of Science” (Cambridge University Press, 2003). Jaynes’s work also used to be available on the Web though I have not looked for it recently.

Flawed Assumptions not Unique to Bayesian Statistics

Mr. Clough makes a good point that priors are important in Bayesian methods and that a poor choice of prior tends to undermine subsequent estimates. However, conventional statistics, like any chain of reasoning, is subject to the same sort of problem. Consider an article published in the New Scientist magazine (3 September 2005, www.newscientist.com/channel/life/mg18725155.400) on a re-analysis of an earlier study that estimated the number of species of bacteria in soil.

The original study, in the early 1990s, used a technique called DNA re-association to estimate the diversity of bacterial DNA in a soil sample. From this, researchers inferred how many species of bacteria were in the sample. This indirect method was used because direct counting of unknown bacteria is difficult since only a few

species can be cultured in a lab. This study estimated that there are about 16,000 species per gram of dirt.

However, the initial study assumed roughly equally-sized populations for the different species. A recent re-analysis of the data, using a different assumption about the distribution of population sizes, estimated that the number of species per gram of dirt is closer to one million.

So, while a Bayesian prior may be flawed, this is no different from any of the many assumptions we must make in a statistical analysis. The important point is that, in all cases, assumptions should be based on evidence.

Sudoku Madness

from Neville Holmes (nholmes@leven.comp.utas.edu.au)

I just read your Sudoku message on the J Forum. By a strange coincidence, one of my students chose Sudoku for his J assignment. He was having a lot of trouble, so I wrote the script below to give him a start. He didn't get all that far with writing some player support but I'll be working with him after I come back from overseas.

Because I'm leaving early Friday for a week of meetings in the US I haven't got time to do much more than send you the code. For it is that I've commented it fairly fully for the student's sake, but against it is that I left the assignments global to make it easy for him to test it.

Perhaps its interest is that I've generalised it way beyond the 3x3x3, which is, I think, a bit of a straightjacket. Also, you should be able to jazz up the code because I'm not an expert by any means, and the code is just one evening's work.

```
NB.  
NB. General Sudoku Arrays  
NB.  
NB. Generate an array of numbers, for example, shape 2 3 4 5  
NB.   which is thought of as a 2x3 array of 4x5 rectangles,  
NB.   or a 2x4 by 3x5 array of numbers, where  
NB.   (a) no number is repeated within  
NB.       0. any 4x5 subarray,  
NB.       1. any 2x4 subarray, and  
NB.       2. any 3x5 subarray,  
NB.   (b) the numbers are whole
```

```

NB.          from 1 to the maximum of 4x5, 2x4, and 3x5.
NB.
NB. Final Functions : Dsply Fnl (try "Dsply Fnl 2 2 3 3" or "Dsply Fnl 3")

NB. Some Utility Functions

Dsply =: <"2      NB. Dsply y : boxes the 4x5 subarrays
PnDy  =: ,"2@{0 1&|:}:@{,"2}@{0 2&|:}
      NB. PnDy y : converts to 8x10 array
InDy  =: Dsply@{ }$({.~ -@{*/})~>@{1&{ }
      NB. x InDy y : constructs & displays intermediate array
Vldt  =: 2&.>.@{4&$}@, @:>.
      NB. Forces 4 integers greater than 1
Mxmm  =: >./@{*/}@:((2 3$2 0 1 3 2 3)&{ }
      NB. Calculates the maximum of 4x5, 2x4, and 3x5

NB. Functions to set up running list of boxes
NB.
NB. 0. The next index (starts as <: of 2x3x4x5)
NB. 1. The overall array as a list (starts as an empty list)
NB. 2. 2x3 table of lists of numbers used so far for each 4x5 subarray
NB. 3. 3x5 ditto for each 2x4 subarray
NB. 4. 2x4 ditto for each 3x5 subarray

SSAry =: ($&a:):@{ NB. sets up the subarray of empty boxes
Strt  =: <: @{*/} ; {i.0}"_ ; 0 1&SSAry ; 1 3&SSAry ; <@{0 2&SSAry}

NB. Functions to calculate the next number
NB.
NB. Require shape as Left argument, running array as right

Ndx  =: #: >@{. NB. 4 element index to next number
UsdFS =: <@{2&{.}@Ndx >@{ >@{2&{ }@}
      NB. x UsdFS y : gets 4x5 used so far
UsdSS =: <@{1 3&{ }@Ndx >@{ >@{3&{ }@}
      NB. x UsdSS y : gets 2x4 used so far
UsdTS =: <@{0 2&{ }@Ndx >@{ >@{4&{ }@}
      NB. x UsdTS y : gets 3x5 used so far
UsdS  =: UsdFS ~. @, UsdSS , UsdTS
      NB. x UsdTS y : gets all numbers used so far
NxtNr =: >:@i.@Mxmm@{ {0: `{~?@#}@.( *@#)@:~. UsdS
      NB. x NxtNr y : selects from numbers not yet used
AdNN  =: {NxtNr <@, >@{1&{ }@} 1]
      NB. x AdNN y : puts the next number to List front

NB. Functions to put away the next number in the three boxes
NB.
NB. Further updates the running array

NwN  =: {. @>@{1&{ }@} NB. picks out the new number
NwFS =: (UsdFS <@, NwN) `{<@{2&{.}@Ndx} `{>@{2&{ }@}
      NB. x NwFS y : updates the 4x5 subarray
NwSS =: (UsdSS <@, NwN) `{<@{1 3&{ }@Ndx} `{>@{3&{ }@}
      NB. x NwFS y : updates the 2x4 subarray
NwTS =: (UsdTS <@, NwN) `{<@{0 2&{ }@Ndx} `{>@{4&{ }@}

```

```

NB. x NwFs y : updates the 3x5 subarray
UdtRA =: <:@>@{.,@] ; 1&{ @] , NwFS ; NwSS ; <@NwTS
NB. x UdtRA y : updates the running array

NB. Functions to do all, and to clean up
NB.
NB. a =. Fnl y : to get the array
NB. Dsply a    : to see the boxed structure
NB. PnDy a     : to see the 2D structure

Good =: -.@(0&e.)@>@{1&{
Once =: ([ UdtRA AdNN)^:(Good@:)]^:(*/@:[])
NB. (Once Strt) y : one attempt at the whole
Try =: ('No'"_"`InDy@.(Good@:)] (Once Strt))@Vldt
NB. Try y : works for once
Cln =: Strt@[^:(-.@Good@:)]
NB. ([ Cln Once) Strt) y :
Fnsh =: ([ ([ Once Strt@:]`).^:(Good@:)]^:_ (Once Strt)
NB. Fnsh y : repeated attempts at the whole
Fnl =: ($ >@{1&{)@Fnsh@Vldt
NB. Fnl y : the structure extracted
NB. Dsply Fnl y : to see the structure

```

NEWS

British APL Association AGM 2006
to be held in the Council Chamber at
the Royal Statistical Society
starting at 2 pm on Friday 19th May 2006

Agenda

1. Minutes of AGM 2005 (as published in Vector)
2. Report from the Chairman (Paul Grosvenor)
3. Report from the Treasurer (Nicholas Small)
(including report from membership secretary)
4. Committee for 2006-2007
5. Appointment of Auditor
6. Questions
7. Any other business (at least 24 hours notice, please.)

Anthony Camacho, Hon. Sec.

Nominations for the committee by 30 April, please. We welcome people who would like to join the committee; details from any member of the committee.

Nominations to 11 Auburn Rd., Bristol BS6 6LS or to acam@blueyonder.co.uk

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADVOCORP Oy	APL+Win, APL+Link, APL+Linux, APL+Unix, APL+Plus Sharefile	poa	Complete APL2000 and Statgraphics(StatPoint) product range and links to various third party products.
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000 APL interpreters and tools	poa	Distributor of APL2000 products for the UK Consulting and maintenance for APL applications
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Download from IBM APL2 ftp site or contact APL Products.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-B74
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL2000	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only

Books and accessories			
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and JSoftware products	poa	
	USA		
	Dyalog and JSoftware products	poa	
ubJL GmbH	Dyalog APL and		
(APL Software Team)	Dyalog Systems Ltd. Products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Acadvent	ARQUE	poa	Software for the quality assurance of university examinations.
APL Software\Services	APL Utilities	poa	Software: mostly AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick_holt@email.com.
Assured Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)

Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win.
	SharpPlot for Microsoft .Net	500	<i>RainPro</i> rebuilt as a fully-managed .Net DLL for use by APL and non-APL applications alike. May be an attractive alternative for Dyalog 11 and Santa Fe users, as it has much improved image support as well as running rather faster, and not cluttering your workspace with APL sourcecode.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/K/CoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Dynamic Logistics Systems GmbH	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.

	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>			
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.

APL Advanced Techniques.... \$59.95

488pp. book, (ISBN 0-9819067-07) including 2-disk set of utility functions (APL*PLUS PC format).

Communications \$200 pc, \$500 mf

Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADVOCORP Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API level interfacing to any system under Windows, TCP/IP network and database connectivity, APL based financial client/server applications, Cognos Planning and ReportNet consultancy.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
Assured Systems	Consultancy	poa	APL+Win and/or J and/or VB custom systems development. Conversion of APL+DOS to APL+Win a speciality.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Ray Cannon	Consultancy	500/day +VAT	APL, C, C++, Assembler, Windows, Graphics on PC and IBM Mainframe. Experience in Insurance, Chemical, and Airline Industries
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivatives plc	Consultancy	poa	Financial trading software in Q, K and kdb+
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work. full members of DSDM consortium and Agile Alliance.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C#, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP

APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.

KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambert Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
Alastair Kinloch	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
Milrinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	poa	Business Forecasting & APL Systems.
Nussbaum gift	Consultancy	poa	IT Consultant with a strong focus on APL.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACTI customisation and issues concerning the Internet and/or web design.
Orbien Consulting Inc.		poa	Consultancy
Graeme Robertson	Consultancy & Training	250-500	Design and write custom software. Maintain and upgrade APL systems. Deliver customized APL training courses
Skelton Consulting	Consultancy	poa	K/Q Consultancy
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.

	MACfns	MACfns has been released. A suite of over 100 high-accuracy, high-speed assembler code utility functions for APL+Win and APL+DOS. MACfns has been thoughtfully designed and implemented for APL programmers. It includes functions for segmented string manipulation, data restructuring, translation and datatype adjustment, and many numerical computations. Typical speedups range from 4-10 times, with reduced workspace usage, datatype conservation, identity detection, and greater numerical precision amongst the other benefits. Documentation of MACfns is extremely detailed and thorough.
ubJL GmbH (APL Software Team)	poa	Analyses, design and programming.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification. APL unicode fonts designed and built for all flavours of APL.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$26.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL Interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email acam@blueyonder.co.uk.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint.
		\$69.95 (edu)	Vox Proxy for educational use.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join Material fee 5,000yen a year
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings	\$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
Acadvent Ltd.	Dr A M Sykes	171 Gower Road, Swansea SA2 9JH, UK. Tel: 01792-201776 Email: Alan.Sykes@which.net

ACM SigAPL		ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADVOCORP Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: adrian@apl385.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: curtis_jones@prodigy.net
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Consultants Japan	Kyosuke Saigusa	1507 shimomizo, sagamihara, kanagawa, Japan 2290015 Tel:+81-042-778-2127 Fax:+81-042-778-2205
APL Germany e.V.	Dr. Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany. Email: reiner@nussbaum-gift.de
APL Italiana S.p.A.	Stefano Lanzavecchia	via Monferrato 1, 20144 Milano, Italy. Email: stf@apl.it
APL Software/Services	Dick Holt	1570 Heather Field Lane, Earlysville VA 22936 USA. Tel: +1 434-973-4255 email: dick_holt@email.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
Assured Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
Beautiful Systems, Inc.	Jim Goff	PO Box 2235 Jenkintown, PA 19046, USA Tel: +1 (215) 635-0375 Fax: +1 (215) 635-9212 Email: info@beautifulsystems.com
Ray Cannon	Ray Cannon	21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
The Carlisle Group	Paul Mansour	544 Jefferson Avenue, Scranton PA, 18510 USA. Tel: +1 570-983-2036
Causeway Graphical Systems Ltd	Adrian Smith	Brook House, Gilling East, YORK, UK. Tel: 01439-788413 Email: adrian@causeway.co.uk
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Yrjokatu 11 D 17, 00120 Helsinki, FINLAND. Tel: +358 9 674 001 Email: sales@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dyalog Ltd.	Morten Kromberg	Grove House, Lutyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-338461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
Dynamic Logistics Systems GmbH	Michael Baas	Wöhlm-Schöffer-Str. 29, 63571 Gelnhausen Germany. Tel: +49 6051 13067 Fax: +49 6051 16142. Email: info@dls-planning.com
DynArray Corporation	Dr James Brown	18360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email:info@DynArray.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivatives plc	Michael O'Neill	First Derivatives House, Kilmorey Business Park, Kilmorey Street, Newry, Co. Down, N. Ireland BT34 2DH. Tel. 028 3025 2242 Fax 028 3025 2060 Email:enquiries@firstderivatives.com
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com

General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gsoft.plus.com
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email:HMW@4extra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.Ltd.uk
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	11 Bolshaya Monetnaya Str., suite 6, St. Petersburg 197101 Russia. Tel:+7 812 325-9797 Fax: +7 812 233-9685 Email:aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel:+45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Japan APL Assoc	Juichiro Takeuchi	Keio Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel:+81(045)563-1411 Fax:+81(045)566-1617 E-mail:takeuchi@ae.keio.ac.jp
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470- 7345 Fax: +1 (952) 470-9202 Email: info@software.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: kimmo.kekalainen@kjk-tieto.com
Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com (in Europe simon@kx.com)
Lambent Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lambenttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel:+1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3. tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Alastair Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0) 7802 430202 Email: alastair.kinloch@btinternet.com
MicroAPL Ltd.	Richard Nabavi	The Rolier Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: apl@ellismorgan.co.uk
Dr. Nussbaum gift mbH	Dr Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany Tel: +49 621 7152190. Email: reiner@nussbaum-gift.de
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email:paul@optima-systems.co.uk
Orblen Consulting Inc.	Ros Fovargue	33 Cherryhill Ave., Toronto, Ontario, Canada, M1C 1W9. email: rfovarg@interlog.com or Ros.Fovargue@sympatico.ca
Qualedi Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
Renaissance Data Sys	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Right Saat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel:+1 303 278 2244. Fax: +1 303 278 6997. Email: info@voxyproxy.com
Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@nildram.co.uk
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com

Skelton Consulting GmbH	Charles Skelton	Weinklinge 9, 70329 Stuttgart, Germany. Email: c.skelton@skelton.de
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Email: bernecky@acm.org
Soliton Associates		Soliton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia
Russian Chapter of SIGAPL		Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc		P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Toronto SIG	Richard Procter	P.O.Box 39, Mansfield, Ontario, L0N 1M0, Canada, email: info@torontoapl.ca
ubJL GmbH		
(APL Software Team)	Kai Jäger	Thunseldastr. 22, 90482 Nürnberg, Germany. Tel: +49 911 482512; Fax: +49 911 482518 Email: jaeger@ubjl.de
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2 [ftp.software.ibm.com/ps/products/apl2](ftp://ftp.software.ibm.com/ps/products/apl2)

WORLD WIDE WEB SITES

Acadvent Ltd.	www.acadvent.com/
ACM SigAPL	www.acm.org/sigapl/
ADVOCORP Oy	www.rett.fi/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.apl385.com
APL Forum	www.aplforum.com
APL Germany e.V.	www.apl-germany.de
APL Italiana S.p.A.	www.apl.it/
APL Journal, Germany	www.rhombos.de/shop/a/show/article/?216
APL Borealis Inc.	www.aplborealis.com
APL Consultants Japan	http://aplcons.com
APL to ASCII	see Waterloo Archive
Assured Systems	www.assuredsystems.com.au/
AUSCAN	http://home.ca.inter.net/rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
Dynamic Logistics	www.dls-planung.de
Systems GmbH	www.dls-planning.com

FinnAPL	www.pyr.fi/apl/
First Derivatives plc	www.firstderivatives.com
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo-allegro.com/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Nussbaum gift	www.nussbaum-gift.de
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualifiedi.com
Renaissance Data	www.aplbooks.com/
Right Seat Software, Inc.	www.voxproxy.com
SigAPL	www.acm.org/sigapl/
Skelton Consulting GmbH	www.skelton.de
Snake Island Research Inc.	www.snakeisland.com
Soliton	www.soliton.com/
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.ca/
ubJL GmbH (APL Software Team)	www.aplteam.de
Waterloo Archive	www.math.uwaterloo.ca/apl_archives/Welcome.html
Jim Weigang	www.chilton.com/~jimw/

APL2000 Users' Meeting Naples Beach, Florida, November 2005

reported by Adrian Smith (adrian@apl385.com)

Location

A little knocked about by the passage of Wilma two weeks before, but the beach was still there and the hotel was remarkably unscathed. Lots of palm fronds all over the minor roads, and the practice netting on the driving range well-shredded, but apart from that, Naples was its usual welcoming self.



We had good weather right through, with the temperature in the mid 80s, but a cooling breeze and clear air. Venus was spectacular in the sky after sunset, and on two evenings we got to see Mercury setting over the sea. For those who missed it, you can always get a replay at Causeway's old Starmap server if you want to exercise some 1974 APL code one more time!

Conference Highlights

In no particular order (personal choice, of course) ...

- **METSIM.** This was the very last talk on the "All APLs" day - it was unscheduled and unprepared and frankly magnificent. John Bartlett is a chemical engineer of genius who has single-handedly cornered the world market in simulating mining projects. It is all written in APL, and (the key) it relies heavily on the ability to model anything from bacteria in ore heaps to massive underwater dredges with simple APL expressions entered by the model builder. Wonderful stuff.
- **Bill Rutiser on Try/Catch.** Bill takes time to think things through, and what he implements tends to work 'right out of the box' every time. This looks good and solid and useful.
- **DNA and APL.** Charles Brenner is making a name for himself as an expert witness in all sorts of exotic paternity cases, identification challenges, and so on. This was a superb introduction to a tough topic, and the software he uses is in APL+DOS and works just fine.

Just for the record, there were 75 non-APL2000 delegates to the main conference and over 40 stayed around for the all APLs day. The theme colour was blue, and the shirts were fine cotton turtle-necks. The Conference CD included many of the presentations, the latest beta of APL+Win and sample workspaces to accompany Bill's two talks.

Day-1 Monday November 7th

Welcome and Introduction

Eric Baelen and Joe Blaze

Eric started by noticing the existence of the new *Kids Programming Language* from Microsoft, and wondered if we should call our favourite the Adult Programming Language? Probably a great way to get banned from the internet! He handed over to Joe Blaze (now president of APL2000) who gave us a quick run-down on the new company structure. *APLNow LLC* was formed as a legal entity to receive the assets of APL2000 from Cognos. It is now an employee-owned company with ownership shared between many of the familiar names from previous conferences. Eric will be on the board of directors, and Joe thanked him for his help in facilitating the new setup.

"As an employee-owned company with each person having a really solid stake, we are able to be a lot more responsive to our customers' needs"

Kermit the Frog**Jon Capps of DataReference**

OK, the frogs came at the end (I have one to prove it) but you can guess what this talk will be remembered by! Jon's company does data migrations, typically getting data off old AS400 systems onto .Net servers. FINAC is an APL shop with a very flexible 'pull it here, push it there' application for moving data between databases fast and reliably. Jon has simply wrapped this as an 'appliance' which has an LED panel on the front (to set the network address) and apart from that is entirely configurable over the intranet. Installation - slot into rack, set address, forget. Apparently the first prototype was bright green (hence the frogs) but the shipping build is a tasteful matt white. John was actively looking for more systems which it would be good to deliver in this way, so if you have a killer app and it is only the installation that is holding you back, give him a call. The worst he can do is throw a frog at you!

Try/Catch Exception-handling for APL+Win**Bill Rutiser**

"It's actually very hard to say anything deep about this stuff" said Bill, in his usual self-deprecating fashion. Basically, what he has done is added the (almost universally accepted) Try/Catch error-handler to APL in a way which should have no impact on existing code. To make it completely transparent to existing programs (which probably have some wonderful ELXHandler technology already in place) Bill took the decision that the only code guarded by the :Try block was the code you could see as you typed it. It has no effect on called functions, and if the error is not fielded by the following :CatchIf or :CatchAll blocks, normal □ELX rules apply.

APL+Santa Fe - a Language Peer to C#**Fred Waid and Jairo Lopez**

The filibuster may be a well-established political technique, but this is the first time I have seen it used to such good effect in an APL conference. The audience was already restive, after Fred and Jairo had managed to spend the entire Sunday 'training session' talking about .Net, showing PowerPoint slides, and completely failing to get to the interesting part - a demo of Santa Fe. This time, it got serious. Fred made the mistake (with about 5 minutes to run) of asking the audience whether he should sit down and let Jairo run the demo. There was something approaching a football-game chant of *Jairo, Jairo* which Fred simply ignored, and proceeded to talk out the time.

All rather disturbing. This time last year, we seemed close to having a beta copy of Santa Fe 'by Christmas'. This year, it seems that there wasn't even a reliable demo version on the developer's hard-drive. Maybe there was some other reason Fred was so determined to make sure we didn't get time to see it, but most of the people I spoke to were deeply sceptical. When I asked why there was no Santa Fe on the conference CD, the best answer I got was "I guess they tried, but they just couldn't get the vapour to condense" which may be a little unfair. There were certainly some issues with the runtime release dates of Visual Studio 2005 which probably screwed things up for them. I think they did get it going for the Wednesday training day, but (in common with most delegates) I had Wednesday down as a rest day and gave priority to the beach. You can read Eric Lescasse's excellent notes after this report - to be fair to Fred and Jairo, Eric came away from this second training day with a much more positive impression.

As for what the Santa Fe language really is, I am no closer than I was last year. I suggest checking *chimera* in wikipedia for the word that first came to mind. Oh, and I missed Warren's talk which included a brief demonstration of Excel and Santa Fe, but caught up with him in time to see what he was having to do to make it work. He needed to write a 'shim' layer in C# to translate the Excel datatypes inbound into Santa Fe, and then back again outbound. Hmmm - this is just not a first-class citizen of the .Net world until it can expose public methods with properly typed arguments. If you read to the bottom of Eric's notes, you will see an example of calling Santa Fe from C# which looks like:

```
utils.Spread sp = new utils.Spread(); // the APL Santa Fe class
int[] total = new int[] {100, 200, 300};
int[,] matrix = new int[3,3] {{110,220,330} , {110,220,330}, ... }
int[,] res = (int[,]) sp.SpreadTotal(total, matrix);
```

Note that the C# code is casting the result to an integer matrix. What I think I need to see is some running code where this casting is not required - that way you can be sure that all the data-types are checked out at *compile* time and that you won't get runtime errors in shipped applications.

Day-2 Tuesday November 8th

New Features in APL+Win Version 6

Eric Baelen

Eric summarized the recent work on APL+Win, much of which is on the conference CD, and which will be released early next year as Version 6.

- Component files now show up many more useful properties when poked in Windows Explorer

- the right-mouse button now fires a specific 'onContextMenu' event to simplify application code which must respond to multiple mouse actions.
- Much better integration for `□MOM` - in particular the parsing of expressions involving dotted names is now handled.
- Better control over implicit output (things that just spill to the session). This can continue to behave as now, but you have the option to bury it, or to treat it as an error.
- The new Try/Catch error-handler
- Monadic `□FTIE` n will report information about a currently tied file
- The new 'colossal files' turn out to be 25-30% faster than component files.
- `□DL` really does 'sleep' and so releases other processes to run.
- Limited Unicode support with `□UCS` which converts between characters and codepoints.

It is likely that support for the ZIP format will be in this build also, as well as several grid-control enhancements.

DNA Identification and APL

Charles Brenner



Often the most enjoyable talks at these conferences are the ones that hardly mention APL, but instead show you how it can be used to really good effect to help to solve a real-world problem. I didn't learn any APL at all here, but I now know an awful lot more about DNA matching (and how to combine probabilities) than I knew before.

"DNA, APL, Sex and Violence. This is a 4-part talk - we will be doing the first 2 parts today."

You can read Charles' excellent paper in full at <http://dna-view.com/DNAtechID.htm>, so I will just mention one or two of the 'user stories' here rather than trying to reproduce the

introduction to the DNA matching techniques. He lists his speciality as 'forensic mathematics' which gets him called as an expert witness on lots of interesting court cases, as well as being asked to help out with the identification of victims of disasters like the twin-towers attack.

I'm not going to talk about programming, as most of you left me behind in 1988!

Of course it helps a lot if you have DNA from both of the people you are trying to match up, but sometimes it is possible to work 'blind' just on the balance of probabilities. The example he gave was of the Hillblom case:

He enjoyed flying light planes, which he crashed 3 times, once fatally. I guess that must have been the last one ...

Hillblom is someone that no-one will have heard of, until you see the H in DHL which he almost wholly owned. As well as crashing light aircraft, he enjoyed the company of young Asian women, and after his death children (called C1 to C4) came out of the woodwork to claim a share in the very considerable fortune. For various legal reasons, it was impossible to get a sample of Hillblom's DNA, but the DNA from the potential children was analysed and conclusively showed that 3 of them had the same father, and the 4th was unrelated. This was enough to prise open the doors of the hospital where Hillblom's DNA was stored, and the paternity was confirmed. A nice piece of detective work, and a good story!

The Lady or the Tiger *Joe Blaze (of BlazeSSI)*

For the story behind the title, refer to this lovely Russian folk tale, which you can read in full at <http://res.krasu.ru/paradigma/2/15.htm>. One of the main parts of Joe's business is helping end-users fill out government forms on-line. This means he has to read the regulations 'like the Talmud' and follow them to the letter. Of course his users don't care what happens behind the scenes, and one of Joe's objectives was to have the system look just the same whether locally installed or running off a web server. This led to Blaze working with APL2000 consultants to build on top of the APL Web component and APL web services architectures.

In Joe's words "this was an enormous culture-shock within the company, with lots of new names and confusions". The APL Web component could move the `ⓘ` parts (cleaned up) to JavaScript code to run on the client side, but they continued to maintain one set of source code in APL and run it through the converter to create the shipped versions. Joe noted that this "was fairly hard, but not as much of a culture-shock as moving to Santa Fe" which is probably a good

summary. He also recommended Eric Lescasses's web site as the only good source of training material if you *really* want to use this stuff.

Q: Having been through the JScript conversion, do you have automated testing?

A: The helpline crew run scripts against the test server. Standard stuff and 'reasons for update' tests. But it's not that easy to deploy and update the APL Webservices on their own kit. It's *not* just another webservice!

Joe's comments on the reasons for moving to .Net were very interesting. "I have absolutely *no* interest in moving to .Net - I read the government regulations like the Talmud so I am being forced to move to .Net. My object is to minimize the disruption to our environment." To get to .Net with the current set of running code (much of it vanilla APL from 66,67,68) Blaze will paste it across into Santa Fe and make the few changes required to get it to run. Then they will create a little DLL (both as a .Net assembly and wrapped as COM) so that this 'little piece' can be called from the rest of the APL system as before. Of course the user-interface will be harder to handle, so first to get the business rules moved over, then Blaze will start to look at the applications as a whole.

All APLs Day Thursday November 10th

This day was a real success story with well over 40 attendees staying on. All the speakers had good material, and showed genuinely new stuff. The talks are mostly well backed up with written material, so I will just note a few of the highlights here.

New Connections from APL2

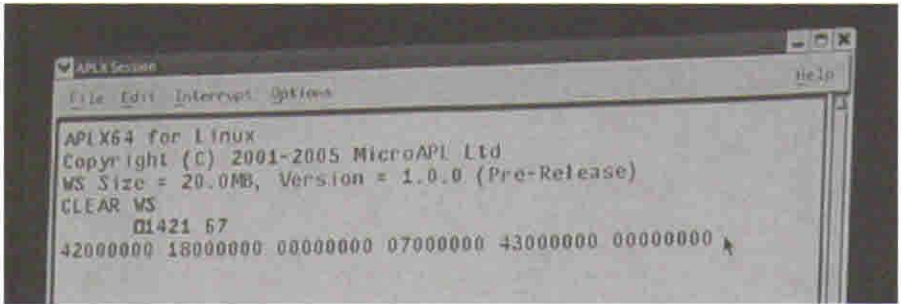
David Liebttag, IBM

David showed the interfaces to Java, and also the work on accessing COM objects from APL2 which was first presented to APL Germany this spring.

Going Native with .Net

Adrian and Jonathan from Causeway

As always, we ignored the written paper and just had some fun. The idea was to present a little 'story' with Jonathan playing a C# programmer and Adrian playing the ancient APLer. I think it worked out pretty well (it kept the audience amused, and all the examples worked) and our little smoothing algorithm got a round of applause! You can read the full story on page 98.

APLX goes 64-bit*Richard Nabavi, MicroAPL Ltd*

Note the (highly undocumented) system call which you can use to see the internal representation of any array in memory. Yes, that bit at the end really is a 64-bit representation of 67 in hex notation! See Richard's paper on page 91

APL and OO*Morten Kromberg, Dyalog Ltd*

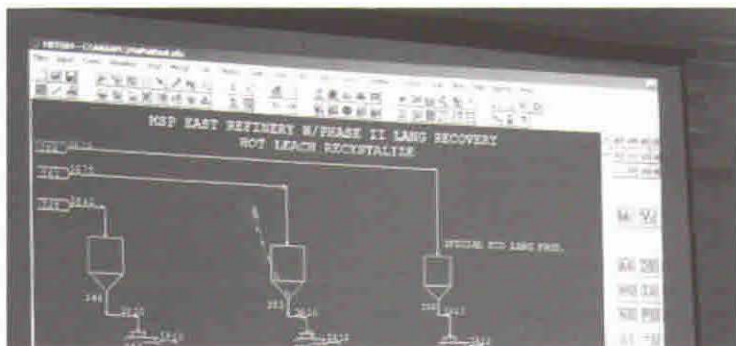
Any takers on how to pronounce "OO"? Morten made a couple of suggestions, of which "Uh Oh" was probably the most appreciated by the listeners. Again, I can leave the details of this talk to Morten to write up, but it was a nice gentle walk-through of the object-based ideas, from their origins in Simula (more or less contemporary with the origins of APL) to the explosion of interest today.

METSIM for APL+Win*John Bartlett*

This system is just amazing. METSIM is not just the tool of choice at the world's major mining companies, it is virtually mandatory. No way is someone going to dig a hugely expensive hole in the ground unless METSIM shows the downstream processing to be economically reasonable.

We have to get a proper write-up on this one, even if the only way is to get Vector to send me to Arizona for a week to interview John!

Here is a typical screen-snap of a typical section of one of John's models:



Just one 'off the wall' definition which I thought was great - *comminution* is one of the many processes you can model, which John defined as "taking a lump of rock and beating the hell out of it".

Wrap Up

A good trip, well worth the time. The absence of a Santa Fe demo in the main conference session was a major disappointment, but there was plenty of other interesting material on show. The company was (of course) as good as ever, and it will be good to see you all again in Denmark in September 2006.

APL+Santa Fe

Previewed at the APL2000 User Conference by Eric Lescasse

Introduction

APL+SantaFe has been presented in detail during the APL2000 2005 User Conference. Put in a few words, APL+SantaFe is a 100% .Net version of APL+. Here are its main characteristics:

- it is a completely new APL written for .Net
- it is a pure .Net language (as VB.Net and C#)
- highly compatible with APL+Win, but about 5 times faster and as fast as C#
- with a few changes to iron the weaknesses of APL
- with a LOT of extensions and improvements, new tools & new options
- usable from within Visual Studio 2005 with intellisense, VS debugger, & everything available under Visual Studio 2005
- with still our so valuable Interactive APL Session
- with full access to Windows Forms and all .Net controls
- with syntax extensions to allow copy/pasting C# code and making it run with minimal changes
- with access to the whole .Net Framework 2.0
- with multi-processor, multi-thread, 64-bit processor & huge memory space support
- with common error handling like C# and VB.Net
- producing 100% managed code
- producing APL .EXE and .DLL files which can be consumed by other .Net languages
- allowing you to do real Object Oriented programs (support for: namespaces, classes, methods, properties, events, inheritance, polymorphism)
- many more enhancements

This article is a mixture of information taken from slides presented by APL2000 at the Conference and from notes I have written down while looking at the APL+SantaFe demos.

APL+SantaFe FAQ

1. **What are the requirements to install APL+SantaFe?**
You need the .Net Framework 2.0
In addition it is highly recommended to install and use Visual Studio 2005
2. **What really is APL+SantaFe? Is it APL?**
Yes
3. **Is it the old APL Interpreter?**
No, it is a complete rewrite, but very similar
4. **Is it faster than APL+Win?**
Yes, about 5 times faster!
5. **But, is it REALLY APL?**
Yes, it is really APL, from top to bottom.
It is APL for .Net, a pure .Net language
There are no compromises, no tricks, just real APL for .Net
As fast as C# and C++ for .Net
6. **Is C# or VB.Net involved?**
Is this a Translator? No
Is VB.Net involved? No
Is C# code generated? No
APL+SantaFe is just a new .Net language, 100% .Net
7. **What do I use? What do I create?**
You create an APL .EXE or an APL .DLL. Porprietary workspaces no longer exists.
Your APL programs, classes, etc. are stored in one or more .apl text files
8. **Does it work with Microsoft Visual Studio?**
Yes, it is entirely integrated to Visual Studio, just like C# or VB.Net
9. **Is there still an APL Interactive Session?**
Yes, you can run APL Code in an Interactive Session, just like always
10. **Can I use other .Net DLL's and EXE's?**
Yes, you can use any .Net DLL or .EXE in APL+SantaFe
11. **Can other .Net language DLL's and EXE's use the DLL's and EXE's I produce with APL+SantaFe?**
Yes any other language can use the DLL's or EXE's I create with APL+SantaFe

12. How about the APL font?

APL+SantaFe comes with a new Unicode APL font. But any Unicode font which includes APL characters would work. APL+SantaFe uses the MS Standard for alternate keyboards: this means the right Alt key (AltGr) creates APL characters and the left Alt key accesses menus

What are the differences with APL+Win?

The function header is treated differently: variables declared in the function header are global instead of local!

All variables used in the function are local by default!

Example:

```

      ▽ R←foo a;b;c
[1]   b←10 ⋄ c←100
[2]   r←a×b÷c
      ▽

```

The APL = primitive has been replaced by an “approximately equal” primitive noted $\approx$.

The reason is that with floating point numbers expressions like: $1.1 - 0.1$ are not strictly equal to 1 but only “approximately equal” to 1 and that APL+SantaFe needed the = symbol to represent assignment.

Example:

```

      :if A≈1
          A do something
      :endif

```

⌈wi is about 98% compatible with APL+Win ⌈wi

But ⌈wi now uses Windows Forms and is therefore 100% .Net managed code.

Example:

```

⌈wself←'fm' ⌈wi 'Create' 'Form'
⌈wself←⌈wi '.ed.Create' 'Edit'
⌈wself←'fm.lv' ⌈wi 'Create' 'Listview'
⌈wself←'fm.bn' ⌈wi 'Create' 'Button'
⌈wi 'onClick' 'function'

```

Existing ⌈wi required you to wait for new controls

Existing ⌈wi did not reveal all methods, properties and events

Existing `⌶wi` provides methods, properties and events specific to APL

New `⌶wi` exposes all new controls, methods, properties and events

New `⌶wi` takes text strings to create object

New `⌶wi` can take XAML to create objects

APL+SantaFe Enhancements

Access to all of Windows Forms and .Net Controls

APL+SantaFe can use the Visual Studio User Interface WYSIWYG designer to create forms

APL+SantaFe can use Data Binding (to connect controls to databases)

With APL+SantaFe you have access to:

- Controls
- Themed Controls
- User Defined Controls
- Gradients
- FlowPanels
- TablePanels
- Transparencies
- All objects available to a C# or VB.Net programmer

Enhanced operators

All APL operators have been included in APL+SantaFe but some have been enhanced.

Example:

```
3 1 +/- 1 2 3 4 5 6 7 8 9 10 11 12
```

Complex numbers have been added to be compatible with APL2

New `⌶ucs` System Function

A new system function has been added to return the Unicode index of APL characters (and vice versa)

Example:

```

    □ucs'~99₁'
175 57 57 9075

    □ucs 175 57 57 9075
~99₁

    □ucs'≈'
8776

```

Users can still use APL exactly as today, but they can also use new ways of doing things.

Two ways for defining APL functions

Note: in the following, a lot of the enhancements have been added to allow you to copy/paste C# code examples you may find in Google and have them run in APL+SantaFe with the least possible changes. However one can still limit himself to using only traditional APL constructs and not use these enhancements if he does not want to.

For example, they can define APL functions in 2 different ways:

```

namespace Demo1{
    public class Class1{

        public f Class1(){
            // Default constructor
            r←"Hello World"
            c←10 ADD 20
            c1←nADD(10,20)
        }

        v r←a ADD b
        r←a+b
        v

        public f r←nADD(a,b){
            r←a+b
        }

    }
}

```

In this example, a function ADD is defined in the usual APL way, while a function nADD is defined using the new C#-like notation.

Note the *f* symbol which is required when using the new C#-like notation. Note also the way arguments are defined in the `nADD` function and how the `nADD` function is called in the `Class1` default constructor. This new notation allows for passing more than 2 arguments to APL functions.

You must use the new C#-like notation if you want other .Net programs to be able to consume your APL+SantaFe programs.

New `monadic` and `dyadic` system functions

Two new System Functions have been added to simplify testing for the presence of a left function argument:

```

:if 0≠⊖nc'a' ♦ ... ♦ :endif      // time consuming
:if ⊖monadic ♦ ... ♦ :endif      // much faster
:if-⊖dyadic ♦ ... ♦ :endif      // also available

```

The return keyword

The new `return` keyword has been added to return values from functions:

```

return 99          // may return a constant
return b           // may return an variable
return 10+b        // may return any APL expression

```

Support for default argument values

When passing arguments to functions, it is now possible to use default argument values:

```
r←nAdd(a,b,c=100)
```

The `nAdd` function may now be called with only 2 arguments: it will be as if 3 arguments will have been passed with the 3rd argument equal to 100.

Function overloads are supported

You may define as many functions with the same name as you want, as long as they vary by the number of their arguments:

```

nAdd(a,b)
nAdd(a,b,c)
nAdd(a,b,c,d,e)

```

```

a + @'c:\program files\' + @'apl2000\text.txt'
// don't want to use escape characters
a
c:\program files\apl2000\text.txt

```

Types

- APL+SantaFe supports ALL .Net types
- In addition APL+SantaFe supports true booleans (1 bit per element) while other languages like C# treat booleans as integers
- In APL+SantaFe every string is used as Unicode

Control Structures

You can still use all the APL+Win traditional control structures. Remind yourself that the = primitive has been replaced by the "approximately equal" primitive: $\approx$

```

:if a  $\approx$  1
...
:elseif a  $\approx$  2
...
:else
...
:endif

```

However, you can also use all of the C# control structures (which will make it much easier to copy/paste C# code into your APL programs). Here are a few examples:

```

if (a  $\approx$  1) {
...
} else if (a  $\approx$  2) {
...
} else {
...
}

i = 0
while (i < a) {
...
i++          // added to APL so that we can copy/paste code from C#
} else {     // in case while does not run even once (extension)
...
}

foreach i in 1..5 {

```

```

...
}

:for i :in 15
...
:endfor

for(i=0; i<a; i++){ // the following also works!
...
} else {           // extension: if the for block never
                   // executes, then else block executes
...
}

// instead of i++ we can use an APL expression

i = 0
do {
    i++
} while (i<a)

```

Microsoft has Code examples everywhere in MSDN: most examples can be copy/pasted to APL+SantaFe with minimum code changes.

There even is a **Paste Special** feature which automatically translates * (times in C#/VB.Net) to × and / (divide in C#/VB.Net) to ÷

Logical 'or' and 'and'

You can use the **or** and **and** keywords in APL expressions:

```

d      a ← (b ≈ c) or c ≈ d           // if b≈c is true then c≈d is not execute
ed     a ← (b ≈ c) and c ≈ d          // if b≈c is false then c≈d is not execut

```

Keywords

In order to allow APL+SantaFe to be able to use C#-like Control Structures and more, APL+SantaFe recognizes 28 or so keywords. However you can still use any of these keywords in your programs as variable or function names. As soon as you do that, you can no longer use the given word as an APL+SantaFe keyword any more in your program.

Try ... Catch ... Finally

C# and VB.Net use Try ... Catch ... Finally constructs to allow writing better and more secure code. The idea is: you Try a block of APL code; if any error occurs in the code, the Catch block is immediately executed and in any case the Finally block of code is always executed. Moreover, APL+SantaFe recognizes and can use ALL predefined .Net exceptions.

Here is an example:

```
try {
    :select a
    :case 1
        ⍎Error'DOMAIN' // cover to throwing errors
    :case 2
        ⍎Error'LENGTH'
    :case 3
        ⍎Error'SYNTAX'
    :case 4
        ⍎Error'MYERROR'
    :case 5
        throw ApplicationException("SFERROR")
    :endselect
    b ← +/a
    ⍎ ~ 'no error'
} catch ('DOMAIN') { // partial match if DOMAIN in error msg
    ⍎ ~ 'domain error caught'
} catch ('LENGTH') {
    ⍎ ~ 'length error caught'
} catch ('SYNTAX') {
    ⍎ ~ 'syntax error caught'
} catch (Exception err) {
    ⍎ ~ err.Message
} catch {
    ⍎ ~ 'all other errors caught'
} finally {
    ⍎ ~ 'finally always runs'
}
```

Miscellaneous:

⍎def works in 2 ways: ⍎def '∇ ... ∇' or ⍎def 'f ...'

⍎def supports ⍎cr and ⍎vr arguments.

The execute APL primitive (⍎) works in APL+SantaFe as in APL+Win . It is 100 times faster (precisely 103 times faster) than dynamic functions in .Net. Execute now runs in the same domain. Execute works in the APL Session

□fmt extensions

- Supports all .Net format strings
- Includes support for DateTime format strings
- Support specifying column widths
- Supports format string for each element in a matrix
- Pattern formats supported
- Supports NumberFormatInfo for culture or program specific settings

□nfi - Culture specific number formatting.

```
nfi=□nfi
nfi.negativeSign+'-' // or: nfi.negativeSign+'-'
// no need to reassign □nfi
// nfi setting is automatically respected by □fmt and *
```

Consuming APL+SantaFe class from C# and reciprocally

A Solution can contain many projects. Each project may be in a different language (C#, APL+SantaFe, VB.Net, ...)

Example: A Visual Studio 2005 Solution may contain:

- an APL Santa Fe project
- a C# Class Library

or vice versa.

Assume we have written an APL+SantaFe DLL containing a **Spread** class in a namespace called **utils** and that our Visual Studio Project called WinSpread is written in C#.

In order to use the APL+SantaFe Spread class from C#, you need to:

add a Reference to utils in the WinSpread C# project (right click References / click Add / click the Browse tab and browse for the APL+SantaFe DLL file to add)

add a using clause:

```
using utils; // to avoid typing utils.Spread and want to
             // only type Spread to call the Spread method
```

Now here is how the C# class would for example use the APL Santa Fe Class:

```

utils.Spread sp = new utils.Spread(); // instantiate class
int[] total = new int[] {100, 200, 300};
int[,] matrix = new int[3,3] {{110,220,330},{110,220,330}, ... }
int[,] res = (int[,]) sp.SpreadTotal(total, matrix); // call the

```

Here is how you would have defined the SpreadTotal function in APL+SantaFe:

```

public f R ← SpreadTotal(total, matrix) {
R ← (int) total ROUNDCOL total SPREAD matrix
}

```

ROUNDCOL and SPREAD being private methods of the Spread class.

Note that, because we want C# to consume the SpreadTotal method we need to strong type the result by casting it to an (int).

The final compiled program is an EXE (or DLL): no one can tell if it was written in C#, VB.Net, Santa Fe, J#, C++ or a mixture of these languages!

Windows Forms

You can use 100% of Windows Forms and .Net controls in APL+SantaFe in addition to wi. Here is a short example:

```

myfm = Form()
mybt = Button()
myfm.Controls.Add(mybt)
mybt.Text = 'hello'
myfm.ShowDialog() // modal form
myfm.Show() // modeless form

```

Reminder: APL+SantaFe gives access to 100% of the .Net 2.0 Framework Classes, Methods, Properties!

Typed Arguments

- APL+SantaFe functions now supports typed arguments
- `fn(int a, long b)` will be seen as a typed call by other .Net language
- `f long fn(int a, string b)` will return a long to other .Net languages

Conclusion

This was just a compilation of my notes taken on the fly at the APL2000 2005 User Conference. I hope you have enjoyed discovering this “soon to be released” completely new version of APL. More will be posted to my Web Site as new information is released and as soon as I can try the Beta version.

DISCOVER

Gérard Langlet: A Man of Distinction

by Sylvia Camacho

We take as given the idea of distinction and the idea of indication, and that we cannot make an indication without drawing a distinction.

G. Spencer Brown, 1969

Gérard Langlet (1940-1996) was a scientist by profession, a polymath by inclination and an expert user of computers from necessity. He published only in specialist French and English journals and died before he was sixty. Consequently his ideas have been little known and difficult to access but the growth of the Web has now offered his admirers an easy remedy. This paper is therefore addressed to curious browsers who may happen upon it in a search among the many fields which captured Langlet's imagination. Readers are warned that he was never overawed by the authorities that he challenged and students would be wise to toe the party line until their examinations are safely passed before exploring the unconventional ramifications of his research.

Langlet questioned the mathematics which is used to underpin the sciences of physics, chemistry and biology, particularly for description and analysis at the smallest scale [2 p.119]:

Continuous fields (and their equations) are in fact an approximation, they would have no meaning at quantum scale, since the simple idea of a quantum should imply that no infinitesimal quantity should ever been (*sic*) thought of, for any plausible mathematical model in physics or biology.

It is impossible to understand a process such as vision without trying to understand the action of one quantum on another quantum.

He began his career as a chemical engineer, but then moved into crystallography and biochemistry and was married to and collaborated with a theoretical microbiologist. The organisation for which he worked, near Paris, has in its extensive description, atomic energy, materials science, condensed state matter, molecular chemistry and information research. In the nature of their business they make wide use of computers and are well-informed about software and Langlet

took a close interest in both the architecture of the machines and the various programming languages available for them. During the 1970s he was introduced to the alternative mathematical notation invented by Kenneth Iverson in the late 1950s and 1960s and implemented as a computer language by IBM under the acronym *APL* (A Programming Language). It was Iverson's additions to the scope of conventional mathematical notation that especially attracted Langlet's attention. He says that he had studied the impact it made across his wide field of interest for over ten years before, in 1992, he made his major presentation to the international APL conference in St Petersburg. However it is quite unnecessary to have experience with APL as a programming language to appreciate the aspects of Iverson's approach that suggested a resolution of Langlet's mathematical problems, which he frequently summarised in this quotation:

We only perceive differences.
Christian Huygens, *Traité de la Lumière*, 1678

Digital means for control or calculation were used well before the Twentieth Century. Jacquard looms and pianolas are hole/no-hole devices. Charles Babbage called his first calculator a Difference Engine and telegraph and telephones made use of on/off relays. Valves, later to be replaced by transistors, came into their own as fast switching devices only after the Second World War but from that time digital technology has become increasingly favoured and when Iverson became involved with the design of IBM computers no one doubted that they would be fundamentally digital machines. This posed some problems for scientists and engineers for whom mathematics was almost synonymous with calculus and continuity. Circuit designers therefore turned to George Boole's algebra for concepts to match on/off switches. This explains how 'or-gates' and 'and-gates' became symbols in the 'logic-blueprints' drawn up by manufacturers of pulse-driven devices. However computers are primarily required to do mathematics, not formal logic and a way had to be found to describe how they could achieve it, starting from simple yes/no choices. Kenneth Iverson, although a mathematician, had been closely involved with the design of digital computers as they were becoming a commercial proposition in the United States in the late 1950s and early 1960s. He realised that conventional mathematical notation was not a suitable medium in which to describe the fundamental operations of such machines and over a few years he invented an alternative; different in three major respects. Firstly, the rules for using any of the conventional notations he chose to incorporate were recast to a rigorous consistency suited to the unintelligent rigidities of a machine. Secondly, the mathematical objects upon which the notation operated were conceived as arrays, of arbitrary dimensions, with respect to which a single number or character is merely an item. Thirdly, he subsumed in his mathematical notation the arithmetic of George Boole's algebra.

Anyone who has to incorporate mathematical expressions within documents created on a modern computer can sympathise with the problems which Iverson faced. In addition to its invented symbols and wide-ranging plunder of several alphabets, conventional mathematical notation is not written along a line but is disposed on a plane, making no attempt to respect font size. Added to this are obscure rules of precedence for the order in which mathematical expressions should be scanned and resolved and all had to be subject to constraints imposed by the limited number of computer codes available for symbol assignment.

Iverson used non-alphanumeric characters for the mathematical operations he needed to define and called them 'primitive functions'. These fall into three groups; first, those with conventional notation equivalents; second, new characters used in the manipulations of arrays; third, characters specific to computer implementation. Computers are imperative machines for which all inputs are commands demanding a response. With Iverson's notation the response to any expression immediately follows the input instruction. This is known as the interactive approach and is characteristic of languages implemented by interpreted as opposed to compiled software. It is more expensive in computer resources but is ideal for teaching and for experiments in mathematics. Iverson was above all a teacher who had a lifetime interest in natural language and its grammatical structure and he considered mathematics to be a language open to the same analysis. It is hoped that the following paragraphs will give some idea of the power and elegance of his notation although it is not in any sense a comprehensive description of APL, being only that small part needed to appreciate Langlet's thesis.

For the four arithmetic operations *plus*, *minus*, *times* and *divide*, Iverson accepted the conventional symbols '+ - x ÷' but, to distinguish between a negative value and a subtraction instruction, he used a high minus immediately preceding and forming part of the number to be signed negative for, as Cornelius Lanczos says [3 p.65]:

To use the minus sign with two different meanings is obviously confusing and damaging to the conceptual understanding of mathematics.

When Iverson came to add *raise to a power* and *extract a root* he made a significant simplification. He used the asterisk '*' for the primitive function, which takes two arguments; to the left the value, to the right the power to which it is to be raised and thus $2*5$ gives 16. What then should he use for $\sqrt{\quad}$ or *root*? To Iverson the answer was obvious; roots are fractional powers and the asterisk will serve for both: the value is still the left argument with the requisite fraction to the right and thus $16*0.5$ gives 4, the square root. The fraction of course might itself be a result returned by an expression on the right. Precedence must always be unambiguous

and this is achieved by a rule that the righthand argument to any function is the result of the entire expression to its right. This makes unnecessary the conventional rules about the order in which evaluations are made and greatly reduces the need for parentheses, although these can be used to force the evaluation of expressions so enclosed, causing the result to be substituted for the contents of the parentheses in the overall working. Functions requiring two arguments he called 'dyadic' and then went on to define 'monadic' uses: asterisk '*' with only a right argument he defined as *exponential* so that $*1$, for instance, returns e , the base of the natural logarithm. Iverson delighted in emphasising such relationships by his choice of symbols. For example *logarithm* is a circle surrounding an asterisk, the left argument being the base (10 for school logarithms) but it defaults to e , the natural logarithm. APL has many such elegancies but only selected examples can be illustrated here because many use special symbols in fonts that may not be easily available to everyone. Fortunately most of Langlet's ideas can be explained using common fonts. For curious readers, sources of free APL interpreters are listed below.

Iverson generalised the idea of a variable to make the fundamental concept for all data that of an array. He left to the code implementing the interpreter the task of recording the size and shape of all data variables and for tracking any changes ordered by the program, allocating and releasing storage dynamically, thus relieving the programmer of the onerous task of reserving appropriate memory for each variable and releasing it for other uses when no longer needed. The programmer can control this if he wishes, finding the shape of any array with command ' ρ ' (Greek *rho*) to ask for the *shape* of the right argument or, with a left as well as right argument as in ' $2\ 4\ 5\ \rho$ ', to *reshape* it (here to two matrices of four rows and five columns).

Character data is distinguished by opening and closing quotes. Numeric character sequences enclosed in this way can be converted to computable numbers with the function *execute*. Conversion the other way, from numeric to character form is done with function *format* (the symbols for these are not in this font). Alphabetic material is thus easily handled and integrated with the mathematical formalism. This flexibility required a large number of primitive functions to be devised for creating, changing, shaping and interrogating multi-dimensional arrays, which can themselves be arguments to most of the primitive functions. We shall only illustrate in this paper those needed to explain Langlet's work. This took its inspiration from Iverson's inclusion among his primitive functions, of notation to operate with binary data, expressed using only the digits zero and one.

In propositional logic Boole's formalism operates with values *true* and *false*, the values of propositions which, in his algebra, are combined using symbols ' $\wedge \vee \sim$ '

for *and*, *or*, *not*. However, Boolean logic can also be understood as the arithmetic of the set 0,1 where 0 replaces *false* and 1 replaces *true*. Iverson realised he could enhance the mathematical significance of Boole's logical notation by including as primitive functions the conventional symbols '=', $\neq$, $>$, $\geq$, $<$, $\leq$, which return the true (1) or false (0) response according to the values of their left and right arguments. This is an exceptionally powerful tool when the arguments to which the tests are applied are arrays of any size and shape.

Iverson included four operators to modify the action of his functions. An operator has one or two functions as its arguments and produces a modified function as a result; the modified function then acts on the argument or arguments to the right (and maybe also to the left) of the operator expression. The operators are *inner product* ($\cdot$), *outer product* ($\circ$), *reduction* ($/$) and *scan* ($\backslash$). For example the *plus dot times inner product* (written $+ \cdot \times$) is conventional matrix multiplication. Iverson allowed an operator's arguments to be any compatible primitive functions. The arguments of the expression containing the operator have to be compatible; since, for instance, in matrix multiplication the values to be multiplied must be in pairs, the first argument must have as many values in its columns as the right argument has in its rows and the number of rows in the left argument must be the same as the number of columns in the right argument. (If they do not the APL interpreter halts with an error.)

The second operator, *outer product* ' $\circ$.' ($\circ$ is called 'jot') takes a dyadic function f to produce a new dyadic function $\circ . f$ (jot dot f). For example, the following expression produces a multiplication table:

```

      1 2 3 4 5   $\circ . \times$  1 2 3 4 5
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25

```

The third operator 'reduction' is monadic. Its single argument is the function to its left and its effect is as if the function were repeated in between each pair of elements of the right argument. So $+ /$ with a row of numbers as its argument returns the sum of the row.

```

      +/ 1 2 3 4 5
15    (i.e. 1+2+3+4+5)

```

Operators apply equally to Boolean primitive functions and Boolean arrays and it was this facility in particular that attracted Langlet's attention. As he played with APL, he realised that, for his investigations, a particular expression was

overwhelmingly significant. This concerned the fourth operator 'V' called *scan*. Scan, like the other operators can be used to modify any compatible function (so $\times \backslash$ 5 9 6.4 100 yields 5 45 288 28800, the cumulative products). Langlet was fascinated by the uses of operators with Boolean logic functions and particularly $\neq \backslash$ (not-equals scan). This, used on a row of binary digits, performs modulo-2 parity propagation. Integers are said to have the same parity if they are either both odd or both even. As we have seen $\neq \backslash$ 1 1 0 0 0 1 0 1 will return 1 2 2 2 2 3 3 4, the cumulative sum, but $\neq \backslash$ 1 1 0 0 0 1 0 1 returns 1 0 0 0 0 1 1 0, the cumulative parity or in other words, the 2-modulus of the cumulative sum, a fundamental of binary arithmetic.

However, Langlet had in mind a more primitive concept particularly applicable to the atomic level physics, chemistry and biology with which he was professionally concerned. In his own words [2 p.120]:

$\neq \backslash$ is the correct mathematical formulation of the *least-action principle* (expressed in decision theory as well as in genetics). This mechanism never introduces noise into the data it processes and operates on a constant volume (global shape, size) of data, therefore it represents the conjunction of an *adiabatic system* – which never produces heat i.e. entropy i.e. disorder – and of a system whose evolution would progress at *constant volume* in thermodynamics. With the usual criteria of classical thermodynamics – which does not take information into account as the essential factor, but postulates continuity, simultaneity, numeric additivity of actions together with some randomness, however modulated by mysterious statistical distributions, the 2nd principle of thermodynamics cannot be respected by living organisms! $\neq \backslash$ now brings a magnificent counter-example.

In 1991 he expressed his ideas in a paper called, *Paritons and Cognitons Towards a New Theory of Information* [5] but it is perhaps easier to approach his thinking from a later paper (1993) which he called by a typically provocative title, *The Axiom Waltz or When 1+1 make 0* [4].

Langlet postulated a Principle of Conservation of Information analogous to the conservation of momentum in mechanics. Information is only expressed in bits and cannot be either averaged or smoothed. He displayed a typical joking slide at a conference:

**Déclaration des Droits du Bit
(Antwerp 1994)**

0. All bits contain information, so they deserve respect.
1. Every small bit is a quantum of information.

Nobody has the right to crunch it.
 Nobody has the right to smooth it
 Nobody has the right to truncate it.
 Nobody has the right to average it with it(s) neighbour(s).

Nobody has the right to sum it, except modulo 2.

Only reversible processes conserve all the information contained in a given system and, as this is what he wanted to do, he needed a Boolean model comparable with linear algebra. In his paper first published in 1993, he defined the axioms for such a calculus. He favoured a matrix mathematics because a self-inverse matrix without numeric approximation would be the ideal operator for transforming information. He intended to show how this is possible for Boolean matrices and he began with order 2 as follows.

Conventional matrix multiplication is defined:

matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ multiplied by matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ results in matrix $\begin{pmatrix} a^2+bc & ab+bd \\ ac+cd & bc+d^2 \end{pmatrix}$

therefore to give a Boolean matrix of $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ the following equations must hold:

$$\begin{aligned} a^2+bc &= 1 \text{ (eq.1)} & ab+bd &= 0 \text{ (eq.2)} \\ ac+cd &= 0 \text{ (eq.3)} & bc+d^2 &= 1 \text{ (eq.4)} \end{aligned}$$

Equations (1)&(4) imply: $a^2-d^2=0$ (eq.5)

therefore $a^2=d^2$ & either $a=d$ or $a=-d$

A determinant equal to 1 is expressed: $ad-bc=1$ (eq.6)

(eq.2) is resolved by either: $b=0$ or $a=-d$

(eq.3) is resolved by either: $c=0$ or $a=-d$

The solution $a=d$ is only possible if $b=0$ & $c=0$

Thus, according to (eq.6) $ad=1$

Thus a & d are both equal to 1 or -1

Two matrices are therefore self-inverse: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ and $\begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix}$

Thus with $a=-d$ (eq.6) becomes: $-a^2-bc=1$ (eq.7)

But adding (eq.7) to (eq.1) would force the resolution $0 = 2$

This result might seem absurd but in fact it indicates that we are dealing with a finite algebra in which these numbers are congruent modulo 2.

Langlet held that information can always be expressed by zeros and ones because our perceptions rely upon differences for sensing our universe. Biological receptors operate in discrete mode just as, at the fundamental level, computer components have only two states. Therefore the most elementary operation is not addition, it is *not-equal*.

Langlet's conclusion is that physical properties and natural algebra are isomorphic at the level of parity which is the logic of 0 and 1, the only elements of the set $Z/2Z$, his abbreviation for modulo-2 integer algebra. This is expressed in APL as $0 \equiv 1 \neq 1$ (the APL name for $\equiv$ being *match*) which translated is, 'it is false to say one does not equal one'. The equations (1), (2), (3), (4) can thus be written in $Z/2Z$, for which sum is equivalent to $\neq$ *not-equal* and product is equivalent to $\wedge$ *logical-and*. So, as $a=d$ and $a=\neg d$ are the same, they must equal 1 if bc is zero. By equation (6), b or c or both must then be zero. But $ad=0$ with $a=d$ also allows $bc=1$ ($b=1$ and $c=1$). Thus in $Z/2Z$ there are four matrices (G stands for *Geniton* by analogy with Hamiltonian, a typical Langlet play on words!):

Identity/anti-id matrices		horizontal Geniton	vertical Geniton
(I)	(anti-I)	(Gh)	(Gv)
1 0	0 1	1 0	1 1
0 1	1 0	1 1	0 1

These are the only transformational matrices in $Z/2Z$ that are not trivial (anti-I is almost trivial because its square $\text{anti-I} \neq \wedge \text{anti-I}$ yields I).

Condition (6) $ad-bc=0$ can be rewritten $ad \neq bc$ in binary algebra (for which multiplication is $\wedge$, so $b \wedge c \equiv b=1$ and $c=1$). This means that the product in $Z/2Z$ has become the function which only returns 1 if both arguments are 1 and otherwise returns zero (that is the meaning of $\wedge$). This is equivalent to a return of the *minimum* of its arguments. So $ad=0$ entails the other two possible matrices which, while not invertible, are reverses of each other.

They are:

(G)	(Gd)	for both of which $a \wedge d=0$. Each is the rotation of the other
1 1	0 1	about the second (bottom left to top right) diagonal
1 0	1 1	

Together with the four matrices above, this set of six forms a multiplying group for the matrix product in $Z/2Z$, as defined by Cornelius Lanczos [3 p.60]:

A... characteristic feature of a group is that an *operation* is given which involves two elements of the group.... A fundamental condition for the existence of a group is that the result of this operation must again be an element of the group.

The expression for matrix multiplication, $\text{matrix} + . \times \text{matrix}$ has as equivalent for $Z/2Z$ $\text{matrix} \neq \wedge \text{matrix}$, ($\neq$ being the equivalent of plus and $\wedge$ the equivalent of times). For instance:

1 2	$+. \times$	1 2	returns	7 10	
3 4		3 4		15 22	while

$$\begin{array}{ccc}
 (G) & (G) & \text{returns} \\
 \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \neq \wedge & \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} & \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}
 \end{array}$$

and

$$\begin{array}{ccc}
 (G) & (Gd) & \text{returns} \\
 \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \neq \wedge & \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} & \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}
 \end{array}$$

and likewise $Gd \neq \wedge G$ returns I

which is the identity (unit) matrix. So G and Gd are each a cubic root of I and as transformational matrices they are analogous to j and j^2 , the complex cubic roots of 1. In complex classical algebra this cannot be achieved without error generated by approximation to e and π and i .

Langlet explains that he chose 'G' to stand for 'Geniton' because it can be used to create matrices of higher orders [4 p.109]:

Are there any matrices in the same algebra, that have the same properties as G and Gd (the inverse of their square), Gh and Gv (self-inverse), and of which the matrices of rank 2 are the sub-matrices, for every value of row R?

If G, being order 2, is renamed G2, we can then create a matrix of order 4 by replacing the three ones in G2 by G2 and the zero by the order 2 zero matrix Z2:

$$\begin{array}{ccc}
 G2 & G2 & \\
 \text{thus} & \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} & \text{which being order 4 we may call G4}
 \end{array}$$

Given that Gd is the inverse of G, what is the inverse of G4?

$$\begin{array}{ccc}
 G4 \neq \wedge G4 & \text{gives} & \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}
 \end{array}$$

symmetrical with G4 about the second diagonal
This could be called Gd4

$$\begin{array}{ccc}
 \text{while } G4 \neq \wedge Gd4 & \text{gives the order 4 unit matrix} & \begin{pmatrix} I2 & Z2 \\ Z2 & I2 \end{pmatrix} \\
 & & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}
 \end{array}$$

as expected.

This reasoning can be extended to a matrix of order infinity. Moreover these properties are preserved after removal of the last line and first column of matrices G4, G8, G16 . . . etc. to give matrices of odd orders. Langlet's conclusion from these procedures is [4 p.119]:

We will have both a construction and a rigorous representation, from 2 to infinity, of the rotation operators j and j^2 in a complex plane. There will be no need to define, as is usual as a precondition, any number other than 0 and 1. Nor will it, above all, be necessary to define i , the imaginary root of -1 , which first required the definition of negative numbers. (A well-organised head being worth more than a well-filled head, Montaigne.)

An outstanding example of the application of this reasoning is his analysis of the hexagonal arrangement of the hundred million rods in the human retina [2].

The Axiom Waltz paper formalised the research Langlet had presented two years previously in *Paritons and Cognitons*. There he reminded his readers of the traditional use of parity checking for computer error detection and went on to demonstrate the intriguing object generated by appending iterations of $\neq$ parity propagation onto a Boolean vector of any length, creating a matrix [5 p.98]:

Let us define the PARITON as the Boolean MATRIX containing as rows, the differing bit sequences of successive parity integrals of an original bit sequence.

For instance:

$\neq$ 011000010111000001101100 gives
010000011010000001001000

and, of course, the result of *not-equal reduce*

$\neq$ 011000010111000001101100

of the same vector returns its parity, 0, the last item.

Langlet is drawing an analogy between the use of *not-equals scan* in his $Z/2Z$ algebra and integrals and differentials within conventional mathematics. He points out that $+$ *plus scan* can be used to plot the integral of any function if computed at frequent intervals in the discrete domain. Therefore $\neq$, *not-equals scan*, can be considered a *parity integral*. Moreover it can be applied iteratively to obtain the *parity integral* of the *parity integral* and this has some very interesting consequences. In particular a periodicity will appear after a number of iterations and, if each iteration is held as a next row, the full cycle will result in a square matrix which has the characteristic that the entire matrix can be retrieved by successive iterations of any row. The number of iterations in the cycle will be that power of 2 greater than or equal to the length of the initial vector. Such a cyclic matrix can be considered the topological equivalent of a cylinder in which case the final column can be viewed, end on, as a polygon or circle; to which Langlet gave the name *cogniton*. It contains, of course, the cumulative parity of each row. The inverse of parity integration, *parity differentiation*, is calculable from this last column or *cogniton*. The procedure is as follows:

calculate the penultimate column using *cogniton* $\neq$ a one-bit right rotation of *cogniton*
 from the penultimate calculate the ante-penultimate column by the same procedure
 iterate until the square matrix is complete.

He points out that the periodicity of the left half of the matrix is one-half of the whole and that of the left quarter is one quarter of the whole, and so on. Thus the sequence of the first 4 bits repeats at the 5th row, the sequence of the first 8 bits repeats at the 9th row, the first 16 at the 17th and so on. The pariton resulting from an initial sequence containing only Boolean ones gives rise to the fractal known as *Sierpinski's Gasket*. This is self-similar at all scales and has the interesting quality that if each row is understood as the base two representation of a single number, the set has no duplicates and its maximum value grows with the size of the matrix. Langlet noted, without drawing any conclusions, that these numbers if the right-hand digit is taken as the most significant, are similar to the *paroxistic series* which is exhibited by various natural phenomena [6 p.130]. For instance a 32-bit matrix converts to:

1 3 5 15 17 51 85 255 257 771 1285 3855 4369 13107 21845 65535 65537 196611
 327685 983055 1114129 3342387 5570645 16711935 16843009 50529027 84215045
 252645135 286331153 858993459 1431655765 4294967295

More generally, paritons, when encoded this way, easily allow to mimic observed spectra, especially EPR (electronic paramagnetic resonance) ones: some specialists (physico-chemists, biologists), who did not know that the spectra resulted from a short APL one-liner with no arithmetics . . . – were often able to “identify” the chemical products which could have produced the spectra!

There is another property of the *pariton*, to which Langlet gave the name *helical transform* because a diagonal of the matrix, when it is being viewed as a cylinder, constitutes a helix. He touches briefly on some references connected with characteristics of the main (bottom right to top left) diagonal but is more interested in the diagonal which runs bottom left to top right. He finds [5 p.103]:

In any pariton, if any sequence I produces a diagonal D, a sequence equal to D produces a diagonal which contains I.

So D will be named $-I$ from now on (no confusion can arise from the minus sign because the pariton involves no arithmetic).

It means that $-I$ is the helix inverse or helix transform of D. . . .

As a corollary, any row of the matrix has its corresponding helix inverse on the cylinder.

Writing just two years before the successful decoding of the human genome, his drawing attention to these helical mechanisms is no accident. Langlet was fully

aware of the tension between the theories of physics and the apparently high immunity to entropic degradation of living systems. He points to the asymmetry of the *pariton* and suggests [5 p108]:

Maxwell would have been fully satisfied to observe that the *pariton* does his hypothetical demon's work. A rapid glance at any *pariton* shows that the organisation of information is 'nil' on the left side and maximum on the right side (the cyclic memory).

The binary integral mechanism is a *self-associative auto-organiser*.

Langlet is referring to James Clerk Maxwell's thought experiment, in which he imagined a demon selecting faster among slower molecules and so creating a hot box from the contents of a cooler box in defiance of entropy. Maxwell's demon is generally thought to have been thwarted by the requirement that it itself must expend energy in obtaining its information about the speed of particles but Ilya Prigogine pointed out that what may apply on average to the universe as a whole and over cosmological time is not true at all scales [7 p.175]:

Often biological order is simply presented as an improbable physical state created by enzymes resembling Maxwell's demon, enzymes that maintain chemical differences in the system in the same way as the demon maintains temperature and pressure differences. . . .

In the context of the physics of irreversible processes, the results of biology obviously have a different meaning and different implications. We know today that both the biosphere as a whole as well as its components, living or dead, exist in far-from-equilibrium conditions. In this context life, far from being outside the natural order, appears as the supreme expression of the self-organizing processes that occur.

In parts of this early paper (first published in Belgium in 1991) Langlet hints at applications of his calculus which he developed more fully over the five years left to him. *The APL Theory of Human Vision* [2] which he presented in Antwerp in 1994 is a good example of his attempts to show how his mathematics could be applied at the molecular level in biochemistry. The references below detail where this and other of Langlet's work quoted here has been published. It is hoped in due course to make everything that he wrote as easily available.

References

- [1] G. Spencer Brown, *Laws of Form*, Geo. Allen & Unwin, 1969, ISBN 04 510028 4
- [2] Ed. Alain Delmotte, Gérard Langlet, *The APL Theory of Human Vision*, Conference Proceedings Antwerp 1994, APL Quote Quad, Vol.25, No.1, ISBN 0-89791-675-1
- [3] Cornelius Lanczos, *Numbers Without End*, Oliver & Boyd, Edinburgh, 1968
- [4] Gérard Langlet, The Axiom Waltz, *Vector*, Vol.11 No.3, 1995, ISSN 0955-1433
- [5] Gérard Langlet, Paritons and Cognitons, *Vector*, Vol.19 No.3, 2003, ISSN 0955-1433
- [6] Ed. Lynne Shaw, Gérard Langlet, *Towards the Ultimate APL-TOE*, Conference Proceedings St. Petersburg 1992, APL Quote Quad, Vol.23, No.1, ISBN 0-89791-477-5
- [7] Ilya Prigogine & Isabelle Stengers, *Order out of Chaos*, Fontana Paperbacks, London, 1985, ISBN 0-00-654115-1

LEARN

Object Oriented Programming in APL+Win

by William Rutiser, APL2000 (wru@apl2000.com)

Talk given at the 2005 APL2000 Users' Meeting

Introduction to the MOM Object System

Elevator description

The MOM object system extends APL with simple basic facilities for object oriented programming, a predominant design and structuring paradigm for modern programming languages. MOM objects are typeless, classless, and inherently polymorphic. They provide syntactic and semantic foundations for future elaboration guided by experience. Entirely internal to APL, MOM objects are not related to Ω WI objects, COM objects, ActiveX objects, or any other family of objects.

A MOM object is a simple container enclosing a set of named APL functions and variables. Each MOM object is distinct from every other MOM object. While MOM objects themselves are unnamed, each is associated with a unique *object-reference* value. Object-reference values join numbers and characters as a third fundamental APL data type. Object-references, just like numbers and characters, can be formed into arrays, assigned to variables, passed as parameters, and saved in workspaces. Where meaningful, they are included in the domains of primitive APL functions.

Syntactically, object members are accessed as *objref.member* where *objref* is a variable holding an object reference and *member* is the name of the member within the object. This use of the dot, ".", is given very high parsing precedence so that this compound name can be used wherever a simple name is allowed. This character retains its traditional meaning and precedence as the inner product operator; the two uses are distinguished by the immediate left context. A function, including user defined functions, causes the dot to be interpreted as inner product. A variable or undefined name will cause the dot to be interpreted as part of a compound name.

Objects are created by the `⌈MOM` system function. Arguments to `⌈MOM` describe the object to be created; its result is a reference to the new object.

During the execution of a member function, the system variable `⌈MSELF` is set to the reference to the containing object. This gives the function access to the object's other members.

Some system functions, such as `⌈NL` and `⌈CR`, are implicit members of every object; they refer only to an object's members. So, just as `⌈NL 3` gives a list of the functions in the workspace, `MyThing.⌈NL 3` gives a list of the functions in the object referred to by the variable `MyThing`.

Questions Answered

What is an object?

In the context of object oriented programming, an *object* is an encapsulation of data and code with certain additional properties that vary by language. The important properties that distinguish objects from other programming constructs are:

- **Identity** – Each object can be distinguished from all other objects including otherwise identical objects.
- **State** – The data enclosed in the object. It may change with time and influence the object's behavior. A MOM object may contain named members that act just like ordinary APL variables; they can be assigned any APL value.
- **Behaviour** – The effects of invoking the object's member functions. These effects may depend on the current object's state and may include changes to the state. MOM objects may contain named members that act just like ordinary APL user defined functions. Also, some system functions are implicit members.

In general, an object is characterized by the combination of its state and behavior. Some simple, but useful, objects may contain no variables or no functions; they serve as containers. Their behavior is to preserve or their contents unchanged.

What is an object reference?

Classical APL is a *value oriented* language. Arrays are values. Function arguments are values. Function results are values. These values are abstract and immutable. They have no identity of their own. APL variables are named things that are temporarily bound to values. The assignment statement replaces the value bound to a name with a different value. We often say that we are assigning something to a variable but Ken Iverson often referred to this as assigning a name to the value.

Most common programming languages, particularly those that are often taught as first languages, are *storage oriented*. The programmer thinks of bits, bytes and words in memory and setting those bits to representations of numbers, characters, and the like. Such languages often have an *indirection* mechanism that allows a program variable to refer to different pieces of memory at different times. In C, the mechanism is called a pointer. In these languages, an object is essentially just a block of storage accessed via a pointer.

Object references are a value oriented abstraction of the pointer. An object references connect variables and expressions with MOM objects. An object reference is a value that identifies a particular object. When assigned to a variable, an object reference value is used to access the members of its object. It is important to remember that the same object reference value can be assigned to several variables at the same time. The reader may find it helpful to visualize an object reference as a length of string tied to a value at one end and an object at the other.

How can this be demonstrated?

Consider the following code. It can be executed in a clear workspace.

First we create two empty objects.

```
x ← □MOM ' ' ' '
y ← □MOM ' ' ' '
```

Their object references are different.

```
x == y ==> 0
x == x ==> 1
```

Put something into the first object.

```
x.data ← "This is data assigned thru x"
```

Look at it.

```
□ ← x.data ==> This is data assigned thru x
```

Put something into the second object.

```
y.data ← "data assigned thru y"
```

Display it.

```
□ ← y.data ==> data assigned thru y
```

The first object hasn't changed.

```
□ ← x.data ==> This is data assigned thru x
```

Change it.

```
x.data ← "New data assigned thru x"
```

Verify that it changed.

```
□ ← x.data ==> New data assigned thru x
```

Verify that the second object didn't change.

```
□ ← y.data ==> data assigned thru y
```

Copy the reference to the first object.

```
z ← x
```

Both x and z refer to the same object.

```

x = z ==> 1
□- x.data ==> New data assigned thru x
□- z.data ==> New data assigned thru x
And y still refers to the other object.
x = y ==> 0
□- y.data ==> data assigned thru y
Switch the values assigned to x and y.
x← y
y← z
See that the two variables have swapped objects.
□- x.data ==> data assigned thru y
□- y.data ==> New data assigned thru x

```

What good is all this?

Object oriented ideas permeate modern programming. Most current languages and systems are built on objects or at least the idea of objects. The traditional APL language has no deliberate support for object oriented ideas. Simulation and emulation of objects in APL, while possible, is tedious, awkward and at best dubiously efficient. The MOM object system is a means to experiment with OOP in APL. The author expects benefits in several areas including:

- Organization of large programs. Objects provide a means to compartmentalize functions and data in a workspace.
- Simplification of control flow. Object polymorphism and indirection of reference can be used to reduce or eliminate many conditional tests to reestablish something that another part of the program has already learned.
- Clarification of design. Objects can be used to directly express the relation between problem domain entities and their representation within an application.
- Separation of concerns. An object can provide a single point of definition for a design decision. Code to access and construct data can be encapsulated with the data itself – need not be duplicated at every point in the program that uses the data.
- Even a rudimentary use of objects should help with increasingly complicated interactions with non-APL programs and host system facilities.
- The difficulty of importing commonly available skills, tools, and literature into the APL world may be reduced.

Are the current design and implementation of MOM complete?

Emphatically not. The design has reached a useful plateau. We have basic objects, means to access their members, and an incompletely implemented means to create them. The implementation of this design is definitely incomplete:

- While many APL functions handle object references as intended, many still give nonce errors and some will crash the interpreter.
- Some functions and other facilities appear to work, but misbehave around the edges – things like prototypes of empty arrays involving object references, creation and manipulation of heterogeneous results, and so forth. Display of the state indicator and error reports desperately need to include information about member functions.
- Since this part of the interpreter is still experimental, many implementation quality issues haven't yet been addressed, including memory use and other performance issues.

Does MOM's presence affect the interpreter's stability?

It's not supposed to. Some of the slowness of progress can be attributed to the effort of ensuring the existence of adequate test coverage for existing code before changing it to accommodate object references. This testing has revealed an occasional bug that had gone unreported since the dawn of time; so it might be claimed that MOM has improved stability.

If your application doesn't use objects, they should cause no trouble.

Can I safely use MOM objects in production code?

That depends of *your definition* of safely. If thorough testing of your application shows no problems and the cost of a failure is only modest embarrassment, go ahead.

What is polymorphism? Why is it useful?

For our purposes, a group of polymorphic objects have common member names but different behavior. Consider an application that maintains records for a fleet of vehicles. Cars, buses, trucks, canoes, and airplanes are unlikely to require all the same data items. However, all have an asset number and geographic location. The application is to produce reports to describe the assets in particular locations, ordered by asset numbers. A program that loops thru each of the asset records, examines various fields to determine which items to print and how to format them has a good chance of degenerating into a large tangle of conditionally executed statements. To add a new vehicle type, we risk breaking the reports for existing types.

Now suppose we arrange for each asset to be represented by an object. The objects for one type of vehicle will have just the data members needed for that type. However, each object will have function members that perform needed actions on that data. For example there might be a *FormatAssetReportEntry* member in each object. In the truck objects, the member function is concerned only with truck reports. Similarly, the canoe objects are concerned only with canoes. Such functions can be simple straight line code. All of the decisions were made once and for all when the object was constructed. The program's main loop could look something like this:

```
:while more_records
  this_vehicle← GetNextVehicleObject...
  its_report_entry← this_vehicle.FormatAsset
ReportEntry
  file Append its_report_entry
:end
```

What about types, classes, and inheritance?

Formal language support for such things is more useful in statically typed compiled languages where everything must be carefully declared so that the compiler will know what to do. The properties declared for a variable (or function) constrain the ways that the variable can be used and the operations that can be performed with it. Some constraints can be verified by the compiler, turning a potential run-time error into a compile-time error.

Dynamically typed interpreted languages don't benefit as much from declarative complexity. At this stage of its evolution, MOM relies entirely on *duck typing*. Duck typing considers the actual properties of an object to be its type. These properties are not limited to those that can be declared in mainstream languages; they can include aspects that are even difficult to express in natural language. If an object doesn't have suitable properties for the way it's used, the program is in error. Perhaps the wrong object is being used correctly or the correct object is being used incorrectly. Exercising the code shows the conflict. Appropriate testing should reveal a problem before it causes embarrassment or damage. APL programmers have worked this way for forty years.

In the polymorphism example, the important characteristics of a "vehicle" object are that it has a *FomatAssetReportEntry* member and that the member delivers an appropriate rendition of the object's state. If it walks like a duck, talks like a duck, and squawks like a duck, it's a duck. For the original definition, see www.rubygarden.org/ruby?DuckTyping. MOM ducks the typing issue entirely. Also see en.wikipedia.org/wiki/Duck_typing.

Reference

The `□MOM` System Function

This function is the sole means for object creation. Its result is a scalar object reference. Its right argument, describes the object to be created. The left argument and other forms of the right argument are reserved for future extensions.

Used nomadically with a scalar zero argument, `□MOM` returns a reference to the sole instance of the special `NIL` object. This object has no state or behavior. It can be used as a placeholder or sentinel value. All references to the null object match each other, and do not match any reference to other objects.

When the right argument is exactly a vector of character vectors, a new object is created. The character vectors in the argument must be the names of currently visible user defined functions and variables. Copies of the named functions and variables become the initial members of the new object. The object's state is not changed by subsequent erasure of the named functions and variables.

Note well that the argument must be a vector of vectors. An un-nested character vector or a character scalar won't be accepted. However, empty names are ignored. This statement creates a new empty object.

```
a_new_thing← □MOM ' ' ' '
```

This one creates an object with four members.

```
StrUtils← □MOM 'DLB' 'DEB' 'DTB' 'CONTAINS'
```

Temporary limitation:

The names in the argument may not contain leading, trailing, or embedded spaces.

The `□MSELF` System Variable

This read-only variable returns the object reference for the most recent incomplete invocation of a member function. When no call to a member function is in progress, `□MSELF` yields the special reference to the `NIL` object.

System Functions as Members

Several system functions are implicit members of every object. As member functions they refer only to the object's member functions and variables. Their arguments are analogous to those of the similarly named ordinary system function.

Temporary limitation:

Only `□CR` and the monadic variant of `□NL` are currently implemented. `□FX` is in progress but not yet operational.

The Match Function

The APL match function (`=`) shows whether two reference values refer to the same object.

Some Simple Patterns

These pattern descriptions begin with a description of a situation where use of the pattern may be appropriate. The pattern itself is the prescription beginning with the word "Therefore". The prescription may be followed by examples and other comments, sometimes including a critique of the resulting situation.

More interesting and complex patterns can be found in the literature. Try a google search for "design pattern".

Simple Collection Objects

You have an assortment of unstructured and perhaps only loosely related data. Items are needed one at a time in different parts of your program. The data items are stored in individual variables with carefully chosen names. However these names tend to clutter the global environment and the data is not readily handled as a whole.

Therefore, maintain the data items as members of a simple collection object. New items can be added as needed and the object represents the collection itself.

```
height←17◊width←37◊brdrcolor←"fuschia"...
win_props← □MOM'height' 'width' 'brdr_color'
'''
handle← OpenNewWindow win_props
```

Simple collection objects behave a lot like the hash objects in some other languages.

At least with the current implementation, you may encounter performance and memory use when the collection is large. Member names share the symbol table, which has a limited number of slots, with ordinary functions and variables.

Function Collection Objects

You would like to refactor a large and complicated function into a confederacy of smaller functions with intention revealing names. You are concerned that the

relation between the new functions will disappear in a multi-screen) fns display. You also need to avoid name collisions with existing and future global functions and want to avoid relying on distracting naming conventions.

Therefore, make the new functions members of a single object.

You could also group related utility or other functions in the same way.

Parameter Objects

You want to pass some stuff to a function, perhaps even another function. The usual APL technique is to pass the stuff as items of a nested array. The name of the parametric function would be passed as a character vector with the called function using the `⍎` (execute) primitive to actually call the function. For this to work, the calling function and called function must agree on the indices of particular items in the passed array. If several functions are involved, each with somewhat different requirements, it may take some effort to establish a consensus. Also note that a numeric index has very little mnemonic value leading to a need for documentation which tends to get out of date when changes are made.

Therefore, keep the parametric data and functions in an object's named members and pass the object to the called function, which may return a complicated result in the same object. The updated object may be passed on to other functions.

Monikers

You want to store an object reference in a context that does not allow object references such as a user defined property on a `⌈WI` or `⌈NI` object. You can store the name of a global variable, the value of which is the object reference. However, both the object references and the places for storage are created dynamically; and there may be many of them. The name of the global variable itself never needs to be used directly.

Therefore, use a utility function, such as `MonikerFor` from the example workspace, that invents a unique name and creates the global variable by assigning the object reference to the invented name.

```
some.name ⌈WI 'Δudp' (MonikerFor theObjectReference)
```

This property can then be used in a handler:

```
t← "⍎(⌈WSELF ⌈WI 'Δudp') , '.method'"
some.name ⌈WI 'onClick' t
```

A simple implementation might create a new global variable for each use. A more complicated implementation might seek to reuse names or store the object

references in an array. In the later case `MonikerFor`'s result might be something like `"MONIKERS[131]"`.

Factory Functions and Objects

You want to create multiple similar, but not identical, objects at several places in your program. The necessary code to populate the members is repetitive and distracting.

Therefore, isolate this code in a function or object, the sole purpose of which is to create and set up a new object. Large objects, or those that use a number of other objects, may be more conveniently dealt with by several functions encapsulated in an object. The OOP design pattern literature frequently calls these factory objects.

Note that the objects created by a particular factory object are analogous to the instances of a class. With duck typing the factory serves as the class definition.

When several factory functions are used together in various combinations, we have something like multiple-inheritance.

Delegation

You find that you have multiple objects that contain copies of the same member functions. This duplication seems wasteful, but more importantly, if the common function needs to be changed, it will need to be changed in many places. Furthermore, these functions are related and share a common purpose.

Therefore, create a single new object that contains the sole copy of the functions. Place an object reference to the new object in each of the original objects. Revise the calls to the duplicated functions to call the member functions of the new object. It may be useful to pass the value of `__MSELF` so that the now remote functions can access members in the calling object.

This delegation of responsibility to other objects is used as an alternative to inheritance in some object oriented languages.

Static Exception Handling for APL+Win

by William Rutiser, APL2000 (wru@apl2000.com)

Talk given at the 2005 APL2000 Users' Meeting

Introduction

This paper introduces try-catch exception handling, a feature recently added to APL+Win.

Early APL interpreters treated most exceptions by printing an error message and then suspending in immediate execution mode. A branch statement entered from the terminal caused execution to resume. Subsequent to the de facto standardization of the core APL language, each interpreter vendor introduced their own facility for dealing with errors. STSC, APL2000's corporate ancestor, added the `⌈ELX` and `⌈DM` system variables. In the event of an error, the interpreter sets `⌈DM` to the formerly printed diagnostic message, and then executes the current latent expression assigned to `⌈ELX`. That expression may include a branch statement to resume execution. Two decades of experience have shown this simple facility to be sufficient for implementing both simple and complex exception handlers.

There are two widely used patterns for the design of `⌈ELX` based handlers. One pattern relies on application wide adherence to protocols for manipulating `⌈ELX`. Local error handlers are dynamically installed by changing `⌈ELX` and removed by restoring or reconstructing the previous handler. In the other pattern, `⌈ELX` is set once and for all to invoke a global exception handling function. This relatively complicated function parses the code near the failing statement to locate and execute local handlers. In either case, effective exception handling code is often difficult to write and maintain. And its very presence can conceal the intent of its surroundings.

Like control structures, this new static textually based exception handling facility is intended to enable clearer and more reliable code.

Preliminary Notes

This paper makes no important distinction between exceptions and errors. Errors are exceptions, and the interpreter describes some exceptions as errors. Exceptions may be usefully classified as *anticipated* or *unanticipated*. Anticipated exceptions are fully understood, and are known to arise and are essential to handle during a

program's routine execution. Consider a `FILE INDEX ERROR` while sequentially reading components. It is, or should be, an anticipated event that is usually handled by terminating the reading loop and closing the file.

Contrast that error with the `XFHOST ERROR`. Its diagnostic message includes a 32 bit numeric code and a brief error description, both provided by the host operating system. After days of research you may only have learned that an unknown bad thing happened in a previously unheard of corner of the system. This is an unanticipated exception. Unanticipated exceptions are unknown or incompletely understood before they happen. They are often triggered by a defect or error in the application program or its software and hardware environment.

This paper uses several forms of the verb "to raise" to refer to the APL interpreter's initiation of an error report or exception. When passive voice is used, the interpreter is often the intended actor. In APL code, descriptions in *italic sans-serif* type replace overly specific and irrelevant details. *Italic body text* indicates both emphasis and definition.

Overview of the Try-Catch Control Structure

The try-catch control structure extends the `APL+Win` control structure family. It is designed to be compatible with and interact with existing `□ELX` handlers. Adding a try-catch structure to an existing application should not interfere with anything outside the structure. The structure's influence is confined to the statements it contains where it always takes precedence over `□ELX`. The following brief examples illustrate its two basic forms.

Catching All Exceptions

```
:Try
    A block of statements to try
:CatchAll
    A block of statements to handle any exception
:EndTry
```

The first block of statements (the *try-block*) is executed. If an exception should arise, it will be handled by the second block (a *catch-block*).

Catching Specific Exceptions

```
:Try
    A block of statements to try
:CatchIf 'FILE' ≡ 4 ↑ □DM
    A block of statements to handle just FILE errors
```

:End

The try-block is executed. If an exception arises, the expression in the :CatchIf statement is evaluated. When its value is 1, the exception is handled by the second block of statements. Otherwise, the exception is passed to the next surrounding exception handler, which may be another try-catch structure or □ELX.

Try-Catch Details

Syntax

The try-catch structure consists of a sequence of several clauses terminated by an end-statement. Each *clause* begins with a control statement which is optionally followed by a block of ordinary statements. These blocks may contain try-catch and other control structures which must be correctly nested but are otherwise unrestricted.

This skeleton structure shows all possible clauses:

```
:Try
    Statements to try
:CatchIf first condition expression
    Statements to handle exceptions satisfying the first condition
:CatchIf second condition expression
    Statements to handle exceptions satisfying the second condition
:CatchAll
    Statements to handle all other exceptions
:EndTry
```

The structure always begins with a :Try clause and ends with a :EndTry statement. As with other control structures, the keywords are case insensitive. The :End keyword may be used instead of the :EndTry keyword.

There must be at least one :CatchIf or :CatchAll clause. There may be no more than one :CatchAll clause. All :CatchIf clauses must precede the :CatchAll clause.

The :CatchIf statement consists of its keyword followed by an expression that yields 1 or 0. The other control statements consist of only a keyword.

Each control statement except :EndTry is followed by a block of APL statements (which may be empty). These blocks may contain additional try-catch and other control structures, which must be correctly nested but are otherwise unrestricted.

As with other control structures, an incomplete or incorrect try-catch structure will cause an OUTER SYNTAX ERROR.

Execution

Execution begins at the first statement in the try-block and continues from statement to statement in the usual way. When control reaches the end of the try-block, control passes out of the try-catch structure. When an exception arises within a try-block statement, the interpreter assigns the usual diagnostic message to `DM`. Control then passes, in turn, to each catch-statement until one *accepts* the exception. The interpreter's handling of an *accepted* exception is complete. If an exception is not accepted by any of the catch-statements, the interpreter passes the exception to a surrounding try-catch structure for handling. When no surrounding structure is present, `ELX` is activated.

The `:CatchAll` statement accepts any exception that reaches it.

The `:CatchIf` statement accepts an exception only if its condition expression evaluates to 1. In most instances, the expression should depend on `DM`.

When a `:CatchIf` or `:CatchAll` statement accepts an exception, the following block of statements is executed, after which, control passes out of the try-catch structure. When the block is empty, the exception is handled but otherwise ignored.

Extent of the try-block

A try-catch structure handles *only* exceptions arising within:

- Statements *textually* contained in the try-block.
- Statements contained in arguments to the execute primitive (`&`) in statements satisfying these criteria.

This recursive rule allows executes within executes.

A try-catch structure *does not* handle exceptions raised in functions *called* from its try-block. These may be handled by a try-catch structure in the function where the exception was raised or by its ambient `ELX` setting.

Exceptions arising directly in immediate execution statements, event handlers, or latent expressions are *never handled* by try-catch structures.

Exceptions in the try-catch structure itself

An exception arising in a catch-block or :CatchIf condition is immediately passed to an enclosing try-catch statement or the ambient $\square ELX$. $\square DM$ is changed to describe the new exception. The $\square DM$ message for the previous exception is lost.

Using the Try-Catch Control Structure

These examples are intended to be illustrative and suggestive rather than complete. Some conclude with a discussion of failure modes, programming style, and other pragmatic matters. The author's suggestions are stated as imperative prescriptions.

Handling a Single Specific Anticipated Error

```

▽ ProcessAllComponents ftn;i;eof
  i←1
  eof←0
  :while ~eof
    :try
      ProcessOneComponent  $\square read$  ftn i
      i← i + 1
    :catchif 'FILE INDEX' = 10+ $\square dm$ 
      eof←1
    :end
  :end
▽

```

Here, a FILE INDEX error is anticipated. A FILE TIE ERROR, probably a consequence of a coding error is not anticipated. Keep in mind that exceptions directly raised in ProcessOneComponent will not be seen by this try-catch; they may be handled by a try-catch in the called function or by its ambient $\square ELX$.

Handling Several Anticipated Errors

Applications involving trigonometry, coordinate geometry, or complex numbers may require the arc tangent of the quotient of a pair of numbers. This example deals with one tricky part of the computation – computing a result when the divisor is very small or zero. The theoretical quotient may be very large or infinite. The division operation can go wrong in two ways: division by zero raises a DOMAIN ERROR; division of an extremely large number by an extremely small number raises a LIMIT ERROR.

```

▽ z ← x principal_arc_tan y
  :Try

```

```

    z← -3 o y ÷ x
: CatchIf 'DOMAIN ERROR' ≡ 12 ↑ □DM
    z← o ÷ 2
: CatchIf 'LIMIT ERROR' ≡ 11 ↑ □DM
    z← o ÷ 2
: End

```

▽

This fairly simple example illustrates a pitfall that must be considered for all exception handling code, in every language, using any exception handling features. This example does correctly catch a DOMAIN or LIMIT ERROR raised in the divide primitive, however this try-catch structure would also catch the same errors if raised by the circular functions primitive. The author is only "pretty sure" that the code is safe as written. But this is an example and not an extract from a real application where the try-block could be much larger and more complicated and consequently harder to analyze with any certainty.

Therefore, when handling specific exception possibilities make the try-block as small and simple as practical to reduce the risk of errors in your analysis of possible exceptions.

The text of □DM contains unused information about the exception's source. It would be possible, with suitable support utilities, to parse the entire message to verify that the caret points to the divide operation.

Therefore, make the catch-if selections as specific as necessary to exclude unanticipated exceptions.

Handling Unanticipated Errors in a Transaction

```

▽ AttemptOneTransaction details
: Try
    Statements to do one transaction.
: CatchAll
    □← "Something bad has happened!!"
    □← "□DM is:"
    □← □DM
    □← "Abandoning this transaction and"
    □← "continuing with the next."
    z← "Failed"
: Return
: EndTry
z← "OK"
: Return

```

▽

Ignoring All Exceptions

This example is supposed to compute the value of a function `Foo` with given arguments. The code to do this raises one or more known exceptions for some argument values. A specified default result value is to be returned when such an exception arises.

```

v Z← default_result FindFooWithDefault args
  Z← default_result
  :Try
    Statements to compute and set Z to Foo(args)
  :CatchAll
  :EndTry
v

```

The `catchall`-statement accepts the exception but its empty handler does nothing, leaving `Z` set to the desired default result.

Using a catch-all clause, empty or not, to handle anticipated exceptions is somewhat risky unless you are certain that no unanticipated exceptions are possible. Therefore, reduce the risk by handling anticipated exceptions as specifically as practical.

Notice that `Z` is set to the default result before the `try`-block is executed with the expectations that the `try`-block will reset it to the correct computed value or leave it unchanged otherwise. A programmer could easily and mistakenly use `Z` for an intermediate result; after all, it is *always* assigned a value a few lines (or pages) down the function. This mistake becomes more likely as the `try`-block becomes larger and the program is passed on to other maintainers. Moving the default result assignment to the `catchall`-block prevents this particular mistake; it also clarifies the assignment's purpose.

A root cause of this programming error is that there are *invisible* control paths leaving the `try`-block. Overlooking such paths may invalidate any conclusion based on examination downstream code.

Therefore, always remember that every primitive operation in the `try`-block potentially contains an invisible branch to the `catch`-block. Practice defensive programming where practical.

Handling Exceptions in Called Functions

This example illustrates a simple global handler for unanticipated errors. It also illustrates `⌈ELX` and the `try-catch` structure working together. Only two called functions are shown here. Imagine that there are many others.

```

▽ FooOne
  Other stuff
  FooTwo
  More other stuff
▽
▽ FooTwo
  Even more other stuff
  "two" + 2
  And more other stuff
▽

```

Calling FooTwo will yield a DOMAIN ERROR. This example just displays the error message and a simple trace-back.

```

▽ CatchDeepErrors;□ELX
  □ELX← " □Error '@ ' , □DM"
  :Try
    FooOne
  :CatchIf '@' = ↑ □DM
    □← □(~ □DM = □TCNL) □ □DM
  :CatchAll
    □ERROR "PANIC"
  :End
▽

```

Execution produces:

```

      CatchDeepErrors
@ @ DOMAIN ERROR
FooTwo[??]  "two" + 2
           ^
FooOne[??]  FooTwo
           ^
CatchDeepErrors[3]  FooOne
                   ^

```

At each SI level, □ELX handles an error by raising a new exception in the preceding SI level. The "@" characters indicate the number of stack levels traversed and distinguish exceptions raised in the try-block from those raised in the called functions. Simple parsing of the accumulated diagnostic messages provides the trace-back.

Some programming language designers have chosen "throw" as the keyword in a statement that programmatically raises an exception. This example *throws* an exception up (or down as the reader prefers) the call stack.

Intercepting an Exception Being Thrown Up the Stack

Suppose, in the preceding example, that `FooOne` normally takes some action to release a resource before returning.

```

▽ FooOne
  Open a file
  FooTwo
  Close the file
▽

```

When an exception is thrown up from `FooTwo`, the file will be left open, perhaps causing trouble in an unrelated part of the application. We can fix this by involving `FooOne` in the exception throwing.

```

▽ FooOne
  :Try
    Open a file
    FooTwo
    :CatchIf '@' == ↑ □DM
      Close the file
      ↓ □ELX
  :End
  Close the file
▽

```

`FooOne` has been changed to catch the exception that the latent exception throws from `FooTwo`. After closing the file, `FooOne` executes `□ELX` to throw the exception to the next level.

In a real application, careful attention should be given to the possibility of something else going wrong while closing the file. This example does not handle exceptions raised while opening or closing the file.

Readers familiar with some other popular programming languages will see this example as one where a `:Finally` clause would be helpful. Such an enhancement is under consideration.

Another Way to Emulate a `:Finally` Clause

In this example, we give the global `□ELX` an additional responsibility. It must execute the value of the variable `ΔFINALLY` at each level.

Change `CatchDeepErrors` to set `□ELX` with:

```
DELX← " ⚡ ΔFINALLY ⚡ □ERROR '@ ' , □DM "
```

With this in place we can use it in FooOne.

```
▽ FooOne;ΔFINALLY
  ΔFINALLY← " Close the file "
  Open a file
  FooTwo
  ⚡ ΔFINALLY
▽
```

Again we should think about problems closing the file.

This idea can be extended and improved in a number of ways. There is a potential VALUE ERROR in the reference to ΔFINALLY. Since the variable will be empty in functions that don't need its effect, it would be only a small matter of programming to make it optional. The magic variable could be replaced with a magic label or private comment in the function itself. Most of the □ELX code should be moved into a new function but the □ERROR invocation must remain in the latent expression. There will be the usual issues with shadowed variables to resolve.

Suspending Execution in a Handler

This trick doesn't really involve exception handling except that a catch-block is a likely place to use it. While experimenting or debugging it may be useful for a program to programmatically suspend for immediate execution. This shows one way to do that. When active, Code Walker will show the stop suspension. The statement label gives, by convention, a convenient common branch target when resuming execution.

```
▽ Suspend
  :Try
    x "do" "something" "wrong"
  :CatchAll
    0 0 ρ (1 + † □lc) □stop 'Suspend'
GOON: 0 0 ρ "place to stop"
  :End
▽
```

The stop could be set manually; however, it would need to be set manually whenever the function is edited or otherwise redefined.

Retrying After Correcting an Exception

Some situations require that the try-block be retried after somehow improving the chances of success – perhaps by creating a missing file or waiting for a shared resource to become available.

```

▽ TryAndTryAgain
  :Try
  RETRY:
    Do something
    :CatchIf some condition
      Fix the problem
      :GOTO RETRY
    :End
▽

```

This use of the dreaded :GOTO statement forms a loop. It will never end unless the problem is really fixed. Therefore, consider using a counter or timer to limit retries.

```

▽ TrySeveralTimes retry_limit; retry_count
  retry_count ← 0
  :Try
  RETRY:
    Do something
    :CatchIf retry_count ≥ retry_limit
      □ERROR "Retry limit exceeded"
    :CatchIf some condition
      Fix the problem
      retry_count ← retry_count + 1
      :GOTO RETRY
    :End
▽

```

The first catch-if condition is unusual in that its condition doesn't refer to □DM.

General Advice

This section, like the suggestions after the examples above, expresses the author's advice, perhaps with more certainty than is really warranted.

Serious exception handling may be hard. This is particularly so for exceptions initiated outside the APL interpreter. Correct handlers depend on detailed knowledge of the source and causes of the exception to be handled. Sometimes, such knowledge is not available and may be obtained only through research and experimentation.

Therefore, carefully consider the costs before attempting to handle situations that can't be readily reproduced. Where possible, fix root causes instead of trying to recover from their manifestations. Remember that a clean termination is usually better than continuing after a bungled recovery.

Handlers for unanticipated exceptions should preserve evidence.

Handle unanticipated exceptions globally; handle anticipated exceptions locally.

Prospects

Prior to this conference and this paper, the try-catch structure has had neither documentation nor significant use in real programs. Reports of testing and experiments by conference participants will help to improve try-catch in its production quality release. Experience and bug reports may be sent to support@apl2000.com. General comments, suggestions, and criticism may be sent to the author at wru@apl2000.com. (Cash bribes included in an email message envelope are not guaranteed to reach the author.)

Some extensions being considered:

- A `Finally` clause.
- A variant of catch-if with pattern matching capability.
- Enabling the use of `AndIf` and `OrIf` statements in catch-if clauses.

Some Plagiarized Wisdom

Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it? — *Brian Kernighan*

There are two ways of constructing a software design. One way is to make it so simple that there are obviously no deficiencies. And the other way is to make it so complicated that there are no obvious deficiencies. — *C.A.R. Hoare*

Dead programs tell no lies.

Don't publish papers when you have no experience with the subject.

Pay no attention to the grumpy old programmer behind the curtain.

Up, Down and Sideways: Extending the Reach of APL

*by Richard Nabavi Managing Director, MicroAPL Ltd
Talk given at the "All APLs Day" at Naples Beach*

APLX, MicroAPL's cross-platform APL interpreter and development environment, was introduced in 2001, although its roots go back many years. One of our goals in developing the product was to extend the reach of APL by providing a solution which would cover a wide range of users' needs, and overcome some of the obstacles which have prevented people using APL. Four years later, it seems appropriate to review how we are doing in attaining this goal, and to look at how we intend to respond to recent changes in the marketplace.

Extending sideways, by supporting all the major platforms

APLX was launched simultaneously on Windows and MacOS, and as a Server Edition on AIX and Linux. Subsequently, a desktop edition under Linux was introduced, so that all the main desktop environments are supported.

The key point here is compatibility. APLX behaves virtually identically on all the platforms. A workspace saved on any platform can be)LOADed on any other - and will run almost always without change, even if it makes full use of the `⎕W` system function for programming user-interface. Component files can be shared across different platforms. In addition, APL code can be easily ported from other APL implementations, especially APL2 and APL+Win, further extending the reach of the product.

One important platform, however, was missing: Solaris. We are now announcing that APLX will be available on Solaris from the 1st quarter of 2006, making APLX a near-universal solution.

Extending sideways, by making the product easy to use

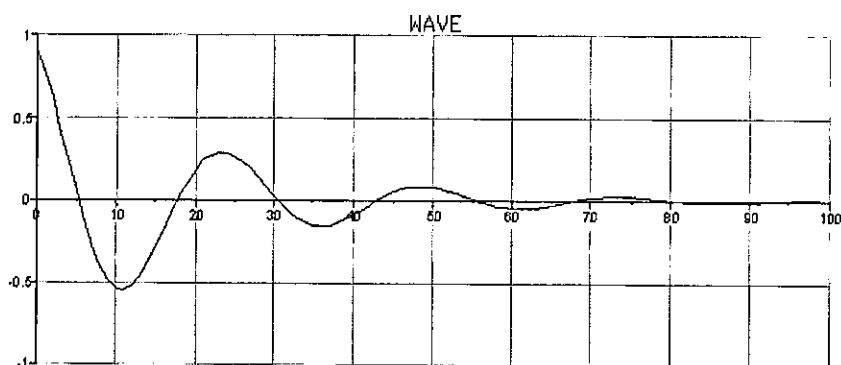
APL has always been a wonderful language for exploring data and experimenting with algorithms. Perhaps, indeed, that is its greatest strength. The original design of APLX aimed at a user interface which would provide very powerful development facilities, without any distracting on-screen clutter.

In the recently-launched Version 3 of APLX, we added an important feature which extended this still further: built-in charting. At its simplest, you can use a

new 'right-click' menu item to bring up an instant chart of an APL variable. For example, if you type:

```
WAVE←(2×0.25×1100)×*-0.05×1100
```

and then right-click on the variable name WAVE, there is now a new 'Display as Chart' item in the pop-up menu. Selecting that item brings up a window containing an instant graph of the data:

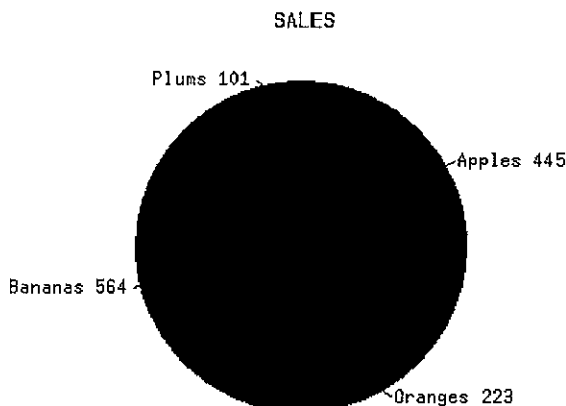


If the variable is a matrix, APLX will make an intelligent guess about which rows or columns are the Y values and which (if any) the X values, but you can use the Chart menu to change these parameters, as well as selecting logarithmic scales or changing the type of the graph (for example, to a Scatter or Bar chart). If the chart is a nested matrix of strings and numeric values, the strings are used as labels of a bar chart. You can change this to a pie chart using the menu. For example, if you type:

```
SALES←2 4p'Apples' 'Oranges' 'Bananas' 'Plums' 445 223 564 101
```

and then use the pop-up menu item Display As Chart, the default will be to draw a Bar chart.

You can use the Chart menu to change this to a pie chart:



If you want to take this a bit further, the new `⎕CHART` system function allows the APL programmer to produce quick charts with almost no programming. The right argument is the data, and optional left argument allows you to specify how the data should be displayed. If you do not specify, APLX makes an intelligent choice for you. For example:

Draw a sine wave. Because no X values are specified, implicit X values of 0, 1, ..., 99 are used:

```
⎕CHART 100.1×1100
```

Draw a sine wave with the X values specified in degrees. We provide a two-column matrix; APLX assumes the second column is the X axis, because it increments regularly:

```
⎕IO←1
⎕CHART (10(10×0,136)×02÷360),[1.5]0,10×136
```

Draw a bar chart:

```
X←2 4p'Apples' 'Pears' 'Oranges' 'Plums' 445 323 345 765
⎕CHART X
```

Draw the same data as a pie chart:

```
'type=pie' CHART X
```

Draw some random data as two, named series, against implicit X values 0, 1, 2..:

```
'x=implicit' 'seriesname=US' 'seriesname=EU' CHART ?2 8p100
```

For the most control over the appearance of the graph, you can use a new **OWI** object class, the **Chart**. This is a control which can display a range of different business, statistical and scientific graphs. The actual data which is displayed in the graph is held in one or more **Series** objects.

Although you can customize many aspects of the display of a **Chart** object, it is designed so that it automatically selects defaults which should give a good appearance for the chart with minimal programming. For example, if a chart is re-sized, by default the font sizes are adjusted appropriately, and reasonable values for the chart scale, tick positions, and labels are automatically chosen.

Charts by default are displayed in color. You can select your own colors for all the major elements of a chart, or you can leave the **Chart** object to select defaults. You can also ask for the chart to be displayed in monochrome.

As well as being displayed within your **OWI** windows, charts can also be printed, copied to the clipboard, and saved to file in various formats. You can also customize a chart by adding your own graphics elements using the **Draw** method.

Extending sideways, by interfacing to external programs and data sources

To be useful, APL systems need to exchange data and interface with other software. APLX includes a comprehensive range of such interfaces, including OCX/ActiveX controls, built-in networking through Sockets, E-mail Clients, an HTTP Client, and a **OWI** Web Browser control. All of these are very easy to use, and all (except the Windows-specific OCX/ActiveX interface) work identically on the different platforms.

New in Version 3 is the **SQL** system function. This provides a built-in, extensible, cross-platform interface to standard relational databases such as Oracle, SQL Server, IBM DB2, MySQL, or PostgreSQL, using Structured Query Language (SQL). It can also be used (via ODBC) to access data from spreadsheets, Microsoft Access, db2 files, and many other data sources. You can use it to retrieve data

from relational databases as nested arrays in the workspace, or to update a database directly from within APL.

In this sequence, we are using the sample database supplied with IBM's DB2:

Connect to the database. The return code 0 0 0 0 means no errors occurred:

```
      □SQL 'Connect' 'aplxdabc' 'DB2'
0 0 0 0
```

Retrieve the number of rows in the 'emp_photo' table:

```
      (RC ERRMSG DATA)+□SQL 'Do' 'select count(*) from emp_photo'
      RC
0 0 0 0
      DATA
12
```

No errors occurred, and the result was 12. We could retrieve all 12 rows immediately using the 'Do' keyword, but instead we'll prepare a database statement which allows more control over the retrieval, as well as allowing us to see the types and names of the columns:

```
      □SQL 'PREPARE' 'CS1' 'SELECT * FROM EMP_PHOTO'
0 0 0 0
      □SQL 'EXECUTE' 'CS1'
0 0 0 1  SQL_CURSH200C4
      □SQL 'DESCRIBE' 'CS1'
0 0 0 0  EMPNO PHOTO_FORMAT    PICTURE
          EMPNO PHOTO_FORMAT    PICTURE
          CHAR 6   VARCHAR 10 BLOB 102400
              0             0             1
```

We'll fetch the first five rows of the result:

```
      (RC ERRMSG DATA)+□SQL 'Fetch' 'CS1' 5
      pDATA
5 3
    2 ↑[2]DATA
000130 bitmap
000130 gif
000130 xwd
000140 bitmap
000140 gif
```

The third column of the result contains binary data (pictures of the employees). At this point, we could display the pictures using a Picture object in □WI.

Close the statement, and disconnect from the database:

```
(RC ERRMSG DATA)+SQL 'Close' 'CS1'  
(RC ERRMSG DATA)+SQL 'Disconnect'
```

Extending downwards, by making the product easily available

One of the obstacles to the use of APL has been the relatively high price of most APL interpreters. To help address this, we have kept the price of APLX competitive. However, that is not enough; we also give away an earlier (Version 1.1) Linux version of APLX, absolutely free of charge for personal use. This has proved very popular, and now has many hundreds of users all over the world.

We also make available time-limited but fully-functional demonstration copies of our Windows and MacOS APLs. These can be downloaded from our website, and operate for slightly over a month. This has proved very popular. Indeed, it has had an unexpected but very welcome effect; a number of universities include an introduction to APL in their undergraduate courses, and the students download one of our demonstration APLs to get some familiarity with the language.

Extending upwards: The new 64-bit world is here!

The launch by AMD of a 64-bit version of the x86 architecture, a couple of years ago, has opened the way for low-cost, 64-bit systems in the mass market. Today, you can buy a desktop machine - or even a laptop - with a 64-bit processor, for just a few hundred dollars. Intel have now followed AMD with a binary-compatible equivalent, and, over the next few months, we can expect 64-bit x86 systems to become commonplace. Microsoft has a full 64-bit version of Windows available to run on these systems, and Linux is of course also available. In addition, Apple has sold 64-bit Macintoshes, based on the PowerPC architecture, for some time. 64-bit Servers are, of course, well established.

Such machines can address extraordinary amounts of memory. 32-bit systems are limited to at most 4GB of memory address space, and usually this is not all available to user processes (Windows, for example, normally has a limit of 2GB). Today, XP Professional x64 Edition supports up to 128GB of RAM and 16 Terabytes of virtual memory. In the future, even more memory will be usable. And the cost of RAM has fallen to such an extent that it is already possible to configure desktop machines with tens of gigabytes of memory for a couple of thousand dollars.

APL traditionally has been at the forefront of making full use of available memory. Back in 1982, MicroAPL made a big splash with the 68000-based

MicroAPL Spectrum computer, which could offer a 1MB workspace size or more, at a time when nearly all micro-computers were limited to just 64KB. Now we are talking about memory sizes several thousand times bigger, but we see the same benefits: direct manipulation, in the APL workspace, of data which other languages have to process piecemeal.

Of course, it is not just the size of workspace which matters; it is also the maximum size of arrays. 32-bit APL systems use 32-bit slots to hold array sizes and dimensions. Thus, even if the processor can address more than 32-bits, it does not follow that an application (such as an APL interpreter) can handle arrays which will take full advantage of this memory.

For this reason, we have decided to introduce a fully 64-bit version of APLX, which will overcome all limitations in workspace and array size for the foreseeable future. APLX64 is this new product. It is initially being made available for Linux and Windows, with a MacOS version following. In APLX64, all array dimensions are 64-bit. In addition, integers are also 64-bit (otherwise you would have to use floating-point numbers to index arrays!). Booleans remain as one bit per element, making it possible to handle huge Boolean arrays without excessive memory requirements.

We believe this product extends the reach of APL upwards to applications which previously were beyond its reach. These include modelling and simulation using huge data sets, and OLAP applications for analysis and aggregation of large volumes of transactional data. In this new world, you do not need to compromise: you can just load the entire database into your APL workspace (the `SQL` system function will be handy here!). In some ways, this is taking APL back to the kinds of application where it used to be strong, but where it lost ground because the amount of data became too big to fit in the workspace.

Conclusion: Up, Down, and Sideways

Extending the reach of APL means ensuring that it is appropriate to a whole range of users and different needs, from the student learning about APL on a low-cost PC, to a retired Professor of Economics analysing stock-market data, to commercial programmers writing cross-platform solutions, to big corporations analysing huge volumes of commercial data.

And the beauty is, all of this can be done with the same product, using the core language specification of APL2, with the same `WI`-based user-interface components, and running in the same way on a whole range of different processors and operating systems.

In Search of a Cool Smooth

*by Adrian Smith and Jonathan Manktelow, Causeway
Talk given at the "All APLs Day", Naples Beach, 2005*

Outline

Rather than bore everyone with yet more PowerPoint and chat, Jonathan and I decided to act out a little scenario, illustrating one possible approach to mixing traditional APL development with the .Net future. I should point out that we picked the example quite carefully; everything from the APL side translated easily into C# and tripwires were avoided. The C# code was mostly written ahead of time, but nothing was hidden, and everything we did was genuine enough on the day. Just be aware that it is not quite as easy as we made out!

Cast in Order of Appearance

Jonathan – the unfortunate C# programmer with a particularly irritating boss.

Jonathan's boss (non-speaking part) – who likes to think that a bit of bluster and some arm-waving are equivalent to a detailed specification.

Adrian – the mad boffin with the cranky old APL system tucked away below the company radar.

The Challenge

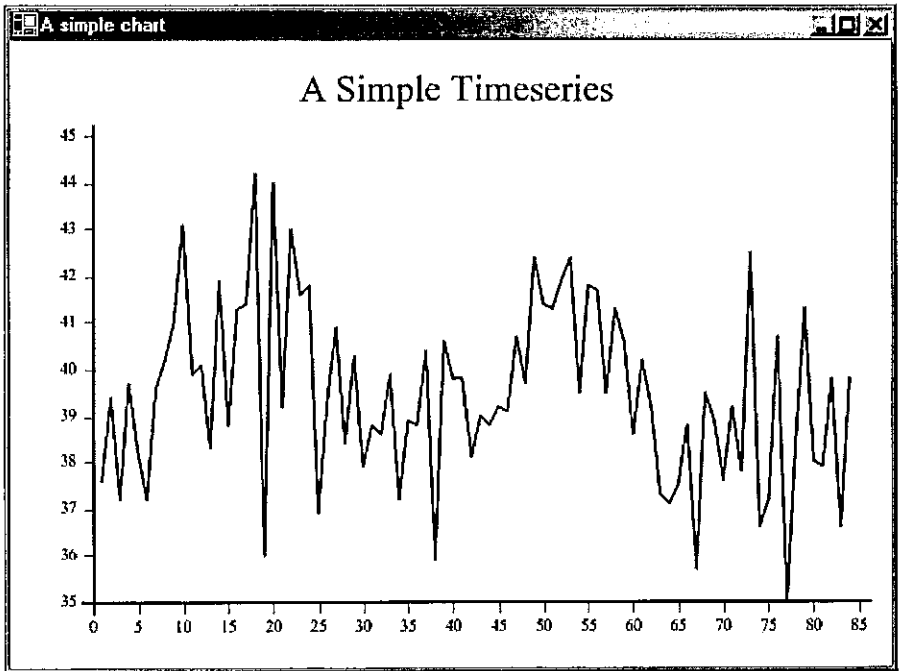
Jonathan's boss is in charge of a project to add an MPG summary to the typical in-car computer system to go alongside the SatNav and all the other fancy toys. He has collected some sample data, and given Jonathan the task of developing a nice timeseries graphic so the car-owner can see how things are progressing over time.

Jonathan winds up the marvellous *SharpPlot* (aka *RainPro*) charting toolkit and proudly shows his boss the results.

```
public Metafile DrawChart()
{
    double[] data = new double[] {37.6,39.4,37.2, .... ,39.8,36.6,39.8};

    SharpPlot.Heading = "A Simple Timeseries";
    SharpPlot.DrawLineGraph(data);
    return SharpPlot.RenderMetafile();
}
```

The data is quite genuine – and gives you a good picture of the seasonal variations in fuel usage, as well as the large random errors introduced by inconsistent filling!



"Wow" says the boss, "You did all that coding in less than a day". It actually took Jonathan 10 minutes, so he tries not to look too smug.

"What we really need" muses the boss "is some kind of cool smooth". He waves a boss-like finger in an up and down kind of way at the data; "See what you can do by this afternoon – I promised to show this to a couple of customers at 3:00".

Jonathan breaks out in a cold sweat, and goes Googling for "cool smooth" which doesn't really help. Then he has a brainwave – maybe that APL nut who hides out on the 5th floor can help. He knows about this sort of thing.

The APL Approach

The APL nut is quite used to users who have no idea what they want. He starts by looking at the data, and mentally discarding concepts like $12+/\$ as far too uncool. Simple moving averages never make a tidy job of the 'ends' and their intrinsic

'top-hat' shape often leads to all sorts of strange artifacts as the averaging period 'beats' with the data.

How about a simple Gaussian? That's pretty cool, and easy to do in APL. Let's have a look ...

```
wt←*-(xd×dieaway)*2
wt←wt÷+/wt
```

... will create a weighting that falls off in a pretty 'bell-curve' way depending only on the absolute x-distance. It is nice, in that every point in the series contributes something to every point on the average, but nearer points count most. The dieaway controls how wide a bell we use – it ranges from 0 (every point has equal weight) to about 5 when the average sees very little other than the nearest point. It looks like a good candidate for a trackbar to let the user play with the settings!

After a few minutes fooling around, the APLer has come up with this:

```
▽ xy←dieaway flex vec;gx;ct;xd;wt;mm
[1] A Generate Gaussian smooth from data array
[2]
[3] mm←1,pvec
[4] gx←mm[1]+(0.01×0,1100)×--/mm A 100 is hard-wired for now
[5] xy←gx,[1,5]0 A Pre-allocate
[6]
[7] A May as well loop here - outer product is a memory hog
[8] :for ct :in 1pgx
[9]   xd←1(1pvec)-gx[ct]
[10]   wt←*-(xd×dieaway)*2
[11]   wt←wt÷+/wt
[12]   xy[ct;2]←vec+.×wt
[13] :end
[14]
▽

1 flex 4 5 6 5 4
1 4.291876722
1.04 4.31099321
1.08 4.331030772
1.12 4.351994593

.....

4.92 4.331030772
4.96 4.31099321
5 4.291876722
```

So we have a piece of working code, and some simple test-data. "Great" mutters Jonathan, looking bemused. "How do I call this from C#?".

With one mighty bound, he was free ...

It is Jonathan's lucky day – Adrian has been playing around with a deceptively simple C# conversion tool that can take APL utilities like this and spit them out as compiled DLLs that any .Net application can call. First, he must agree the 'interface' with Jonathan and then add a few extra comments to the APL code to assist in the translation process.

They decide that the class will be called **CoolSmooth** (to conform with Microsoft naming style) and that it will have a method called **Flex**. Rather than passing in the dieaway as an extra argument, Jonathan decides it would be good to have a **Flexibility** property, ranging from 10 to 100 (because users like percentages) with a default value of 10. He is very uncomfortable with the idea of getting the result as a matrix (C# programmers much prefer arrays of arrays) so they agree that **Flex** can return a 2-element array with the X and Y values needed to plot the smooth.

Here is what Adrian needed to code to make this all work:

```

    ▽ CoolSmooth flex
[1]  AAs Constructor for CoolSmooth taking optional flexibility (default 10)
[2]  A:Constructor():this(10)
[3]
[4]  Δflexibility←flex
    ▽

    ▽ SetFlex newvalue
[1]  A Set the Flexibility adjuster (constrained 0-100)
[2]  A:PSet Flexibility Set responsiveness of fitted Gaussian
[3]  A: int newvalue
[4]
[5]  Δflexibility←100[0]newvalue
    ▽

    ▽ value←GetFlex
[1]  A Get the Flexibility adjuster
[2]  A:PGet Flexibility Responsiveness of fitted Gaussian
[3]  A: int value
[4]
[5]  value←Δflexibility
    ▽

```

```

v flexed←Flex vec;dieaway;gx;xy;ct;xd;wt;mm
[1]  A Generate Gaussian smooth from data array
[2]  A Creates 100 points spanning range of original data
[3]  A Uses a gaussian curve weighted by the x-distance from each point.
[4]  A:Public
[5]  A: double[] vec
[6]  A: double[][] flexed
[7]
[8]  dieaway←0[0.01×Δflexibility  A Default parameter = 10
[9]
[10] mm←1,pvec
[11] gx←mm[1]+(0.01×0,t100)×--/mm  A 100 is hard-wired for now
[12] xy←gx,[1.5]0  A Pre-allocate
[13]
[14] A May as well loop here - outer product is a memory hog
[15] :for ct :in 1pgx
[16]   xd←|(1pvec)-gx[ct]
[17]   wt←--(xd×dieaway)*2
[18]   wt←wt÷÷wt
[19]   xy[ct;2]←vec+.×wt
[20] :end
[21]
[22] flexed←c[1]xy
v

```

Let's look at the `Flex` function first, to see what changed from the original APL code. Line[22] simply changes the result into an array of arrays, and line[8] uses a global variable instead of a left argument. That leaves lines [4-6] which appear to APL as comments, but are used by the code-conversion to give data-types to the argument and result. By default, all functions in a class are 'private' (and invisible from outside) so we need a hint that this function is a public 'method' of the class.

Hopefully, the two functions to *Set* and *Get* the **Flexibility** are simple enough to be self-evident. The converter looks for comments like:

```
[2]  A:PSet Flexibility
```

and uses these to create Properties in the generated C# code. The property will be available for Read/Write/Both depending on which functions are marked (the actual function names do not matter at all).

In order to compile the converted APL code, we need a small amount of C# boilerplate, which is saved in a simple text variable:

```

        csΔflex
using System;
using AE=Causeway.SharpArrays;

namespace Causeway
{
    !CoolSmooth=Flex,SetFlex,GetFlex,CoolSmooth,Δflexibility /doc
}

```

This mostly just passes straight to the generated source-code, except for lines beginning *!ClassName* which invoke the translator with a list of function and variable names. The */doc* flag makes the translator create the XML documentation that Visual Studio uses for popup tips and auto-completion of code.

Here is the complete C# source:

```

using System;
using AE=Causeway.SharpArrays;

namespace Causeway
{
    public class CoolSmooth
    {
        int _flexibility = 10;

        // ===== Flex =====
        // Generate Gaussian smooth from data array
        // Creates 100 points spanning range of original data
        // Uses a gaussian curve weighted by the x-distance from each point.

        public double[,] Flex(double[] vec)
        {
            int[] mm;
            double dieaway;
            double[] gx,xd,wt;
            double[,] xy;

            dieaway = AE.Max(0,0.01 * _flexibility); // Default parameter = 10
            mm = AE.Join(1,vec.Length);
            gx = AE.Plus(mm[0],AE.Times(AE.Times(0.01,AE.Join(0,AE.Range(100))),
                -AE.MinusReduce(mm))); // 100 is hard-wired for now
            xy = AE.Laminate(1.5,gx,0); // Pre-allocate
            // May as well loop here - outer product is a memory hog
            for (int ct = 1; ct <= gx.Length; ct++) {
                xd = AE.Magnitude(AE.Minus(AE.Range(vec.Length),gx[ct-1]));
                wt = AE.Exponential(AE.Negate(AE.Power(AE.Times(xd,dieaway),2)));
                wt = AE.Divide(wt,AE.PlusReduceFirst(wt));
                AE.IndexAssign(xy,ct,2,AE.SumTimes(vec,wt));
            }
            return AE.MatrixToRagged(AE.Transpose(xy));
        }
    }
}

```

```
// ===== SetFlex =====
// Set the Flexibility adjuster (constrained 0-100)

void SetFlex(int newvalue)
{
    _flexibility = AE.Min(100,AE.Max(0,newvalue));
}

// ===== GetFlex =====
// Get the Flexibility adjuster

int GetFlex()
{
    return _flexibility;
}

// ===== CoolSmooth =====
public CoolSmooth():this(10) {}
public CoolSmooth(int flex)
{
    _flexibility = flex;
}

// ===== Properties of CoolSmooth =====
public int Flexibility {
    set { SetFlex(value); }
    get { return GetFlex(); }
}
}
}
```

And here is the accompanying XML:

```
<?xml version="1.0"?>
<doc>
  <assembly>
    <name>flex</name>
  </assembly>
  <members>
    <member name="M:Causeway.CoolSmooth.Flex(System.Double[])">
      <summary>
        Generate Gaussian smooth from data array
      </summary>
    </member>
    <member name="M:Causeway.CoolSmooth.#ctor(System.Int32)">
      <summary>
        Constructor for CoolSmooth taking optional flexibility (default 10)
      </summary>
    </member>
    <member name="P:Causeway.CoolSmooth.Flexibility">
      <summary>Responsiveness of fitted Gaussian</summary>
    </member>
  </members>
</doc>
```

The source-code can be compiled from the command prompt like this:

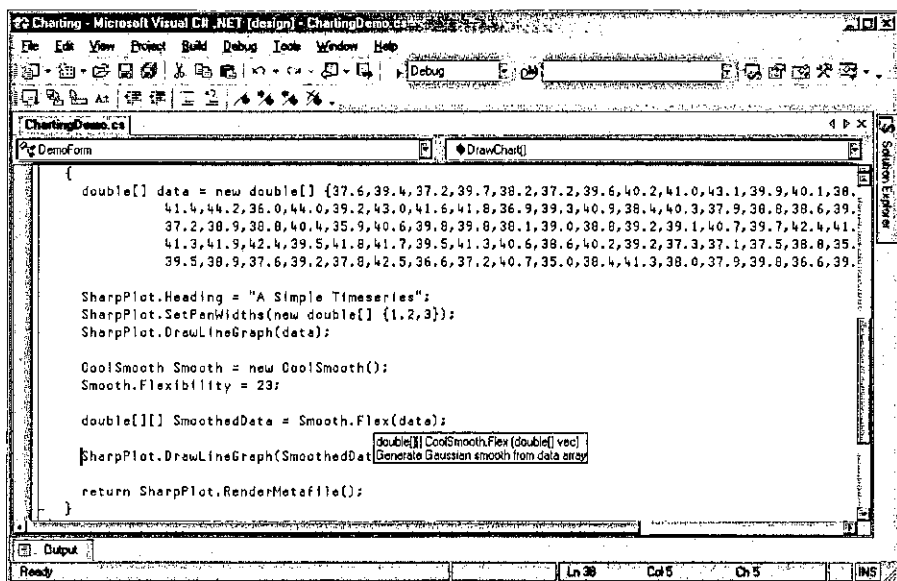
```
D:\Tools\aplc>csc /t:library /r:sharparrays.dll flex.cs
Microsoft (R) Visual C# .NET Compiler version 7.10.6001.4
for Microsoft (R) .NET Framework version 1.1.4322
Copyright (C) Microsoft Corporation 2001-2002. All rights reserved.
```

```
D:\Tools\aplc>dir flex.*
30/01/2006 23:22          1,958 flex.cs
30/01/2006 23:25          4,096 flex.dll
30/01/2006 23:22           582 flex.xml
```

So Adrian copies flex.dll, flex.xml and sharparrays.dll (just under 500Kb) to his handy APL2000 keydrive and hands it over to Jonathan to plug in to the finished application. Yes, I'm sure Jonathan could have coded it up in C# in a couple of days, but this is a classic example of the APL mode of thought in action. Adrian never sat down and designed the algorithm – he just created some handy test data and typed away until the output looked right.

Happy Ending

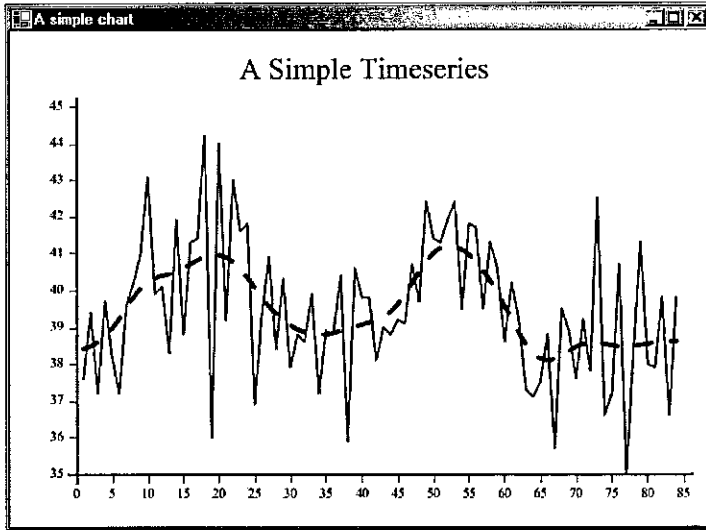
Calling the new DLL from C# is a breeze:



Jonathan added the lines to create an instance of the CoolSmooth class and set the Flexibility to 23 – of course he could have passed this as an argument to the

constructor. The screen-snap was taken with the mouse hovering over the Flex method name – notice how the leading comment-line from the original APL function has appeared as the second line of the tip! This is why we made the XML file to accompany the DLL. If you look carefully, you can see exactly where the ‘intellisense’ is coming from for the methods and properties.

So let’s test it – hit that F5 key to compile and run ...



... well, the audience thought it was cool! This has to be the first time a simple dashed line got a spontaneous round of applause.

Summary

My feeling is that .Net is beginning to compete on level terms with APL as a general-purpose application development tool. C# is a good language, the Framework is an excellent utility library, and the Gui designer is reasonable. When we have Framework-2 and XAML for the Gui, then I think C# may be ahead. What .Net is *not* (at least not yet) is an environment for mucking around with ideas. For that you need a **session** and a **workspace**, and an environment that lets you stop halfway through a function, play with the data, fix up the next line and **continue**.

The little scenario Jonathan and I played out illustrates a useful role for APL in even the most conventional development shop.

Objects for Dyalog APLers (Part 2 of 3)

by Morten Kromberg

Chapter 5. Properties

The first four chapters of the *Introduction to Object Oriented Programming For APL Programmers* were published in Vector 22.1. With luck, we are now able to find our way around simple classes and instances, and are ready to explore the next piece of OO functionality.

A *Property* is a member which is *used* in the same way as a field, but *implemented* as a one or more functions: There is usually a pair of functions, which are commonly referred to as the *getter* and the *setter*.

To explore properties, we are going to modify our `TimeSeries` class so that an instance represents a year of monthly data. In case we do not have observations for all the months, we would like to be able to indicate which months we do have data for:

```
:Class Monthly

:Field Public Obs      A Observed values
:Field Public X        A X values for observations (Months)
:Field Public Degree+3 A 3rd degree polynomials by default
▽ r←PolyFit x;coeffs
  :Access Public
  A Use cubic fit of observed points to compute values at x
  coeffs←Obs⊖X°.×0,1Degree A Find polynomial coeffs
  r←coeffs+.×0x°.×0,1Degree A Compute fitted f(x)
▽

▽ make args
  :Implements Constructor
  A args: Obs [X]
  args+,(c×(1=⊂args))args A Enclose if simple1
  Obs X+2fargs,c1pargs
  'Invalid Month Numbers' ⌈SIGNAL (⌈/X⌈112)⌈11
▽

:EndClass A Class Monthly
```

¹ `⌈` is the power operator, which takes a function (enclose) on the left, number of applications (1 if simple, else 0) on the right. When right operator argument is boolean, power can be read "if", in this case "enclose if simple".

Bold text highlights the changes: We have added a new field called *X*, which will contain the *X* co-ordinates of our observations. The *PolyFit* function has been enhanced to use *X* rather than (*tpObs*) in constructing the right argument to matrix division. The constructor *make* has been enhanced so that it generates *X* if it was not provided:

```

      (NEW Monthly (1 4 7 9 10)).X A Instance discarded at end of line
1 2 3 4 5

      m1←NEW Monthly ((1 4 7 9 10)(1 3 5 7 9))
      m1.X
1 3 5 7 9

```

Because we think we might extend our class to handle very long timeseries at some point in the future, and since the matrix division algorithm only uses the actual observations anyway, we have decided to use a “sparse” data structure: We only store the points for which we have values.

We will experiment with defining a variety of properties which can present the sparse data in ways which may be more convenient to the user.

:Property Simple

A reporting application is likely to want to see the data as a 12-element vector, so it can be tabled with other series and easily summarized. We can support this requirement using the following property:

```

:Property Simple FullYear
:Access Public
  ▽ r←Get
    r←(Obs,0)[X::12]
  ▽
:EndProperty

```

Simple is the default type of property, so the keyword *Simple* is not actually required above. Although *FullYear* is implemented using a function, it looks, tastes and feels² like a field (or a variable) to the *user* of the class:

```

ts1←NEW Monthly ((1 4 2 7 3) (1 2 3 7 8))
ts2←NEW Monthly ((1 3 2 4 1) (1 3 5 7 9))
ts1.FullYear[2] A February
4

```

² However, if the access function crashes, it will not smell like one.

```

      f(ts1 ts2).FullYear
1 4 2 0 0 0 7 3 0 0 0 0
1 0 3 0 2 0 4 0 1 0 0 0

      ts1.FullYear+12
SYNTAX ERROR...

```

The last statement above illustrates that `FullYear` is a read only property, which is due to our not having written the corresponding `Set` function – we'll get to that in a minute. APL is (of course!) able to perform indexing on the data returned by a property, as can be seen in the expression where we extracted data for February. However, the `Get` function *did* generate all 12 elements of data, which might have been extremely inefficient if we had more data.

The `:Access Public` statement makes the property visible from outside the instance. Without this, the property could only be used by methods belonging to the class.

:Property Numbered

Numbered properties are designed to allow APL to perform selections and structural operations on the property, delaying the call to your `get` or `set` function until APL has worked out which elements actually need to be retrieved or changed:

```

:Property Numbered ObsByNum
:Access Public
  ▽ r←Shape
    r←12
  ▽
  ▽ r←Get args
    r←(Obs,0)[X←args.Indexers]
  ▽
  ▽ Set args;i
    :If (pX)<i+X←args.Indexers
      X←X,args.Indexers ⋄ Obs←Obs,args.NewValue ⌞ Add it
    :Else
      Obs[i]←args.NewValue ⌞ Update
    :EndIf
  ▽
:EndProperty

```

In order for APL to perform indexing and other structural operations, it needs to know the `Shape` of the property. With this knowledge, APL is able to perform the same indexing and structural operations which would be allowed on the left side of the assignment arrow in a selective specification expression. The getter is called

when APL encounters a function which actually needs the data in order to proceed.

To investigate how this works, edit the `Get` function for the property `ObsByNum` in the class `Monthly` from the workspace `OO4APL\Monthly`, so that it outputs the contents of `args`. Indexers to the session on each call – and experiment with different operations on the property – for example:

```
ts1←NEW Monthly ((1 4 2 7 3) (1 2 3 7 12))
ts1.ObsByNum[2]
Get: 2
4

~2↑ts1.ObsByNum
Get: 11
Get: 12
0 3

1↑0↑ts1.ObsByNum
Get: 12
3

ts1.ObsByNum
Get: 1
Get: 2
...etc... 3...11
Get: 12
1 4 2 0 0 0 7 0 0 0 0 3
```

As can be seen above, the number of function calls is equal to the number of elements which are accessed. In a future release of Dyalog APL, it may become possible to declare the *rank* of the getter and setter, so they can work on several indices in a single call.

When APL calls a getter or setter for a numbered property, the argument is an instance of type `PropertyArguments`, which will contain the following variables:

Indexers	An integer vector of the same length as the rank of the property (as implied by the result of the <code>Shape</code> function).
Name	The name of the property, which is useful when one function is handling several properties (we'll show how to do that later).
NewValue	Within a setter, this is an array containing the new value for the selected element of the property.

Note that `Indexers` will always contain indices in the index origin of the *class*. This means that the access functions do *not* need to adapt to the index origin of the calling environment – APL adjusts the indices as appropriate. Conversely, the user of a class does not need to know the index origin used within the class. This is a good example of *encapsulation* or *information hiding*, important principles of object orientation which make it easier to share and use code without needing to understand details of the implementation.

:Property Keyed

It is common for object oriented languages to allow indexing using names or other *keys*, which are not necessarily numeric indices into an array. For example, the collection of sheets in an Excel workbook can be indexed by sheet name:

```
XL←NEW 'OleClient' 'Excel.Application'
XL.ActiveWorkbook.Sheets.Item[c'Sheet2'].Index2
```

Note: To preserve compatibility, `OleClient` objects created using `WC` will still expose `Item` as a method rather than a property:

```
'XLW' WC 'OleClient' 'Excel.Application'
XLW.ActiveWorkbook.Sheets.(Item 'Sheet1').Index
2
```

In version 11.0, *keyed* properties make it possible to support indexing of properties using arbitrary keys. If we create vector of month names to use as keys:

```
Months←'Jan' 'Feb' 'Mar' 'Apr' 'May' ... 'Nov' 'Dec'
```

Alternatively, if we have the Microsoft.Net framework available on our machine, we can extract the names from the environment:

```
[Using-'System.Globalization' A DotNet "Globalization Namespace"
Months←12↑(DateTimeFormatInfo.New 0).AbbreviatedMonthNames
```

This will define a 12-element vector³ of 3-letter month names in the local language. We can now define a property to do indexing using local month names:

³ `DateTimeFormatInfo.AbbreviatedMonthNames` has 13 elements, the last element (which is empty) is provided for use in 13-month calendars.

```

:Property Keyed ObsByName
:Access Public
  ▽ r←get args;i
    □SIGNAL((pMonths)v.<i←Months;1>args.Indexers)/3 A INDEX ERROR
    r←(Obs,0)[X;i]
  ▽
  ▽ Set args;i;m
    □SIGNAL((pMonths)v.<i←Months;1>args.Indexers)/3 A INDEX ERROR
    m←(pX)<i A New months?
    X←X,m/i ♦ Obs←Obs,m/args.NewValue A Add new months
    Obs[X;i]←args.NewValue
  ▽
:EndProperty

```

The ObsByName property can be used as follows:

```

1 ts1.ObsByName[c'Jan']
   ts1.ObsByName['Oct' 'Nov' 'Dec']÷4 5 6

```

The entire index and array of new values (when provided) is passed in a single call to a *keyed* getter or setter. APL will verify that the shape of the sub-array identified by the Indexers conforms to the shape of the result of get, and in the case of set, the new values. This is why 'Jan' must be enclosed in the first example above.

Unlike a field, a property does not map directly to an underlying array. The property provides a “virtual” array, the elements of which can be generated on demand – and created when updated. As an extreme example of this, the following class presents the APL interpreter as an infinitely large keyed property, with all possible APL expressions as keys, and the results of the expressions as the corresponding values:

```

:Class APL
:Property Keyed ValueOf
:Access Public
  ▽ r←Get args
    r←⍺''1>args.Indexers
  ▽
:EndProperty
:EndClass A APL

```

Allowing:

```

iAPL←□NEW APL
iAPL.ValueOf['2+2' '÷/t5']
4 15

```

Numeric Keyed Properties

Note that you can use *keyed* properties with integer indices, and use them in the same way as in a *numbered* property. You might do this if you were worried that a numbered property would be inefficient due to the number of calls to your access functions, or if a simple property would cause too much data to be generated. The drawback of this approach is that the only direct selection operation which is possible on a keyed property is square bracket indexing, and – since APL passes the indices unchanged – the indices will have to be provided in the index origin of the class.

For readers familiar with the concept of function rank, it is possible to think of a numbered access function as having rank zero, while a keyed access function has infinite rank. Future versions of Dyalog APL may allow you to control the rank of your access functions.

Default Properties

The `:Indexer` keyword defines a *default property* for a class. If a class has a default property, square bracket indexing can be applied directly to a ref to the instance. If we edit the definition of the `ObsByNum` property and replace `:Property` and `:EndProperty` by the keywords `:Indexer` and `:EndIndexer`, we can apply indexing as follows:

```

    ts1←NEW Monthly ((1 4 2 7 3) (1 2 3 7 8))

    ts1[2]           A Short hand for the following expression
4
    ts1.ObsByNum[2] A Equivalent to the above
4

    pts1           A But: ts1 is a scalar

    2↑ts1
DOMAIN ERROR

```

The last two expressions expose a problem with indexers: An instance (or rather: a reference to an instance) *is* a scalar. The APL standard allows square bracket indexing on instance as a *conforming* extension, since indexing the instance would give a `RANK ERROR`. However, all other selection or structural operations will view the instance as a scalar. For example, the expression `(2↑ts1)` should return `ts1` followed by a prototypical instance of the `Monthly` class, but fails because none has been defined (we'll look at how to define class prototypes when we take a closer look at *constructors*).

Version 11.0 adds support for APL2's "squad indexing" primitive, a dyadic primitive function which provides a functional alternative to square bracket indexing:

```

      4⍵'ABCD'          R 'ABCD'[4]
D
      2 (1 2)⍵2 2ρ'ABCD'  R (2 2ρ'ABCD')[2:1 2]
CD

```

In version 11.0, *monadic* `⍵` returns the entire right argument, as if the elided left argument had selected everything along all dimensions. If the right argument is an instance, monadic `⍵` returns the entire contents of the default property.

```

      ρ⍵ts1
12
      2+⍵ts1
1 4
      ⍵ts1
1 4 2 0 0 0 7 3 0 0 0 0

```

Note that the 2nd expression above only called the numbered getter twice, once for each element of the complete selection operation `2+⍵`. If we had picked `ObsByName` as the `:Indexer`, we would be able to index the instance using month names:

```

      ts1←NEW TS2 ((1 4 2 7 3) (1 2 3 7 8))
      ts1['Jan' 'Jul']
1 7

```

(Obviously), there can only be one default property.

Triggers

If you want the "lightweight" characteristics of a *field*, where APL handles data access directly without your having to write access functions, but you would like to be able to react to changes to the field, you can use a *trigger*. A trigger is a function which is called when one or more fields (or other variables) are modified. For example, we could update our `Monthly` class so that it calculates the coefficients when one of the variables involved in the calculations change, rather than doing it on every call to `PolyFit`. `PolyFit` can be modified to use the pre-calculated coefficients:

```

▽ PreCalc args
  :Implements Trigger X,Obs,Degree
  □←'PreCalc: ',args.Name
  :If OK←((pObs)=pX)^Degree<pObs
    Coeffs←Obs□X°. *0,1Degree A Find polynomial coeffs
  :EndIf
▽

▽ r←PolyFit x;coeffs
  :Access Public
  A Fit polynomial to observed points to compute values at x
  'Unable to Fit Polynomial' □SIGNAL OK+5
  r←Coeffs+. *qx°. *0,1Degree A Compute fitted f(x)
▽

```

PreCalc writes to the session when it is called, so we can keep an eye on things:

```

      ts1←□NEW Monthly ((1 4 2 7 3) (1 2 3 7 8))
PreCalc: Obs
PreCalc: X
      1←ts1.PolyFit 1 2 3 4
      1.7 2.0 3.5 5.3

      ts1.Degree←2
PreCalc: Degree
      1←ts1.PolyFit 1 2 3 4
      1.0 2.7 4.0 4.8

      ts1.X←1 2 3 7
PreCalc: X
      1←ts1.PolyFit 1 2 3 4
      Unable to Fit Polynomial

```

The trigger mechanism in version 11.0 is not a purely object oriented feature. It can be used to track any variable in a workspace. For example:

```

▽ foo args
  :Implements Trigger A
  'A ←'A'at',1+□SI{α,['',(≠w),']}'□LC
▽
▽ goo
  A←2
  A←'Hello'
▽

```

If we now run `goo`, we will see the following output:

```

A ← 2 at goo[2]
A ← Hello at

```

Warning: Applications should not depend on the exact timing of, or the number or calls to, a trigger, beyond relying on each trigger function to be called at least once following the completion of the primitive function (usually assignment) which set the trigger variable. As can be seen in the above example, the trigger functions were called *after* the completion of the entire line of code containing the trigger event. In the second case, `goo` was no longer on the stack at the time when `foo` was called.

Chapter 6. Constructors and Destructors

As we have seen, a *constructor* is a function which is called to initialize the contents of an instance, immediately *after* the instance has been created – but *before* it can be used. In the examples we have seen so far, we have used it to:

- Set an initial seed in our Random number generator class
- Initialise data for a new instances of `Monthly` and `TimeSeries`

A constructor is used when it makes sense to provide initial values for fields, or allocate external resources associated with the object. For example, a constructor might open a component file, or connect to a database and execute a query.

A *destructor* is called when the instance is about to disappear. In a destructor, you can close files and database connections, or free up any other resources which will no longer be required. The following is a simple class which provides access to Excel Workbooks, using Excel as an OLE Server:

```
:Class ExcelWorkbook
:Field Public Application
:Field Public Workbook
:Field Public Sheets
:Field Private Opened+1 A Did we open it?

▽ make book;i;books
:Implements Constructor
'Application' OWC 'OleClient' 'Excel.Application'

:If 0≠Application.Workbooks.Count A Currently open books
  books=Application.Workbooks.(Item"tCount)
:AndIf (pbooks)≥i+books.FullName≠book A Already open?
  Workbook=i+books
  Opened+0 A No we did not
:Else
  :If book^.=' ' A No book named => Create one
    Workbook+Application.Workbooks.Add 0
  :Else
    A Open the requested book
    Workbook+Application.Workbooks.Open book
```

```

        :EndIf
    :EndIf
    ...

    Sheets=Workbook.Sheets.(Item":Count) A []Workbook.Sheets
v
v close
:Implements Destructor
:If Opened A Close it if we opened it
    :Trap 0 A If workbook somehow damaged
        []+'Closed workbook ',Workbook.FullName
        Workbook.Saved+1
        Workbook.Close 0
    :EndTrap
:EndIf
v
:EndClass A Class ExcelWorkBook

```

The constructor takes the name of a workbook as its argument, and ensures that the book is open. If the name is blank, a new workbook is created. The three public fields of this class are:

Application	A handle to Excel.Application, so we can do things like make Excel visible and tell it not to issue prompts.
Workbook	Handle to the Excel workbook instance corresponding to our workbook.
Sheets	A vector containing all the sheets in the book, for easy access.

The following illustrates the use of the first two fields.

```

wb1=[]NEW ExcelWorkBook 'c:\temp\book1.xls'
wb1.Application.Visible=1
wb1.Workbook.FullName
C:\temp\Book1.xls

```

The following examples show how we can read data from the sheets, make a change, save the workbook, and close the object:

```

wb1.Sheets.Name
Sheet1 Sheet2 Sheet3

```

```

wb1.Sheets[1].(Name UsedRange.Value2)
Sheet1  1  2  3  4  5  6  7  8
        9 10 11 12 13 14 15 16

wb1.Sheets[1].(Range 'A1:B2').Value2+2 2p'Kilroy' 'was' 'here' '!'

wb1.Workbook.SaveAs 'c:\temp\book2'

[]ex 'wb1'
Closed workbook: C:\temp\Book1.xls

```

Note that the destructor is called when the *last* reference to an instance disappears. If you expunge a name and the destructor is not called as you expected, look for a leaked local variable or a temporary global created by hand, containing a reference that you had forgotten about.

The system function []INSTANCES will return a list of refs to existing instances:

```

wb1+[]NEW ExcelWorkBook 'c:\temp\book1.xls'
wb2+[]NEW ExcelWorkBook 'c:\temp\book2.xls'
books+wb1 wb2

[]instances ExcelWorkBook
#.[Instance of ExcelWorkBook] #.[Instance of ExcelWorkBook]
  ([]instances ExcelWorkBook).Workbook.Name
Book1.xls Book2.xls

)erase wb1 wb2
p[]instances ExcelWorkBook
2

```

There are still 2 instances.

Display Form

The function []DF (Display Form) can be used to set the “display form”, of an instance (or namespace) – which defines the result returned by monadic format (#) of the instance. By default, the display form shows the parent space and the name of the class, as in:

```
#.[Instance of Monthly]
```

This tells us that we have an instance of `Monthly` which was created in the root (#). []DF is often used in a constructor to set the display form to something which helps to identify the particular instance. For example, if we added the following line to the constructor of `Monthly`:

```
[]DF #,[1.5]['Timeseries'] (Months[X], '=', [1.5]Obs)
```

Then the display form would be something like:

```

    □-ts1-□NEW Monthly ((1 4 2 7 3) (1 2 3 7 8))
[Timeseries]
Jan = 1
Feb = 4
Mar = 2
Jul = 7
Aug = 3

```

We could also update the trigger function to keep the display form up-to-date.

Niladic Constructors and the Class Prototype

All the constructor methods we have written so far have taken an argument which has been used to initialise each new instance. If the constructor is a niladic function, this means that the class does not need any instantiation parameters. It is possible for a Dyalog APL class to have two constructors: One which takes an argument, and one which does not. A constructor with no argument is known as the *default constructor*. If □NEW is called with no parameters following the class, the default constructor will be called. For example, we could have defined a niladic constructor for the Random class, as an alternative to testing whether the argument is zero in the monadic constructor:

```

▽ make args
:Implements Constructor
InitialSeed←args
Reset
▽

▽ make0
:Implements Constructor
make InitialSeed
▽

```

Often, the default constructor will simply call the main constructor, supplying default arguments. With the above constructors and the RL function from the beginning of chapter 4, the Random class could be used as follows:

```

r1-□NEW Random A Nothing strange about this...
Random Generator Initialised, Seed is 42

r1.RL 1234 A Change □RL to 1234 (returns old value)
42
(3+r1).RL 0 A Creates two instances using default constructor
Random Generator Initialised, Seed is 42
Random Generator Initialised, Seed is 42
1234 42 42

```

```

r0=0/r1      A Keeping none causes creation of a prototype
Random Generator Initialised, Seed is 42

```

```

(1↑r0).RL 0  A 1↑ causes creation of a new prototype
Random Generator Initialised, Seed is 42
42

```

```

1↑r0.RL 0
0

```

In the last example, APL determines the type of the result by calling the function `RL` on the prototype and doing a `0p` of that result (`42`). `1↑` on this gives the result. **Note:** The exact timing of prototype creation during the execution of expressions involving empty arrays may change before version 11.0 is generally available.

Chapter 7. Shared Members

Almost all the members we have used up to this point have been *instance* members. An *instance field* has a distinct value for each instance. The code in *instance methods*, or associated with *instance properties*, generally refer to instance data. *Shared* fields have the same value in *all* instances. Shared methods and property accessor functions access shared data.

The meaning of the terms *public* and *private* is the same for shared members as it is for instance members. Public members are visible from outside, private members can only be seen by code which is defined and running inside the class.

The `Months` field, which contains abbreviated month names within the `Monthly` class, is an example of a field which might as well be shared – it is going to be the same in all instances. We can add:

```

:Field Shared Public Months

```

... to the beginning of our `Monthly` class, and subsequently verify that the property is in fact shared:

```

ts1←NEW Monthly ((1 2 3 4)(5 6 7 8))
ts2←NEW Monthly ((3 4 5 6)(5 6 7 8))
ts1.Months[3]←'Mch'

ts2.Months
Jan Feb Mch Apr May Jun Jul Aug Sep Oct Nov Dec

)erase ts1 ts2
Monthly.Months
Jan Feb Mch Apr May Jun Jul Aug Sep Oct Nov Dec

```

As the last expression illustrates, public shared properties can be referenced through the class itself, without using an instance. In this case, it is probably a good idea to declare the field as “read only”:

```
:Field Public Shared ReadOnly Months=12f
      (DateTimeFormatInfo.New #).AbbreviatedMonthNames
```

Most of the time, shared fields are likely to be either read only or private – or hidden behind some kind of property with careful validation in the setter.

Shared members are available to code which defines instance methods and properties. As we have seen above, they can be used as if they were members of all instances. On the other hand, instance data is *not* visible to shared code, since this would presume the selection of a particular instance. Note that code which is executed when the a class script is fixed, is shared code, executing within the class. Thus, it will not be able to see the value of instance fields, except to define default values for instance fields in :Field statements.

Shared Methods

Shared methods will either be access methods which are used to manipulated shared data, or methods which provide a service which is related to the class but does not require or apply to an instance. For example, if we extend our `ExcelWorkbook` class with a method which lists the workbooks in a folder, we could provide this as a shared method:

```
▼ r←List folder;[]USING
  :Access Public Shared

  []USING~'' A DotNet Search Path
  :Trap 0
    r←((System.IO.DirectoryInfo.New folder).GetFiles'*.xls').Name
  :Else ◊ r←#
  :EndTrap
▼
```

This would allow us to find out which workbooks there are, which is information we might need *before* we open one:

```
ExcelWorkbook.List 'c:\temp'
Book1.xls Book2.xls Book3.xls
```

Chapter 8. Inheritance

It is possible to define a class which *extends* -- or *inherits* the features of -- an existing class. *Inheritance* is considered to be one of the most important features of Object Orientation, because inheritance provides a well-defined mechanism for sharing code.

The screenshot shows a Microsoft Excel window titled "Microsoft Excel - Widgets.xls". The spreadsheet contains data for "Widget Sales" from January to December, including Actual, Budget, Variance, and Forecast values. The status bar at the bottom indicates "Ready".

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Widget Sales													
2		Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Year
3	Actual	10	10	11	14	0	0	0	0	0	0	0	0	45
4	Budget	10	11	12	13	14	15	16	17	18	19	20	21	186
5	Variance	0	-1	-1	1	0	0	0	0	0	0	0	0	-1
6	Forecast	10	10	11	14	14.5	15.8	17.1	18.4	19.7	21	22.3	23.6	197.4

Imagine that we would like to create a simple budgeting and forecasting application which uses Excel spreadsheets like the above for data entry.

Departmental managers will initially enter budget and subsequently actual data and send their workbooks to us at regular intervals for reporting and consolidation. Our plan is to read data from workbooks in the above format, and use code written in APL to calculate the variance to budget for those months where both actual and budget data is available and finally produce a Forecast. Something along the lines of:

```

▽ Calculate;real;ts
  real←+/v\φActual≠0          A # months of real data
  Variance←12↑real↑Actual-Budget A Compute VAR where we have data

  :If real>0
    ts←NEW Monthly(real↑Actual)
    ts.Degree←1 A 3rd degree forecasts are a bit too exciting
    Forecast←(real↑Actual),real↑ts.PolyFit12
  :EndIf
▽

```

We will call this special type of workbook a "PlanBook" (there is an example file called `widgets.xls` in the `OO4APL` folder). The `PlanBook` class will extend `ExcelWorkbook`, providing additional properties called `Actual`, `Budget`, `Variance` and `Forecast`, which access data via the `Sheets` field which `ExcelWorkbook` makes available.

The constructor in our new `PlanBook` class will examine the contents of the workbook which has been opened and look for signs that it is a well-formed `PlanBook`. We might start with a class definition like the following:

```
:Class PlanBook : ExcelWorkbook

:Field Private RowNames='Actual' 'Budget' 'Variance' 'Forecast'
:Field Private Instance DataRange A Will point to Excel Data Range

▽ make book;z
:Access Public Instance
:Implements Constructor :Base book

:If Sheets.Name≠,'Plan'
:AndIf 6 14<=pZ-Sheets[1].UsedRange.Value2
:AndIf ((pRowNames)↑2+z[;1])=RowNames
    DataRange=Sheets[1].Range 'B3:M6'
:Else
    (book,' is not a valid Plan Workbook')[]$SIGNAL 11
:EndIf

▽

:EndClass A Class Plan
```

A derived class is declared by following the class name with a colon and the name of the class which we wish to extend. The `:Base` keyword in the `:Implements Constructor` causes a call to the base constructor, using the result of the expression following `:Base` as the argument. In this case, the argument to the `PlanBook` constructor, which will contain the name of the workbook, is passed unmodified to the `ExcelWorkbook` constructor.

Once the workbook is open, `make` takes a look at the `Sheets` member, which we have *inherited* from `ExcelWorkbook`, to verify that the workbook has a single sheet named "Plan", that this has at least 6 rows and 14 columns, and that the second column contains the four names which we expect. If all is well, we create a private field called `DataRange`, which gives us easy access to the 4×12 range of cells containing the data.

If the workbook does not look right, we signal an error, which will be reported by `NEW` and prevent the instance from being completely created. In this case, the

base destructor is called as the instance disappears and the workbook is closed again.

```
w←NEW PlanBook'c:\temp\book1.xls'
c:\temp\book1.xls is not a valid Plan Workbook
w←NEW PlanBook'c:\temp\book1.xls'
^
Closed workbook C:\temp\Book1.xls

w←NEW PlanBook'c:\temp\widgets.xls'
w.Sheets.Name
Plan
```

All that remains to make it possible for us to write `Calculate` in the way we envisaged, is to provide the four “timeseries” as simple properties:

```
:Property Actual,Budget,Variance,Forecast
:Access Public Instance
  ▽ r←get args
    r←DataRange.Value2[RowNames←args.Name;]
  ▽
  ▽ set args
    DataRange.Value2[RowNames←args.Name;]←args.NewValue
  ▽
:EndProperty
```

We can now open the `PlanBook` and work with it:

```
w←NEW PlanBook'c:\temp\widgets.xls'
w.(←Actual Budget)
10 10 11 14 0 0 0 0 0 0 0 0
10 11 12 13 14 15 16 17 18 19 20 21

w.Calculate
tw.(Variance Forecast)
0 -1 -1 1 0 0 0 0 0 0 0 0
10 10 11 14 14.5 15.8 17.1 18.4 19.7 21 22.3 23.6
```

And the spreadsheet has been updated. We can now:

```
w.Workbook.Save
)erase w
Closed workbook C:\docs\sales\widgets.xls
```

And our work is done.

Inherited Members

We have seen that the `Sheets` member, inherited from `ExcelWorkBook`, is available as a public field of instances of `PlanBook`. Public methods are (of course) also inherited by the derived class:

```
w.List 'c:\docs\sales'
Book1.xls Book2.xls Widgets.xls
```

However -- it might be more useful to have a more specific version of `List`, which only lists valid `PlanBooks`. We can use the base method to get the list of workbooks, and then validate them by opening each one as a `PlanBook`:

```
▽ r←List folder;m;i;z
  :Access Shared Public
  A Extend ExcelWorkBook.List to list only PlanBooks

  r←BASE.List folder
  m←(pr)p1
  :For i :In tpr
    :Trap 0 0 z←NEW PlanBook(folder,'\ ',i>r)
    :Else 0 (i>m)+0 0 :EndTrap
  :EndFor
  r←m/r
▽
```

Which allows:

```
ExcelWorkBook.List 'c:\temp'
Book1.xls Book2.xls Widgets.xls

PlanBook.List 'c:\temp'
Closed workbook C:\temp\Book1.xls
Closed workbook C:\temp\Book2.xls
Closed workbook C:\temp\Widgets.xls
Widgets.xls
```

It is possible to access the `ExcelWorkBook` version of a method via an instance of `PlanBook`, by using a technique called *casting*. Dyadic `CLASS` allows us to access members of the base class by providing a view of the instance as if it were an instance of that class. You can only *cast* to a class which is in the class hierarchy for the instance:

```
(ExcelWorkBook CLASS w).List 'c:\temp'
Book1.xls Book2.xls Widgets.xls
```

Public members of the base class become public members of the derived class, and are (of course) also accessible to the code in the derived class. We have to refer to

the base class version of `List` as `□BASE.List` because the derived class has defined its own version of `List`. Note that `□BASE` is “special syntax” which searches the class hierarchy for a particular member, and can only be used if it is followed by a dot and a base member name. `□BASE` is *not* a function which can be used to return a reference to the base class.

Private members of the base class remain hidden from the outside. As any other user of `ExcelWorkbook`, the implementor of `PlanBook` is insulated from private changes to the implementation of the base class – which helps make it possible for both implementors to get a good night’s sleep. Which is even more important if you are the implementor of both.

Version 11.0 does not implement *protected* members, which are a kind of halfway house – visible to code in derived classes, but invisible from the outside.

Benefits of Inheritance

If we needed to process a new type of workbook, we could create a second (third and fourth) class deriving from `ExcelWorkbook`. The benefits of working this way are:

- Easy reuse of the work done in writing the base class, without duplicating code.
- Classes are easier to learn to use, due to shared features. For example, the `List` method is available in all classes which derive from `ExcelWorkbook`.
- The rules of class inheritance guarantee that, so long as the behaviour of public members is not changed, future enhancements to the base class will be immediately available to the derived classes, unless they *decide* to implement different behaviour.

The rules of class inheritance minimize the burden of maintenance, training and documentation – so long as we adhere to some simple rules. Of course, there *are* a few rocks on the road. For one thing, the very attraction of the above benefits can quickly lead to unnecessarily general base classes with a huge collection of members, which can end up being both inefficient and difficult to learn. Huge quantities of documentation are not necessarily a good thing if only 5% of it is relevant to the job at hand.

As requirements change, deciding where in the hierarchy to add new functionality may require much thought. The bad news is that the design of complex systems is always going to require insight into the problems which need to be solved, today and in the future – regardless of the technology or paradigm

used. The good news is that OO provides excellent tools both for the redesign and reimplementing of, and easy migration to, a new set of classes. Given that the public interface to a class is so well defined, it is possible to completely re-factor the implementation at any level in the hierarchy with a minimum of effort, and continue to provide the same interface to those application components which cannot easily be rewritten.

Inheriting from Several Classes

In our `PlanBook` example, although we focused on extending the `ExcelWorkbook` class, we also used functionality from the `TimeSeries` class. In this case, the choice to make `PlanBook` an extension of `ExcelWorkbook` was fairly easy – but in many other situations, it can be difficult to decide which class to extend. Dyalog APL has followed C# in only allowing you to derive from *one* other class. Some OO platforms allow *multiple* inheritance, with rules for how name conflicts are resolved and constructors and destructors cascade. Unfortunately, while all the OO features we have discussed so far in this guide, including *single* inheritance, are supported in much the same manner in all OO systems, there is less agreement on the details of multiple inheritance. This is one of the main reasons why C# has avoided it, and we decided to follow suit.

Given the way classes work, it may be easier to work around not having multiple inheritance than it would be to understand any particular flavour of it. We had no great difficulty in using the `TimeSeries` class from within `PlanBook`. Even if inheritance was not available at all, we would have been able to provide the important functionality of `PlanBook` quite easily:

The statement:

```
:Implements Constructor :Base book
```

Would have to be replaced with:

```
:Implements Constructor
Workbook←[]NEW ExcelWorkbook book
Sheets←Workbook.Sheets
```

In the `List` method, we would have to replace:

```
r←[]BASE.List folder
```

By:

```
r←ExcelWorkbook.List folder
```

The big difference between the result of doing this and the original `PlanBook` is that the other public members of `ExcelWorkbook` would not be exposed. Extensions to `ExcelWorkbook` would not be immediately available to users of `PlanBook`. Whether this is a drawback or a simplification depends on your requirements. In the long term, you might be better off with a class which only exposed the specific `PlanBook` functionality. If you subsequently decide to expose it, all you need to do is to expose the new `Workbook` variable as a public field, which would allow:

```
w←NEW PlanBook2 'c:\temp\widgets.xls'

w.WorkBook.List 'c:\temp'
Book1.xls book2.xls widgets.xls

w.WorkBook.Application.Version R Excel Version Number
11.0
```

If we add the `List` method from `PlanBook` to `PlanBook2`, then `w.List` would call the `PlanBook` version of `List`, and `w.WorkBook.List` would allow access to the `ExcelWorkbook` version, so both would be available.

Code Reuse with :Include

If you have a set of utility functions or methods which you wish to include in a number of classes, but these classes may be deriving from something else, `:Include` provides another alternative to multiple inheritance. For example, if we decide that the `List` and `Delete` methods of our `ExcelWorkbook` are generally useful, we can create a namespace called `ExcelTools` and define the functions in it:

```
)cs ExcelTools
Using''

▽ Delete file
[1] :Access Public Shared
[2] (System.IO.FileInfo.New file).Delete
▽

▽ r←List folder
[1] :Access Public Shared
[2]
[3] :Trap 0
[4] r←((System.IO.DirectoryInfo.New folder).GetFiles'*.*.xls').Name
[5] :Else o r←0
[6] :EndTrap
▽
```

If we remove `List` from `PlanBook2` and add the line:

```
:Include #.ExcelTools
```

This will import the functions in `ExcelTools`, without using inheritance. The difference between this and using inheritance is that the `:Included` functions actually become methods of the class they are imported to, so they can reference members of this class if required – which would not be the case if they were inherited methods of a base class.

Whether you use inheritance or `:Include`, the source code is *not* copied, so if you trace into and modify a method, the change will be made in the original namespace or class, and any instance or class which currently contains it will immediately see the new version.

Summary of Chapters 5–8

New features of Dyalog APL version 11.0 which we have introduced while discussing Properties, Constructors, Destructors, Shared members – and Inheritance.

<code>:Implements Trigger field</code>	The function will be called whenever field is modified (upon completion of the line).
<code>:Implements Constructor</code>	The function is used to initialize a new instance of the class.
<code>:Implements Destructor</code>	Function used to clean up or de-allocate resources an instant before the instance disappears.
<code>⎕INSTANCES</code>	Return instances of [or derived from] class(s)
<code>⎕DF</code>	Set Display Form
<code>index⎕array</code>	Squad indexing
<code>⎕instance</code>	Return indexable part (indexer of an instance, else return entire right argument)
<code>:Property [Simple]</code> <code>:EndProperty</code>	A property where getters and setters pass the entire array at once.

:Property Numbered :EndProperty	Getter and setter work on a single item of the property at a time, identified by numeric indices along each dimension. Requires Shape function.
:Property Keyed :EndProperty	Entire Indexer passed in a single call to getter and setter. Square bracket indexing only permitted.
:Indexer :EndIndexer	A property which is referenced if square bracket indexing is performed directly on the instance.
:Field ... Shared ... :Property ... Shared ... :Access ... Shared ...	Fields, Properties and Methods which are shared by all instances, and accessible through the class.
:Field ... ReadOnly	A read only field.
:Base ...	Following :Implements Constructor, specifies a call to the base constructor.
□BASE.member	A reference to a public member in the closest class in the hierarchy which exposes it
:Class name : baseclass	Class name which <i>derives from</i> or <i>extends</i> baseclass.

J-ottings 47: RU14 txt msgs? pls 1!:1 on

by Norman Thomson

APL used to be described by its deriders as a system of hieroglyphics. Then around 1990 or so J was unveiled which used only ordinary keyboard symbols to achieve an even greater degree of shorthand without hieroglyphics, making it surely the earliest form of text messaging. Now that text messaging is commonplace, could J become more widespread if presented as a scientific text messaging *lingua franca*?

Of course for the newcomer, the J vocabulary in its entirety is far too great to be accepted in a single bite, or even many great swallows. But it could make sense to think in terms of a basic acceptable subset, leading to a gradual infusion of more advanced shorthands.

The arithmetic and logical verbs

+ - * % ^ ! < > = >: <: ~:

pose no great problems since these symbols, or symbols like them, are already widely used in, say, e-mails, where no mathematical symbols are possible. The only unfamiliar form is monadic * meaning signum.

The symbols derived from the above for double, halve, square and square root, together with increment and decrement, should be easily accommodated, viz.

+: -: *: %: <: >:

as should the di-grams +/ and */ for sum and product of lists (Σ and Π in maths terms).

Series, that is lists, are an essential part of the discourse of mathematics, and operations to join (,), tally (#) and reverse (I.) are a necessary element of any programming language which admits strings. Head/behead and tail/curtail are desirable additions to the basic set, albeit a degree of memorisation/indoctrination is required before it becomes ingrained that { means 'take one', } means 'take all but one', dot means 'first' and colon means 'last'.

Intelligent texting requires some appreciation of the special parsing rules of J, in particular the concepts of *hook* and *fork*, which lead to understanding why $(+/\%#)v$ is 'mean of v ' and $(*>:)n$ is $n(n+1)$. Also @ denoting 'after' is needed so that $(-:@(*>:))n$ means 'halve after n times $(n+1)$ '. The rule in text messaging terms is that @ stops on meeting the first verb to its right, so that $(-:@*>:)n$ means 'semi-multiply n by $n+1$ ', giving the same result as before but by a subtly different route. As another example $*@+/\%#$ means $(*@+/\%)\%#$, that is a single fork articulated as '(signum of sigma) divided by tally', as opposed to $*@(+/\%#)$, that is 'signum after mean', which is not the same thing:

$(*@+/\%#)1\ 2\ 3$	NB. (signum of $1+2+3$) divided by 3
0.333333	
$(*@(+/\%#))1\ 2\ 3$	NB. signum of mean of 1 2 3
1	

Of course if a J interpreter is available grammar verification is always to hand!

These ideas can be tested on the formula quoted in Gene MacDonnell's article "Metlov's Triumph" in Vector Vol. 21 no. 4. The core problem involved counting numbers of triangles within triangles, which leads to the formula

$$* \ -: \ @ \ * \ +$$

The rule that the right scope of @ is a single verb means that the unambiguous interpretation, parenthesised for clarity, is the fork

$$* \ (-: \ @ \ *) \ +$$

meaning 'semi-multiply the sum by the product'. The same six symbols could be written in a different order

$$-: \ @ \ * \ * \ +$$

meaning $(-: \ @ \ *) \ * \ +$ that is 'semi-product multiplied by sum', a different computation but leading to the same result.

A symbol for 'repeat' is a valuable text messaging addition. Suppose that 'joddling', that is 'joining double the last', turns up as an intermediate building block in a bigger application.

This idea is expressed comprehensibly and unambiguously as

```

      jdl=.,+:@{:          NB. the verb to joddle
      jdl 1 3 6
1 3 6 12

```

The idea of the power conjunction as a shorthand for 'repeat' falls naturally into place:

```

      jdl^:3(1 3 6) N      NB. joddle three times
1 3 6 12 24 48

```

One of the most frequent ways in which lists turn up in mathematics is as coefficients of polynomials. #. is a convenient shorthand for 'polynomial evaluation', so

```

      4 #.1 2 3           NB. coeffs. in descending power order
27

```

means evaluate the polynomial $x^2 + 2x + 3$ when $x=4$.

To convey the idea that a polynomial is a single entity which is represented by a coefficient list, 'box' it (<), and use the reverse shorthand 'open' (>) to bring the boxed polynomial back to coefficient list form. At the same time define &> as a shorthand (or arguably a longhand!) for each – the intricacies of how it works do not require explanation at the text messaging level. Thus

```

      _1 0 1 2 #.each <1 2 3      NB. values of  $x^2+2x+3$ 
2 3 6 11

```

The sum of the first four values of the series $u_n = 1 + 2n + 3n^2$ is then

```

      +/1 2 3 4 #.each <1 2 3      NB. +/ is sigma
62

```

A more substantial example of using text messaging techniques is to explain the terms "internal rate of return" (IRR) and "annual percentage return" (APR). The phenomenon of 5% compound interest growth can be described equivalently by any one of four numbers namely 5, 0.05, 1.05 and $(1.05)^{-1}$. These differences are expressible in J terms as:

```

p=.percentage rate          (p>_100)
r=.fractional rate so r=.0.01*p      (r>_1)
k=.multiplication factor so k=.:r    (k>0)
d=.discount factor so d=.%k or %>:0.01*p  (d>0)

```

To be meaningful, values must be in the ranges indicated. Negative growth, that is decay, is indicated by $p < 0$, $r < 0$, $k < 1$ and $d > 1$.

Conversion from p to d is described above and is consolidated as

```
ptod=.%>:@(0.01&*)
ptod 20
0.83333
```

The reverse operation d to p is given by $dtop=.100&*@<:@$. Both $ptod$ and $dtop$ require the use of the conjunction $\&$ to specify the particular multiplication involved. The underlying concept of IRR is based on the "worth" of a project, defined as the final return reckoned in present money. This is expressed by

$$\text{worth} = . + / i s * d \wedge i . \# i s$$

where $i s$ is an income stream with values after the first assumed positive, and d stands for the discounted value of money. The IRR is obtained by taking $\text{worth} = \text{zero}$, that is by solving the polynomial $0 = d \wedge i s$. Thus for a project with an income stream $_100 \ 0 \ 0 \ 200$, that is an initial investment (outflow) of 100 in year 0, with a return of 200 in year 3 and no intervening inflows, the initial investment has increased by a multiple of 2. The cube root of 2 ($3\% : 2$ in shorthand) is 1.26, and so the IRR is 26%.

The significance of "internal" is that equating to zero creates a system in which ultimate worth is neither increased nor decreased for the solved value of d , which is therefore that rate of continuous money decline which would need to occur to make the overall project do no more than break even. If the *actual* rate of money decline exceeds the IRR, then it would have been better not to have undertaken the project. IRRs should in general be used in a *relative* sense to compare alternatives, rather than as absolute measures – the higher the IRR of a project alternative, the more flexibility there is for its success compared with other proposals. For valid comparisons, project proposals should have approximately the same initial outlays and the same income stream lengths. If these conditions are not met further considerations concerning investment of surplus and realised funds should be taken into account.

Obtaining values for IRRs is a matter of solving polynomial equations for which income streams provide the coefficients. Now comes the *coup de grace* for those of your text messaging friends or colleagues who have not already been swept off their feet. The d in $0 = d \wedge i s$ is merely a dummy denoting the form of the solution, the solution itself is supplied by just two keystrokes. Not only does this convey the message, but, given a J interpreter, the solution as well:

```

is=._100 20 50 70 80      NB. an income stream
p.is
+-----+
|80|_1.2196 _0.20558j1.1462 _0.20558j_1.1462 0.75579|
+-----+

```

from which the d value of 0.75579 is extracted and converted to p form by

```

dtop 0.75579
32.312

```

to obtain an IRR of 32.3%. The coefficients in the argument for p. are in *ascending* power order, while those for the polynomial evaluator #. are in *descending* order. Thus the value of 0.75579 is confirmed by

```

0.75579 #.|.is
0.00046177      NB. effectively zero

```

It is always nice to have a simple case confirmed, so using the first example above

```

p. _100 0 0 200
+-----+
|200|0.7937 _0.3969j0.6874 _0.3969j_0.6874|
+-----+

```

from which 0.7937 leads to confirmation of the p form obtained earlier.

```

dtop 0.7937
25.99

```

Extracting the d value above consists of opening the second box resulting from p.

```

roots=.>@{:@p.      NB. extract list of roots

```

and obtaining the single non-negative, non-complex value, the latter property being tested for by comparison with conjugates as provided by + applied to complex numbers. From tally it is one small step to #~ as shorthand for 'select', which leads to two further building blocks

```

real=#~ (= +)      NB. select real values
pos=#~ >0          NB. select positive values

```

The process of obtaining the internal rate of return then consists of four steps in sequence:

```

irr=.dtop@pos@real@roots NB. internal rate of rtn for an is
irr is
32.312

```

In words IRR is calculated as: convert to p form the positive real root of the income stream regarded as polynomial coefficients.

IRR calculations mirror those for fair loan repayment rates. The formula $(rk^n)/(k^n-1)$ gives the fraction of an amount loaned which must be repaid in each of n periods when interest $p\%$ is paid on the declining balance. The words "declining balance" reflect the fact that IRR calculations make an implicit assumption that the inflows are reinvested at the calculated compound interest rate, which can sometimes lead to a rosy-eyed picture of absolute IRR values. Using the input stream as an illustration, the precise progress of each flow as it is reinvested is made evident by

```
k=.1.323121          NB. extra precision used
(20*k^3)+(50*k^2)+(70*k^1)+80-(100*k^4)
8.75549e_5
```

Sometimes it can be convenient to enter an arithmetic formula such as $(rk^n)/(k^n-1)$ by entering it more or less directly in the style of traditional programming languages. This option is readily available to text messengers through the words 'monad' and 'dyad' thus:

```
PA=.dyad : '(x.*(1+x.)^y.)%_1+(1+x.)^y.'
          NB.P(ayment)% A(mount)
```

The PA fraction, and the consequent repayment amount for a loan of 10000 are given by

```
0.005 PA 12          NB. factor for monthly rate of 0.5%
0.086066
10000*0.086066      NB. monthly repayments
860.66
```

From the point of view of a lender the transaction is an income stream of 10000 followed by 12 monthly payments of 860.66 whose IRR is therefore

```
irr_10000, 12#860.66
0.49992          NB. i.e. monthly rate of 0.5%
```

illustrating that IRR matches exactly the fair rate of loan interest paid by equal instalments on a declining balance. 'Matches exactly' is the monadic meaning of the symbol $-:$, so in symbols

```
r -: irr _1,n#(r PA n)
```

relates IRR succinctly to loan repayments rates. IRR is reduced if payments are deferred, whereas if a loan is paid off early using the same proportionate repayments, the IRR increases. These statements are confirmed by

```

    irr _10000, (11#860.664), 0, 860.664  NB. last inflow delayed
0.49378
    irr _10000, 6#2*860.664                NB. inflow rate doubled
0.92988

```

The related term APR takes into account that the equivalent annual rate is not 12×0.005 , corresponding to 6%, but rather this rate compounded over the 12 periods:

```

    1.005^12
1.06168                NB. APR is 6.17%

```

To describe APR, use the conversions to and from p to k :

```

    ptok=.>:@(%&100)
    ktop=.100&*@<:
    apr=.ktop@(ptok@irr ^ <:@#)
    apr _10000, 12#860.66
6.168

```

Note on computational practicalities: irr may fail if the income stream is too long, or if the data generates a pathological situation for p .'s underlying root solver. In such circumstances, alternative numerical methods must be employed such as the Newton Raphson method, which can be expressed in the adverb

```

Newton=.1 : ']-x.%x.D.1'(^:_)("0)

```

Suppose that a loan of 1000 is repaid in 24 instalments of 45.27. The APR is worked out by

```

    ms=(.24$45.27),_1000          NB. money stream reversed
    fn=#.&ms                        NB. define a polynomial
    dtop fn Newton 0.9             NB. guess d=0.9 and apply N-R
0.67447                          NB. monthly rate of return =
0.6745%
    1.0067447^12                  NB. convert to annual rate
1.084                             NB. APR = 8.4%

```

By way of summary here is the proposed basic mathematical text messaging vocabulary:

```

+ - * % ^ ! < > = >: <: ~: +/ */
+: -: *: %: <: >: ^: {.: {.: }. }.: @ &
, # |. #~ #. < > each=.&> p. monad dyad

```

Txt msg OK for sums? LOL ☺

PROFIT

Source Code Management of APL Code

*by Richard Renich, SoftMed Systems, Inc.
Talk given at the 2005 APL2000 Users' Meeting*

Abstract

This document gives a quick introduction to source code management (SCM). It identifies problems specific to using text based SCM tools to manage APL code. It suggests some solutions for those problems.

SCM Overview

Source code management means many things to many people. The common aspect is a system for recording and recalling revisions of code. The basic notion is that you can write some code, commit the code, make some changes, and submit again. You can then return to the code as it was at the previous commit. Each commit (or submission) of code creates new revisions of any files that have changed. The code can be recalled by date or by an identifier assigned during a commit. Most SCM systems have the ability to assign a name to specific revisions of files so that the code can be recalled by name as well.

A good introduction to source code management can be found at Perforce's website www.perforce.com/perforce/technical.html, specifically *High-Level SCM Best Practices* and *Perforce Life-Cycle Modelling*.

SoftMed currently uses component files to manage APL code. All other code is managed by *Perforce*, a commercial SCM tool. We are in the process of converting our APL code management to Perforce.

Typical Use

Once you have loaded your workspace, you would normally sync up with the SCM repository. That brings in changes that others have made since your last

sync. Then you would make code changes and commit those changes. How frequently you sync and commit depends on a number of things, like how many people are working on the code and what quality of code you want committed.

In a broader time-frame, you will make branches of the trunk, which holds all your code, and merge the branches back to the trunk. The trunk is just a branch from which most other branches are taken. The manner in which you work with a branch and the trunk is analogous to the manner in which you work with a workspace and the SCM repository. You periodically bring changes from the trunk into the branch and merge the changes of the branch back to the trunk. Again, the frequency of these activities depends on your environment.

It might seem that having all these copies (branches and revisions) of the code would use a lot of disk space. Virtually all SCM systems store only the latest revision of each file in a branch as a whole file, and a list of lines to be removed and inserted to produce an old revision from the next newer one. Some systems use a similar technique even on binary files. Some systems keep only a pointer to the original for the first revision of each file in a branch. So using a third party SCM tool can provide significant disk savings over a simple home grown system.

SCM Benefits

SCM systems are designed to allow multiple programmers to collaborate on the same set of source code. While they don't always prevent two people from editing the same code at the same time, they identify when this happens before any work is lost. Some SCM tools include utilities to help resolve such conflicts, and most facilitate the use of third party tools for this. Some SCM systems allow each developer to indicate the files he is working on, which can help avoid these problems in the first place.

SCM systems maintain a revision history that permits retrieving the source code as it existed at various points in the past.

SCM systems allow code to branch. That is, there can be multiple sets of source code that were derived from the same source. The branches can then evolve independently. Some changes can be propagated among the branches. A common example of this is a product that has been customized for a specific client. The changes specific to that client stay in that branch. Changes to the trunk can be brought into the client branch. Branches can also be used to experiment with a new design or a new feature.

By learning and using a professional SCM tool you take advantage of the work and expertise of programmers familiar with SCM. You also don't need to maintain as much code to be able to use SCM.

Character Set

APL code uses characters not in the ASCII character set. If this is ignored then some graphic characters will be treated as control characters by the SCM tool. The text that represents functions (`⌈CR` or `⌈VR`) could be converted to ANSI characters. A better solution is to use Unicode characters. Many modern programs, including most SCM systems, work well with Unicode. There is a function, UCS, available from APL Now's web site that can perform the conversion.

Binary Source Data

Variables in the workspace can be a part of the finished product. Even when they are not, if they aid in the development of the product they should be managed. If they do not aid in development and are not part of the finished product, they should be erased. So all variables should be managed. This reasoning holds just as well for workspace dependant systems variables (`⌈IO`, `⌈CT`, `⌈PP`, `⌈LX`, `⌈ELX`, `⌈ALX`).

Prototypes of empty arrays need to be preserved. This must be done correctly even for nested data.

There is a difference between `A←0 ⋄ A←1⊥A,1 0 1 0 0 1 ⋄ A ⌈NAPPEND ⌈1` and `A←2 ⋄ A←1⊥A,1 0 1 0 0 1 ⋄ A ⌈NAPPEND ⌈1`. This makes it important to pay attention to data-types as well as prototypes when working with the text representation of variables. Otherwise there would be a problem if the variable `A` were stored and recalled between the 2nd and 3rd statements.

The solution is to use `⌈ATR`. `⌈ATR` is a suite of functions to convert between APL and Text Representations.

The representations of many APL arrays whose atoms are all character have been designed to allow easy editing in a standard text editor. The representation of a function takes advantage of this. Care must be taken to allow a character matrix to retain its width even when it has no lines.

Two Masters

The APL workspace is one authority for the APL code. It is where changes are made and saved. A local file-system is another authority. It is where changes are received from collaborators. If changes are made in both places, some of those changes could get lost.

The solution is to keep a signature of each object as it was last synchronized with the file-system. If the signature in the WS has changed, it needs to be written to the file-system. If the signature in the file-system has changed, it needs to be read into the workspace. If both have changed, you have misused the system or the system is broken. To avoid having a change in both places, the SCM system is called only when the workspace and the file-system are synchronized.

File System Names

It would not be wise to try to use any of "□△△" as a part of a filename. These map to "+^~". These were chosen to work reasonably well in both DOS and Linux.

APL object names are case sensitive. DOS file names are not. This could lead to problems if you have two names that differ only in case, say `▼WHERE▼` and `where←`. The solution is to append a suffix to the name that is unique to the case of the name. This is always done so that there is no way to have two file-system names that correspond to the same APL name.

For the same reason, variables and functions have the same extension (`.atr`). Grouping objects and storing the group by its own name would cause similar problems.

Many Workspaces to Many Objects

Code reuse is generally considered a good thing. Because of this, the source code for one object may be needed by multiple workspaces. Obviously a workspace includes many objects. This results in a many to many relationship.

To avoid problems with two masters, we already have a list of objects and their signatures. To add the directory name where the object resides in the file-system is not much extra burden. This allows objects to be grouped into libraries.

The signatures must not be managed. They are specific to an instance of the workspace. The list of directory names for the objects does need to be managed. So signatures and directory associations must be kept in separate variables.

Utility Clutter

The utilities required to use SCM should not clutter the workspace under development. The solution is to use a UCMDS file.

Ability to Reconstitute a Workspace

It is a prerequisite to be able to use the SCM tool to check out a directory hierarchy that holds all objects of a workspace. From that, it should be easy to reconstitute any workspace stored there.

Subversion

At <http://subversion.tigris.org>. Subversion is "a compelling replacement for CVS" written by the authors of CVS. It, like CVS, is free software.

Perforce

At www.perforce.com. Perforce is a commercial SCM tool. It is very similar to Subversion. The main differences between these can be traced to the licensing. Subversion is free software, so there is no license per user. Subversion has no need to track users, so it scales well in this respect. Because Perforce uses a per seat license, the number of users is limited. It has a centralized list of users. It takes advantage of this to offer features not available in Subversion.

Perforce has a public interface in C++. We have used that to create an ActiveX object to invoke Perforce. It has the ability to override the class used to represent files. This allows us to solve the problem of two masters by intercepting reads and writes to the file-system. We have not yet completed this code.

Credits

Jeff Pedneau wrote our current APL SCM system in APL. Jeff Pedneau, Joe Hatfield, and I have been working toward using Perforce to manage our APL code.

Index to Advertisers

Dyadic Systems Ltd	2
Vector Back Numbers	5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles to the Vector Working Group via the Editor:

The Editor, Vector
81 South Hill Park
London
NW3 2SS

Tel: +44 (0) 1439-788385
Email: editor@vector.org.uk

Please supply as much material as possible in machine-readable form. Our preferred format is HTML, using simple markup – we apply Vector's style sheet. Source code can be tagged with `<pre>` or `<code>` tags as usual. You can further distinguish language within these tags (which would be important if, for example, you were comparing J and APL expressions) with the class attribute. For example: `<code class="J">MEAN=:+/%#</code>` would be typical. Authors wishing to use MS Word should install a copy of the APL385 Unicode TrueType font and use Unicode APL symbols if their interpreter permits 'copy as Unicode' from the APL session. Otherwise we can convert from Dyalog APL, APL+Win, APLX, and APL2 mappings if necessary.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Address:

Postcode / Country:

Telephone Number:

Email Address:

UK private membership	£20	☐
Overseas private membership	£22	☐
Airmail supplement (not needed for Europe)	£4	☐
UK Corporate membership	£100	☐
Corporate membership overseas	£110	☐
Sustaining membership	£500	☐
Non-voting UK member (student/OAP/unemployed)	£10	☐

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

☐ I authorise you to debit my Visa/Mastercard account

Number: Expiry date: /

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

Signature: _____

Data Protection Act:
The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Officers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2005/2006 Committee

Chairman	Paul Grosvenor 01293-562700 paul@optima-systems.co.uk	Optima House, Mill Court Spindle Way, CRAWLEY West Sussex, RH10 1TT
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stephen Taylor sjt@vector.org.uk	81 South Hill Park LONDON NW3 2SS
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Ian Clark earthspot2000@hotmail.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stephen Taylor	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852) Bob Hoekstra (01483-771028), Marc Griffiths, Ian Clark	

Typeset by APL-385 with MS Word for Windows
Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
Brook House, Gilling East
YORK YO62 4JJ
Tel: 01439-788413
Web: www.causeway.co.uk

Dyalog Ltd
Grove House, Lutyns Close,
Chineham Court, BASINGSTOKE,
Hants. RG24 8AG
Tel: 01256-338461
Email: sales@dyalog.com
Web: www.dyalog.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk, Denmark
Tel: +45 70 26 13 26
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield, E Sussex TN22 5AA
Tel: 01825-768050
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Kx Systems
555 Bryant St., No. 375
Palo Alto CA 94301 USA
Tel: +1 866 kdb fast
Email (general enquiries): info@kx.com
In Europe: Simon Garland, simon@kx.com

Optima Systems Ltd
Optima House, Mill Court, Spindle Way, Crawley,
West Sussex, RH10 1TT
Tel: 01293 562700
Email: paul@optima-systems.co.uk

APL Italiana
via Monferrato 1, 144 Milano, Italy.
Email: stf@apl.it

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850, USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Soliton Inc
44 Victoria Street, Suite 2100
Toronto, Ontario M5C 1Y2, Canada
Tel: +1 (416) 364 9355
Email: sales@soliton.com
Web: www.soliton.com

First Derivatives plc
First Derivatives House
Kilmorey Business Park, Kilmorey Street,
Newry, Co. Down, N. Ireland BT34 2DH.
Tel: 028 3025 2242
Email: enquiries@firstderivatives.com

The Carlisle Group
544 Jefferson Avenue, Scranton PA, 18510
USA.
Tel: +1 (570) 963-2036
Web: www.carlislegroup.com